

MINECRAFT: BEDROCK EDITION · CREATOR GUIDE

The Complete Guide to Advanced Minecraft: Bedrock Edition Add-Ons

Building custom mobs, items and blocks from an empty folder to a finished .mcaddon

Custom mobs

Custom items

Custom blocks

Models & textures

Behaviour JSON

Molang

Script API

Packaging

17 sections

Bedrock 1.21.x file formats

Windows workflow

Blockbench · VS Code · bridge. v2

Contents

1 Introduction

- 1.1 Who this guide is for
 - 1.2 What "add-on" actually means
 - 1.3 How this guide is organised
-

2 Apps

- 2.1 Choosing the right tools
 - 2.2 Download guide
 - 2.3 Navigation
 - 2.4 Other information
-

3 First steps of creating an addon

- 3.1 Planning the add-on
 - 3.2 Choosing your namespace
 - 3.3 Creating the pack skeletons
 - 3.4 Adding a pack icon
 - 3.5 Adding a language file
 - 3.6 Verifying the empty packs load
 - 3.7 Setting up your editor
 - 3.8 Extra things and tips
-

4 Creating mobs textures and body

- 4.1 Understanding Bedrock geometry
 - 4.2 Setting up the Blockbench project
 - 4.3 Blocking out the body
 - 4.4 UV mapping
 - 4.5 Creating the texture
 - 4.6 Exporting the model
 - 4.7 Other information for mobs
 - 4.8 Extra tips for mobs
-

5 Creating item textures and layout

- 5.1 How items differ from mobs

- 5.2 Item texture rules
 - 5.3 Drawing an item icon
 - 5.4 Registering the texture: `item_texture.json`
 - 5.5 Item names: the language file
 - 5.6 Attachables: items with real 3D models
 - 5.7 Other information for items
 - 5.8 Extra tips for items
-

6 Creating block textures shape/geo

- 6.1 How blocks differ from mobs and items
 - 6.2 Block texture rules
 - 6.3 Drawing block textures
 - 6.4 Registering block textures: `terrain_texture.json`
 - 6.5 Assigning textures to faces
 - 6.6 Custom block geometry
 - 6.7 The block's inventory icon
 - 6.8 Block names: the language file
 - 6.9 Other information for blocks
 - 6.10 Extra tips for blocks
-

7 Common mistakes byte one

- 7.1 Setup and manifest mistakes
 - 7.2 Naming and identifier mistakes
 - 7.3 Blockbench and geometry mistakes
 - 7.4 UV and texture-mapping mistakes
 - 7.5 Texture image mistakes
 - 7.6 Item texture mistakes
 - 7.7 Block texture mistakes
 - 7.8 Language file mistakes
 - 7.9 Workflow mistakes
-

8 Creating folders

- 8.1 The behaviour pack structure
- 8.2 The resource pack structure
- 8.3 `texts/languages.json`
- 8.4 Which folders do you actually need?
- 8.5 Naming conventions inside folders

9 Importing the textures

- 9.1 What has to be imported
 - 9.2 Exporting from Blockbench into the pack
 - 9.3 Checking the exported geometry
 - 9.4 Writing the client entity file
 - 9.5 Writing the render controller
 - 9.6 Importing sounds
 - 9.7 Verifying the import
 - 9.8 Re-importing after changes
-

10 Coding

- 10.1 Where to put the code
 - 10.2 JSON in ninety seconds
 - 10.3 Molang: Minecraft's little expression language
 - 10.4 Coding mobs: the server entity file
 - 10.5 Behaviours (mob AI)
 - 10.6 Component groups and events
 - 10.7 Filters
 - 10.8 Spawning
 - 10.9 Loot tables
 - 10.10 Trade tables
 - 10.11 Item properties
 - 10.12 Coding blocks
 - 10.13 Recipes
 - 10.14 Animations
 - 10.15 Particles
 - 10.16 Functions and commands
 - 10.17 The Script API
 - 10.18 Debugging your code
-

11 Common mistakes byte two

- 11.1 Folder mistakes
- 11.2 Identifier and linking mistakes
- 11.3 Entity behaviour mistakes
- 11.4 Spawn rule mistakes

- 11.5 Loot table mistakes
 - 11.6 Item mistakes
 - 11.7 Block mistakes
 - 11.8 Recipe mistakes
 - 11.9 Animation mistakes
 - 11.10 Script API mistakes
 - 11.11 General coding mistakes
-

12 Creating the manifests

- 12.1 What a manifest does
 - 12.2 The complete behaviour pack manifest
 - 12.3 The complete resource pack manifest
 - 12.4 Linking the two packs
 - 12.5 Manifests for scripts
 - 12.6 Manifests for world templates
 - 12.7 Manifests for skin packs
 - 12.8 Getting and managing UUIDs
 - 12.9 Manifest checklist
-

13 Finalising the addon

- 13.1 Final testing pass
 - 13.2 Cleaning up
 - 13.3 Folder structure for packaging
 - 13.4 The three file formats
 - 13.5 Making a `.mcpack` (single pack)
 - 13.6 Making a `.mcaddon` (the full add-on)
 - 13.7 Exporting from bridge. v2
 - 13.8 Automating packaging
 - 13.9 Documentation to ship alongside
 - 13.10 Sharing
 - 13.11 Version control and archiving
 - 13.12 Finalising checklist
-

14 Importing addon to Minecraft

- 14.1 Windows
- 14.2 Android
- 14.3 iOS and iPadOS

- 14.4 Xbox, PlayStation and Nintendo Switch
 - 14.5 Activating the add-on in a world
 - 14.6 Confirming it worked
 - 14.7 If the import fails
 - 14.8 Updating an installed add-on
 - 14.9 Uninstalling
-

15 Common mistakes byte three

- 15.1 Manifest mistakes
 - 15.2 Packaging mistakes
 - 15.3 Import mistakes
 - 15.4 Post-release mistakes
 - 15.5 Compatibility mistakes
-

16 Conclusion

17 Extra section: ideas

- 17.1 Beginner ideas — one system each
- 17.2 Intermediate ideas — several systems together
- 17.3 Advanced ideas — full projects
- 17.4 Themed ideas — for when you want a subject, not a system
- 17.5 Ideas for practising specific skills
- 17.6 Choosing what to make

1 Introduction

This guide teaches you how to build an **advanced Minecraft: Bedrock Edition add-on** from an empty folder to a finished `.mcaddon` file that you can double-click and play. "Advanced" here means the real thing: brand-new mobs with their own models, textures, animations, AI, loot and spawn rules; brand-new items with custom behaviour; brand-new blocks with custom shapes and states; recipes, particles, sounds, translations, and optional scripting.

Nothing in this guide uses a "mob maker" website or a drag-and-drop app that hides the files from you. Those tools cannot produce advanced add-ons, because an advanced add-on is a folder full of JSON files that you control. Every tool recommended here shows you the files and lets you edit them.

1.1 Who this guide is for

You do not need to know how to program. You do need to be willing to read and edit text files that are full of curly brackets. If you have ever changed a setting in a config file, you already have the skill required. The Coding section explains JSON from the ground up before it asks you to write any.

You need a **Windows PC** (Windows 10 or 11) running **Minecraft for Windows** (the Bedrock version bought from the Microsoft Store). Add-ons can be made on other platforms, but Windows is the only platform where the game's own add-on folders are directly editable, which makes testing about ten times faster. If you play on Xbox, PlayStation, Switch, iOS or Android, you can still make the add-on on a PC and import the finished `.mcaddon` onto those devices.

1.2 What "add-on" actually means

A Bedrock add-on is made of up to two packs:

- A **behaviour pack (BP)** — the "server" side. It contains the rules: what a mob does, how much health it has, what an item does when you eat it, what a block drops when you break it, what recipes exist, what spawns where.
- A **resource pack (RP)** — the "client" side. It contains the appearance: models, textures, animations, sounds, particles, and the text that appears on screen.

Neither pack can see inside the other. They are linked only by **identifiers** — strings such as `mypack:frost_wraith` — that appear in both packs and must match *exactly*. Almost every problem a new add-on maker has comes down to an identifier that does not match, or a file that is in the wrong folder. Keep that sentence in mind; it will save you hours.

1.3 How this guide is organised

The guide runs in the order you will actually work:

1. Install and learn the apps.
2. Set up the empty pack skeletons.
3. Make the art — mobs, then items, then blocks.
4. **Common mistakes byte one** — everything that goes wrong during art.
5. Build the folder structure and import the art properly.
6. **Coding** — the long one, where the add-on becomes real.
7. **Common mistakes byte two** — everything that goes wrong during coding.
8. Write the manifests, finalise, and import into Minecraft.
9. **Common mistakes byte three**, conclusion, and a pile of add-on ideas.

The three "common mistakes bytes" are checkpoints. When something is broken, jump to the byte that covers the stage you are in and read it top to bottom — the answer is usually there.

① Version note

Minecraft Bedrock changes its file formats regularly. Every `format_version` shown in this guide is a real, working value at the time of writing (game version 1.21.x). Formats are **not** silently updated by the game: an old `format_version` keeps behaving the old way, which is deliberate and safe. If a component in this guide does not work, the first thing to check is whether your `format_version` is high enough to include it. The official documentation at learn.microsoft.com/minecraft/creator lists the version each component was added in.

◆ The single most useful habit

Test after **every** small change. Add one component, reload, look at it in game. If you write two hundred lines and then test, you will spend an hour finding which line broke it. If you write four lines and test, you already know.

2 Apps

2.1 Choosing the right tools

There is no single app that makes an advanced add-on. Advanced add-ons need four different jobs done, and the best tool for each job is a different program:

Job	Best tool	Cost	Where
3D models, texture painting, animation	Blockbench	Free	Microsoft Store / blockbench.net
Writing and validating the JSON files	Visual Studio Code + Bedrock extensions	Free	Microsoft Store / code.visualstudio.com
Guided add-on project management	bridge. v2 (optional but excellent)	Free	bridge-core.app / Microsoft Store
Pixel-art texture editing	Paint.NET (or GIMP, free)	~£8 / free	Microsoft Store / getpaint.net
Zippping the finished pack	NanaZip (or Windows' built-in zip)	Free	Microsoft Store
Testing	Minecraft for Windows + Minecraft Preview	Owned	Microsoft Store

A note on what was deliberately **not** chosen. Tools such as Tynker, MCreator's Bedrock mode, "Add-Ons Maker for Minecraft PE", and the various phone apps are block-based or form-based editors. They generate a small, fixed set of things — a recoloured cow, a sword with different damage — and they hide the JSON. You cannot make a new mob with an original model, a custom AI goal set, its own loot table and its own spawn rules in them, and when they do let you export, the exported files are usually written to an old `format_version` you then have to fix by hand. Learning the real files is genuinely faster than fighting a tool that cannot reach where you are going.

★ The minimum viable setup

If you want to install as little as possible: **Blockbench** + **Visual Studio Code** + **Minecraft for Windows**. That trio can make any add-on in this guide. Everything else on the list is convenience.

2.2 Download guide

This sub-section covers, for each app, how to install it and exactly which prompts, permissions and options you will see. Install them in the order given — Blockbench first, because it is the one you will open first.

2.2.1 Blockbench

What it is: a free, open-source 3D model editor built specifically for Minecraft. It edits Bedrock geometry (`.geo.json`), paints textures directly onto the model, and creates Bedrock animations (`.animation.json`). It is made by JannisX11 and is the tool almost every Bedrock add-on creator uses.

Installing from the Microsoft Store

1. Open the **Microsoft Store** app and search for `Blockbench`.
2. The correct listing is published by **JannisX11**. Check the publisher name — there are unrelated apps with similar names.
3. Click **Get**, then **Install**. It is free and there is no trial or paid tier.
4. When it finishes, click **Open**.

Installing from the website instead (equivalent, and updates slightly sooner):

1. Go to `blockbench.net` and click **Download**.
2. Choose **Windows Installer (.exe)**. Take the installer, not the portable `.zip`, unless you specifically want a portable copy.
3. Windows SmartScreen may show "*Windows protected your PC*". This is normal for any installer that is not signed by a large company. Click **More info**, then **Run anyway**. Only do this because you downloaded it from `blockbench.net` yourself — never click through SmartScreen for a file you got from a link someone sent you.
4. In the installer, choose **Install for me only** unless you are the PC administrator and want it available to all users. "For me only" avoids needing an admin password.
5. Leave the default install location. Tick **Create a desktop shortcut** if you want one.

Prompts and permissions on first launch

- **Theme / colour scheme picker** — cosmetic only. Pick whatever you like; you can change it later under `File > Preferences > Themes`.
- "**Allow Blockbench to check for updates?**" — say **Yes**. Blockbench updates often and formats change with the game.
- **Anonymous usage statistics** — optional. Saying no changes nothing about how the app works.
- **Windows Firewall prompt** ("Allow Blockbench to communicate on these networks") — this appears only if you use the plugin store or the "Connect to Minecraft" feature. **Private networks: tick. Public networks: leave unticked.** Blockbench does not need to accept connections from the internet.

- **Microsoft Store version only:** the Store version is sandboxed, so the first time you save a file it will show a normal Windows "Save As" dialog. Once you have saved into a folder, it remembers access to that folder. If it ever refuses to write somewhere, save to your Documents folder instead — that always works.
- **Plugin install prompts** — when you install a plugin (see below), Blockbench asks "*Do you want to install this plugin? Plugins can access your files.*" Only accept plugins from the official in-app plugin browser.

Plugins worth installing immediately (`File > Plugins > Available` tab):

- **Minecraft Title Generator** — not essential, skip if you like.
- **Animation Utils** — extra tools for editing keyframes. Very useful later.
- **Blockbench Bedrock Entity Wizard / "Molang Editor"** (names vary by version) — adds syntax help when typing Molang expressions into animation fields.

2.2.2 Visual Studio Code

What it is: a free text editor from Microsoft. With the right extensions it understands Bedrock's JSON files, autocompletes component names, and underlines mistakes in red *before* you load the pack into the game. This is the single biggest time-saver in the whole workflow.

Installing

1. Microsoft Store → search `Visual Studio Code` → publisher **Microsoft** → **Get** → **Install**. (Alternatively `code.visualstudio.com` → **Download for Windows**.)
2. If you use the website installer, the setup wizard shows a **Select Additional Tasks** page. Tick these:
 1. ☒ **Add "Open with Code" action to Windows Explorer file context menu**
 2. ☒ **Add "Open with Code" action to Windows Explorer directory context menu**
 3. ☒ **Add to PATH**

The two Explorer options let you right-click your add-on folder and open the whole project in one click. Strongly recommended.

- ☐ Leave "*Register Code as an editor for supported file types*" unticked unless you want VS Code to become your default for every `.txt` and `.json` on the PC.
- On first launch you get a **Welcome** page and a theme picker. Cosmetic.
- "**Do you trust the authors of the files in this folder?**" — this appears the first time you open any folder. For your own add-on folder, click **Yes, I trust the authors**. Trusting a folder allows extensions to run inside it. Only ever trust folders whose contents you created or downloaded from a source you trust.
- "**Would you like to install the recommended extensions?**" — safe to say yes for the ones below; you can also install them manually.

Extensions to install — click the Extensions icon in the left bar (four squares) or press `Ctrl+Shift+X`, then search each name:

- **Blockception's Minecraft Bedrock Development** (publisher: *Blockception*). The important one. It provides JSON schemas for every Bedrock file type, so you get autocomplete for component names, red underlines for typos, and hover documentation. Install this even if you install nothing else.
- **Minecraft Bedrock Definition Generator** (publisher: *destruc7iOn*). Adds "generate a definition file" commands. Handy, optional.
- **JSON5 syntax** or **Comment tagged templates** — optional, ignore for now.

After installing Blockception's extension, reload VS Code when prompted. It may ask "**Enable Minecraft schemas for this workspace?**" — say yes.

2.2.3 bridge. v2

What it is: a dedicated Bedrock add-on IDE. It knows about packs as a concept: it has file templates ("create a new entity"), a visual component browser, a built-in `.mcaddon` exporter, and a preset system that generates whole working entities in a few clicks. It is free and open source.

It is genuinely useful, especially for beginners, but it is listed as **optional** because it occasionally lags behind the newest format versions, and because you should understand the raw files anyway. Many creators use bridge. to scaffold files and VS Code to edit them.

Installing — browser version (recommended)

1. Open **Microsoft Edge** or **Google Chrome**. bridge. v2's web version needs the File System Access API, which Firefox and Safari do not support.
2. Go to `editor.bridge-core.app`.
3. Click **Install** in the address bar (a small monitor-with-arrow icon) to install it as an app. This gives it its own window and a Start-menu entry. Optional but nicer.
4. It asks you to create or open a project. When you choose a folder, the browser shows:
 1. "Let site edit files? bridge. wants to view and edit files in [folder]" → click **Edit files**. This is the permission that lets it write your add-on. Without it, bridge. can do nothing.
 2. You may then get "View files? bridge. wants to view files" → **View files**.
 3. The permission is re-requested each new session by design. That is the browser protecting you, not a bug.
5. Optionally, bridge. offers to store projects in the browser's own virtual file system. **Do not use this** for real work — use a real folder on disk so you can back it up and use Git.

Installing — Microsoft Store version

1. Microsoft Store → search `bridge.` → publisher **bridge. (bridge-core)**. Free.
2. The desktop version has direct file access, so there is no per-session permission prompt.

First-run options in bridge.

- **Choose a colour theme** — cosmetic.
- "Enable telemetry?" — optional, no effect on function.

- **Project creation dialog** — asks for: project name, author name, target Minecraft version, and tick-boxes for **Behaviour pack**, **Resource pack**, **Skin pack**, **World template**, and **Use bridge's scripting API types**. For a normal add-on tick **Behaviour pack** and **Resource pack** only.
- **"Where should bridge. save projects?"** — point this at a normal folder you control, e.g. `Documents\MinecraftAddons`.

2.2.4 Paint.NET (or GIMP)

What it is: an image editor for making and touching up textures. Blockbench can paint textures itself, and for many add-ons that is enough — but a real image editor is better for drawing item icons, working with layers, and precise colour work.

Paint.NET

1. Microsoft Store → search `Paint.NET` → publisher **dotPDN LLC**. It costs a small amount (around £8) in the Store; the Store purchase supports the developer and gives automatic updates.
2. The identical app is **free** from `getpaint.net` — click **Download**, then choose the **classic desktop installer**. This is not piracy; the developer offers both deliberately.
3. Installer options: choose **Express** unless you want to change the install path. It may offer to install the **.NET Desktop Runtime** if missing — allow it.
4. First launch may ask about **automatic updates** (say yes) and **crash reporting** (optional).

GIMP (completely free alternative)

1. Microsoft Store → search `GIMP` → publisher **The GIMP Team**. Or `gimp.org` → Download.
2. Installer: choose **Install for all users** (needs admin) or **Install for me only**. Either works.
3. GIMP is more powerful and much less friendly. If you have never used an image editor, start with Paint.NET.

Critical setting for both: textures must be saved as **PNG with transparency**, at their exact pixel size, with **no interpolation/smoothing**. In Paint.NET, when you resize an image use `Image > Resize` and set **Resampling: Nearest Neighbor** — anything else blurs pixel art into mush.

2.2.5 NanaZip (or 7-Zip)

What it is: an archive tool used at the very end to zip your packs into a `.mcaddon`. Windows' built-in "Send to → Compressed (zipped) folder" also works, so this is optional — but a proper archiver makes it much easier to check *inside* the zip that the folder structure is right, which is the single most common packaging mistake.

1. Microsoft Store → search `NanaZip` → publisher **M2Team**. Free and open source (it is a modern fork of 7-Zip).
2. On first launch it asks to **integrate into the Windows Explorer context menu** — say **yes**, that is the whole point.
3. Windows 11 note: NanaZip appears directly in the new right-click menu. 7-Zip requires "Show more options" first.

Alternative: **7-Zip** from 7-zip.org. Same function, older interface, no Store listing.

2.2.6 Minecraft for Windows and Minecraft Preview

What it is: the game itself, plus its beta channel. You need Minecraft for Windows specifically — the "Bedrock" version. Minecraft: Java Edition **cannot** load add-ons, and nothing in this guide applies to it.

1. Microsoft Store → search [Minecraft for Windows](#). If you already own Minecraft: Java & Bedrock Edition, Bedrock is included.
2. Install and sign in with your Microsoft account.
3. **Minecraft Preview** is a separate free install for anyone who owns the game — search [Minecraft Preview](#) in the Store. It runs the next version early. Install it: it is the safest place to test add-ons because it keeps its own separate worlds, so a broken add-on cannot damage your real saves.
4. First launch asks to sign in to a Microsoft account and may ask about **realms / marketplace notifications** — decline freely, none of it affects add-ons.

The one setting you must change: in Minecraft, go to [Settings > Creator](#) and turn on:

- ☒ **Enable Content Log GUI** — shows errors in-game as a scrolling overlay.
- ☒ **Content Log Level: Verbose** (or at least "Warning").
- ☒ **Enable Content Log File** — writes the log to disk so you can read it after the fact.

Without the content log you are debugging blind. With it, Minecraft tells you the exact file and line number that is broken. This is the most important five seconds of setup in the entire guide.

2.2.7 Optional: Node.js and Regolith

Only for large projects. **Regolith** is a build system for Bedrock add-ons: you keep clean source files, and it runs "filters" over them (auto-generating repetitive JSON, bundling scripts, copying to the game folder on save). Install **Node.js LTS** from nodejs.org and **Go** if you want Regolith; see bedrock-oss.github.io/regolith. Skip this on your first add-on — it solves a problem you do not have yet.

2.3 Navigation

This sub-section is a map of each app: where things are, how to get between them, and what the settings actually do.

2.3.1 Navigating Blockbench

Starting a project. On launch you get the **Start Screen**. The panel of project types is the important part. For add-ons you will use:

- **Bedrock Model** — for mobs and any custom-shaped block. Produces a [.geo.json](#). (In some versions this is listed as "Minecraft Entity"; if asked for a format, choose Bedrock and not "Java Block/Item".)

- **Bedrock Block** — a Bedrock model constrained to a single block's bounds. Use for custom block shapes.
- **Generic Model** — free-form, avoid unless you know why.
- **Do not use Java Block/Item or Optifine Entity** — those export the wrong format entirely.

Recent projects appear on the right; the **Format** of a project can be changed later via **File > Convert Project**.

The main window layout. From top-left, clockwise:

- **Menu bar** — **File**, **Edit**, **Transform**, **Filter**, **View**, **Animation**, **Tools**, **Help**.
- **Toolbar (left edge)** — the mode tools: **Move**, **Resize**, **Rotate**, **Vertex Snap**, and in Paint mode the **Brush**, **Eraser**, **Paint Bucket**, **Colour Picker**, **Draw Shape**, **Gradient**.
- **Mode tabs (top-centre)** — **Edit**, **Paint**, **Display**, **Animate**. These are the four workspaces:
 - **Edit** builds the model (cubes, bones, positions, UV).
 - **Paint** paints the texture directly on the 3D model or on the flat texture.
 - **Display** sets how the model appears in hand / on head / in a GUI (mostly for Java; largely unused for Bedrock entities).
 - **Animate** creates animations with a timeline.
- **Viewport (centre)** — the 3D view. Left-drag to orbit, scroll to zoom, middle-drag (or **Shift** + drag) to pan. Press **1**–**6** for front/back/left/right/top/bottom orthographic views, **0** for perspective.
- **Outliner (right, top)** — the tree of **bones** (called "groups") and **cubes**. This tree is the model's structure and its names become the bone names your animations refer to. The small icons on each row toggle visibility, lock, and export.
- **Element panel (right)** — numeric fields for the selected cube: **From** / **To** (the two opposite corners in model units), **Size**, **Pivot**, **Rotation**, and **Inflate**.
- **UV panel (right, lower)** — where the cube's faces sit on the texture image. In Bedrock's "box UV" mode this is one number pair per cube; in "per-face UV" mode each face is placed separately.
- **Textures panel (right, bottom)** — the list of texture images loaded into this project. Right-click a texture for **Save**, **Save As**, **Change file**, **Resize**, **Properties**.
- **Status bar (bottom)** — shows the current format, selected element, and a progress area.

Key menus and what they do

- **File > New** — new project (asks for format again).
- **File > Open Model** — opens **.bbmodel** (Blockbench's own working file), **.geo.json**, **.json**.
- **File > Save** — saves the **.bbmodel**. **This is not the file Minecraft reads**. Keep it; it is your editable source.
- **File > Export > Bedrock Geometry** — **this produces the .geo.json that goes in your resource pack**. Do this every time you change the model.
- **File > Export > Bedrock Animation** — produces the **.animation.json**.

- **File > Project** (or the wrench icon) — **Project Settings**: model **Identifier** (`geometry.something`), **Texture Width/Height**, **Visible Bounds** (the box the game uses to decide if the mob is on screen — too small and the mob vanishes when you look slightly away), and **Format**.
- **File > Preferences > Settings** — global settings. Useful ones: **Snapping** (how far cubes jump when dragged — set to 1 for pixel-perfect, or 0.5/0.25 for fine work), **Restrict cube sizes**, **Autosave**, **Default resolution**.
- **File > Preferences > Keymap** — rebind shortcuts.
- **Edit > Add Cube** (`Ctrl+N` ... varies) — new cube. **Edit > Add Group** — new bone.
- **Transform** menu — **Rotate**, **Flip**, **Center**, **Scale**, and **Element Origin** tools.
- **Filter** menu — bulk operations (rename, scale everything).
- **View > Screenshot Model** — quick render, useful for a pack icon.
- **Tools > Validate** / the plugin-provided validators — catch UV overlaps and out-of-bounds cubes.
- **Help > Developer Tools** — a browser console; occasionally shows the real reason an export failed.

Paint mode specifics. Select a texture in the Textures panel first — painting does nothing without one. The **Texture Editor** (a flat 2D view of the texture) can be opened by double-clicking the texture, and you can paint in either the 2D or 3D view; they are the same image. Under the brush tool's settings: **Size**, **Opacity**, **Softness** (keep at 0 for crisp pixels), **Blend mode**, and **Mirror painting**.

Animate mode specifics. Covered fully in the Coding section, but for navigation: the **Animations panel** (left) lists animations; the + button creates one and asks for a name such as `animation.frost_wraith.walk`. The **Timeline** at the bottom holds **keyframes**. Selecting a bone in the Outliner then changing Rotation/Position/Scale creates a keyframe at the playhead. Right-click a keyframe for **Interpolation** (Linear / Catmull-Rom / Step) and for **Molang** expression entry. The **Loop** button on the animations panel sets `loop: true`. **Animation length** is the number in the top-right of the timeline.

2.3.2 Navigating Visual Studio Code

Opening your project. Always use **File > Open Folder** and choose the folder containing *both* your BP and RP. Opening single files loses you the search, the file tree, and half the extension features. (With the context-menu option enabled, right-click the folder in Explorer → **Open with Code**.)

The layout

- **Activity bar (far left)** — five icons:
 - **Explorer** (`Ctrl+Shift+E`) — the file tree of your project.
 - **Search** (`Ctrl+Shift+F`) — search across every file. Essential for finding every place an identifier is used.
 - **Source Control** (`Ctrl+Shift+G`) — Git, if you use it.
 - **Run and Debug** — not used for add-ons.
 - **Extensions** (`Ctrl+Shift+X`) — install/manage extensions.
- **Side bar** — contents of whichever activity-bar icon is selected.

- **Editor area** — your open files as tabs. Drag a tab to the right edge to split the view: hugely useful for keeping the BP entity file and the RP entity file side by side.
- **Panel (bottom, `Ctrl+``)** — **Problems, Output, Terminal**. The **Problems** tab lists every JSON error the extension has found, across all files. Check it before every test.
- **Status bar (bottom)** — line/column, indentation setting, encoding, and the **language mode** (should say `JSON` or `JSON with Comments`).

Settings that matter (`Ctrl+,` opens Settings; type the name in the search box):

- **Files: Auto Save** → set to `afterDelay`. Minecraft reads files from disk, so an unsaved file is an untested file. This one setting prevents a whole class of "my change did nothing" confusion.
- **Editor: Tab Size** → `2`. Bedrock JSON conventionally uses two spaces.
- **Editor: Format On Save** → on. Keeps your JSON tidy automatically.
- **Editor: Word Wrap** → `on` if you dislike horizontal scrolling.
- **Files: Exclude** → add patterns to hide clutter from the tree.
- **Explorer: Compact Folders** → turn **off**. Bedrock has many single-child folders (`entity/`, `models/entity/`) and compacting them makes the tree confusing.

Commands you will use constantly

- `Ctrl+P` — quick-open a file by typing part of its name.
- `Ctrl+Shift+P` — the Command Palette; type `Format Document`, `Change Language Mode`, etc.
- `Ctrl+Space` — force autocomplete. Inside a Bedrock JSON file with Blockception's extension installed, this lists valid component names at your cursor. This is how you discover components without memorising them.
- `Ctrl+Shift+F` — project-wide search. Search your identifier (`mypack:frost_wraith`) to see every file that references it.
- `Ctrl+/` — comment a line. **Careful:** strict JSON does not allow comments. `.json` files in Bedrock *do* tolerate `//` comments in most cases, but not all parsers agree — avoid comments in files you ship.
- `Alt+Shift+F` — format the document (fixes indentation).
- Hover over a component name — Blockception's extension shows what it does and which format version added it.

2.3.3 Navigating bridge. v2

- **Project sidebar (left)** — a tree of your packs. bridge. groups them logically ("Entities", "Items", "Blocks", "Recipes") rather than showing raw folders, which makes it much easier to find things early on. The **toggle** at the top switches between this friendly view and the real folder view.
- **Top bar** — the project name (click to switch projects), a search box, and the **Compiler / Export** controls.
- **+ / New File button** — opens the **file type picker**: Entity, Item, Block, Recipe, Loot Table, Trade Table, Animation, Animation Controller, Render Controller, Particle, Function, Spawn Rule, Feature, Biome, Script, and more. Choosing one runs a small wizard that asks for the identifier and then writes a *correct, current-format* stub file. This is the single best reason to use bridge.

- **Component browser (right, when an entity file is open)** — a searchable list of every `minecraft:` component with a description. Click to insert. This turns "what is that component called again" into a two-second lookup.
- **Presets** (`File > New > Preset` or the preset button) — generates whole working systems: a rideable entity, a projectile, a fully-configured block. Read the generated code; it is a good teacher.
- **Tabs** — bridge. can open a file in a **Text** view or a **Tree/Visual** view. The tree view is safer for beginners (it cannot produce invalid JSON); the text view is faster once you know the format. Toggle at the top-right of the editor.
- **Settings** (gear icon, bottom-left) — the important ones:
 - **General > Target Minecraft version** — controls which schemas and format versions bridge. suggests. Set it to the version you are building for.
 - **Editor > Formatting** — indentation, bracket style.
 - **Projects > Output folder / "com.mojang" path** — lets bridge. write directly into the game's `development_*_packs` folders so you can test without copying files by hand. Set this up; it removes a whole manual step.
 - **Extensions** — bridge.'s own plugin store (extra presets, custom components, script bundling).
- **Export** → `.mcaddon` — packages both packs into a distributable file, correctly structured, in one click.

2.3.4 Navigating Paint.NET

- **Tools palette (top-left)** — Rectangle Select, Lasso, Magic Wand, **Pencil** (use this, not Paintbrush — Pencil draws single hard pixels), Paint Bucket, Colour Picker, Eraser, Zoom.
- **Colours window (bottom-left)** — primary/secondary colour, with a hex box. Type hex values directly for consistent palettes.
- **Layers window (bottom-right)** — layers list, with **Add**, **Duplicate**, **Merge Layer Down**, and **Layer Properties** (blend mode, opacity). Flatten before exporting PNG.
- **History window (top-right)** — every action, clickable to undo far back.
- `Image > Resize` — always set **Resampling** to "Nearest Neighbor" for pixel art.
- `Image > Canvas Size` — change the image dimensions without scaling the art (use this to grow a texture sheet).
- `View > Pixel Grid` — turn on. Makes 16×16 work far easier.
- `File > Save As` → choose **PNG**. If it asks about flattening, click **Flatten**. If a "PNG bit depth" dialog appears, choose **Auto-detect** or **32-bit** to preserve transparency.

2.3.5 Navigating Minecraft (for testing)

- **Settings > Creator** — the content log switches described above, plus **Enable GameTest Framework** and **Enable debugging**, used for scripting.
- **Settings > Profile** — where installed resource packs and behaviour packs live *globally*.
- **Create New World > Behaviour Packs / Resource Packs** — per-world pack selection. This is where you activate your add-on. Packs in `development_behavior_packs` appear here under **My Packs**.

- **Create New World > Advanced > Experiments** — toggles for beta features. Some components require an experiment enabled; the component's documentation says which. Note that **enabling an experiment permanently marks the world**.
- **Settings > Storage** — lets you delete individual packs and worlds. Useful when a broken pack refuses to reload.
- In-game commands you will use: `/reload` (re-reads behaviour pack functions and some data — **not** models or textures), `/summon mypack:frost_wraith`, `/give @s mypack:frost_shard`, `/setblock ~ ~ ~ mypack:frost_lantern`, `/gamerule sendcommandfeedback true`.

2.4 Other information

Where Minecraft keeps its files. Everything lives under one folder. Paste this into File Explorer's address bar:

TEXT

```
%localappdata%\Packages\Microsoft.MinecraftUWP_8wekyb3d8bbwe\LocalState\games\com.mojang
```

For **Minecraft Preview** it is:

TEXT

```
%localappdata%\Packages\Microsoft.MinecraftWindowsBeta_8wekyb3d8bbwe\LocalState\games\com.mojang
```

Inside you will find:

Folder	What it is
<code>development_behavior_packs</code>	Put your BP here while developing. Read fresh each time the game loads a world.
<code>development_resource_packs</code>	Put your RP here while developing.
<code>behavior_packs</code>	Installed (imported) behaviour packs. Do not edit by hand.
<code>resource_packs</code>	Installed resource packs. Do not edit by hand.
<code>minecraftWorlds</code>	Your saved worlds. Each world has its own <code>behavior_packs</code> / <code>resource_packs</code> copies.
<code>skin_packs</code>	Skin packs.

The `development_` folders are the point of the whole workflow: files there are read directly from disk, so you can edit, leave the world, re-enter, and see your change — no re-importing.

★ Work directly in the development folders

The fastest setup is to make your project folder be the folder inside `development_behavior_packs`, and open that in VS Code. Some people prefer to keep sources elsewhere and copy on save (which is what Regolith automates). Either is fine — just never edit in two places at once.

Backups and version control. JSON files are tiny and text-based, which makes them perfect for **Git**. Install Git, run `git init` in your project folder, and commit after every working state. When you break something at 1 a.m. you can go back to the last version that worked. If Git is too much, at minimum copy the whole folder to a dated zip before any big change.

Naming rules you must follow everywhere. Minecraft is picky:

- Use **lowercase letters, numbers and underscores only** in file names, folder names and identifiers. No spaces, no capitals, no hyphens in identifiers.
- Identifiers take the form `namespace:name`, e.g. `frostpack:frost_wraith`. Choose one namespace for your whole add-on and never change it.
- **Never use the `minecraft:` namespace** for your own content. It is reserved, and using it can overwrite vanilla content or be rejected outright.
- File extensions are meaningful: `.geo.json`, `.animation.json`, `.entity.json`, `.render_controllers.json`, `.rc.json`. Bedrock mostly finds files by scanning folders, but sticking to conventions keeps tools working.

Official documentation. Bookmark these:

- `learn.microsoft.com/minecraft/creator` — the official Bedrock creator docs, including the full component list, schemas and samples.
- `bedrock.dev` — a community-maintained mirror of the auto-generated docs, organised by version. Often faster to search. Its "Entity Documentation" page is the definitive component list.
- `wiki.bedrock.dev` — community tutorials and explanations of *why*, which the official docs mostly lack.
- The **vanilla behaviour and resource packs** — download from `github.com/Mojang/bedrock-samples`. These are the actual files the game uses for every vanilla mob, item and block. When you want to know how something is done, open the vanilla file and read it. This is the most useful reference in existence and it is free.

◆ Download the vanilla packs now

Seriously — do it before you continue. Almost every question this guide cannot answer ("how does the vanilla wolf sit down?") is answered by opening `bedrock-samples/behavior_pack/entities/wolf.json` and reading it.

Hardware and performance. None of these tools are demanding. Blockbench runs on integrated graphics. The only heavy thing is Minecraft itself. If Blockbench is slow, turn off **anti-aliasing** and **shadows** in its preferences.

Working offline. Blockbench, VS Code and Paint.NET all work fully offline once installed. bridge.'s web version needs to be loaded once; after that its service worker caches it, but the desktop version is safer if your internet is unreliable.

3 First steps of creating an addon

Before a single texture is drawn, spend twenty minutes on setup. This section covers deciding what you are making, choosing a namespace, creating the two pack skeletons, and confirming that an empty add-on loads in game. Confirming the empty shell works first means that when something later fails, you know it is your new content and not your setup.

3.1 Planning the add-on

Write down, in plain English, before you open any app:

1. **What is in it?** List every mob, item and block. For each, one sentence: *"Frost Wraith: a floating hostile mob that spawns in snowy biomes at night, attacks with a slowing touch, and drops frost shards."*
2. **What does each thing need?** A mob needs a model, a texture, at least an idle animation, AI, health/damage stats, a loot table, a spawn rule and a name. An item needs an icon, properties and a name. A block needs textures, a shape, mining properties and a name.
3. **What depends on what?** If your item is dropped by your mob and used to craft your block, build them in that order so you can test each link as you go.
4. **What is the minimum first version?** Cut it down brutally. A single mob that spawns and can be killed is a real, finished add-on. Add the other six mobs afterwards. Half-finished add-ons that were planned huge are the number-one reason people give up.

★ Scope discipline

Your first add-on should be **one mob, one item, one block**. That combination touches every system in Bedrock exactly once. Once it works end to end, adding the tenth mob takes twenty minutes because you are copying a pattern you already understand.

3.2 Choosing your namespace

The **namespace** is the prefix on every identifier you create: in `frostpack:frost_wraith`, `frostpack` is the namespace.

Rules and advice:

- Lowercase letters, numbers and underscores. No spaces, no capitals, no hyphens.
- Make it unique to you or your pack — `frostpack`, `aaron_mobs`, `deepdark`. If two add-ons both use `custom:sword`, they will collide and one will break the other.
- Never use `minecraft:`, and avoid `mc:`, `bedrock:` and other names that look official.

- **Pick it once and never change it.** Changing a namespace later means editing dozens of files and re-writing every world that already contains your blocks — existing placed blocks and spawned mobs will simply vanish, because the game can no longer find their definitions.

The examples in this guide use the namespace `frostpack`. Substitute your own throughout.

3.3 Creating the pack skeletons

You need two folders. Create them inside the game's development folders so testing is instant.

1. Open File Explorer and paste this into the address bar, then press Enter:

TEXT

```
%localappdata%\Packages\Microsoft.MinecraftUWP_8wekyb3d8bbwe\LocalState\games\com.mojang
```

1. Open `development_behavior_packs` and create a folder called `frostpack_BP`.
2. Go back, open `development_resource_packs` and create `frostpack_RP`.

① If those folders don't exist

`development_behavior_packs` and `development_resource_packs` are only created after the game has launched at least once. Launch Minecraft, load any world, quit, and look again. If they are still missing, create them yourself with exactly those names — the game will use them.

At this point each folder is empty. A pack with no `manifest.json` will not appear in game at all, so the very next step is a minimal manifest for each. The Creating the manifests section explains every field in detail; for now, copy these two files so you have something loadable.

```
frostpack_BP/manifest.json
```

JSON

```
{
  "format_version": 2,
  "header": {
    "name": "Frost Pack Behaviours",
    "description": "Behaviours for the Frost Pack add-on.",
    "uuid": "PASTE-UUID-1-HERE",
    "version": [1, 0, 0],
    "min_engine_version": [1, 21, 0]
  },
  "modules": [
    {
      "type": "data",
      "uuid": "PASTE-UUID-2-HERE",
      "version": [1, 0, 0]
    }
  ]
}
```

frostpack_RP/manifest.json

JSON

```
{
  "format_version": 2,
  "header": {
    "name": "Frost Pack Resources",
    "description": "Models, textures and sounds for the Frost Pack add-on.",
    "uuid": "PASTE-UUID-3-HERE",
    "version": [1, 0, 0],
    "min_engine_version": [1, 21, 0]
  },
  "modules": [
    {
      "type": "resources",
      "uuid": "PASTE-UUID-4-HERE",
      "version": [1, 0, 0]
    }
  ]
}
```

3.3.1 Getting your UUIDs

A **UUID** is a 36-character unique identifier that looks like `f0e1d2c3-b4a5-4968-8776-a5b4c3d2e1f0`. Every pack needs **two of its own** (one for the header, one for each module), and no two packs anywhere may share

one.

Get four fresh ones from any of these:

- `uuidgenerator.net` — click, copy, repeat.
- In VS Code, install the **UUID Generator** extension, or press `Ctrl+Shift+P` and run `Insert UUID` if your extension provides it.
- bridge. v2 and Blockbench generate them automatically when they create a manifest for you.
- PowerShell: type `[guid]::NewGuid()` and press Enter.

Paste one into each of the four `PASTE-UUID-n-HERE` slots. They must all be different.

⚠ **Never copy someone else's manifest UUIDs**

If you download an add-on and reuse its manifest, both packs now claim the same identity. Minecraft will load one and silently ignore the other, and you will spend an evening wondering why your pack does not appear. Always generate new UUIDs — including when you copy your *own* pack to start a second project.

3.4 Adding a pack icon

Create a **128×128 PNG** named exactly `pack_icon.png` and put one in each pack folder's root. This is what shows in the pack list. It is optional — a missing icon just shows a blank pane — but a pack with no icon looks broken.

You can make one in Paint.NET (`File > New` , 128×128), or use Blockbench's `View > Screenshot Model` on your mob later and crop it.

3.5 Adding a language file

Every name your add-on shows to the player comes from a translation file. Without it, players see raw strings like `item.frostpack:frost_shard.name` .

Create `frostpack_RP/texts/en_US.lang` — a plain text file, **not** JSON:

TEXT

```
pack.name=Frost Pack Resources
pack.description=Models, textures and sounds for the Frost Pack add-on.
```

You will add entity, item and block names to this file as you create them. Note that translation keys are added by other sections; the format is `key=Value` , one per line, with no quotes and no commas. Lines starting with `##` are comments.

❗ Why the pack name appears twice

`pack.name` and `pack.description` in `en_US.lang` override the `name` and `description` fields in the manifest. You can also put `pack.name` directly in the manifest as literal text and skip the lang entries. Using the lang file is better practice because it lets you translate your pack later.

3.6 Verifying the empty packs load

Do this now, before making any content.

1. Launch Minecraft.
2. `Play > Create New World`.
3. In the left menu, click **Behaviour Packs**. Under **Available**, you should see **Frost Pack Behaviours**. Click it, then click **Activate**.
4. Click **Resource Packs**. You should see **Frost Pack Resources**. Activate it too.
5. Set **Game Mode: Creative**, turn on **Cheats**, and create the world.

If both packs appear and the world loads, your setup is correct. If a pack is missing, see the checklist below.

◆ Troubleshooting a pack that will not appear

- Is `manifest.json` in the pack's **root**, not in a sub-folder? A common error is `frostpack_BP/frostpack_BP/manifest.json` caused by unzipping.
- Is it named `manifest.json` exactly — not `manifest.json.txt`? Turn on **File name extensions** in Explorer's View menu to check.
- Are all four UUIDs different and correctly formatted?
- Is `format_version` the number `2`, not the string `"2"`?
- Is there a trailing comma anywhere? JSON forbids a comma before a closing `}` or `]`.
- Is `min_engine_version` lower than or equal to your actual game version?

3.7 Setting up your editor

Open the folder that *contains* both packs in VS Code. If your packs are in the two `development_*` folders, they are not next to each other, so use a **workspace**:

1. VS Code → `File > Open Folder` → select `frostpack_BP`.
2. `File > Add Folder to Workspace` → select `frostpack_RP`.
3. `File > Save Workspace As` → save `frostpack.code-workspace` somewhere convenient.

Now both packs are in one tree, project-wide search covers both, and you can reopen everything in one click. This matters more than it sounds: you will constantly need to check that an identifier in the BP matches the RP.

3.8 Extra things and tips

Set up a dedicated test world. Make one Creative, flat, always-day world with cheats on and keep it forever. Use `/gamerule doDaylightCycle false`, `/gamerule doMobSpawning false` while you are testing summons, and `/gamerule sendcommandfeedback true`. A superflat world makes it easy to see mobs and place blocks without terrain in the way.

Learn these commands now. They are your entire testing loop:

TEXT

```
/summon frostpack:frost_wraith  
/give @s frostpack:frost_shard 1  
/setblock ~ ~ ~ frostpack:frost_lantern  
/kill @e[type=frostpack:frost_wraith]  
/reload
```

`/reload` re-reads behaviour pack data (entity files, loot tables, functions, recipes). It does **not** reload the resource pack — models, textures and animations need you to leave the world and re-enter, and sometimes to fully restart the game.

Keep a scratch file. A `notes.txt` in your project with your namespace, your identifier list, and any Molang expression you worked out the hard way. You will forget them.

Turn on file extensions in Windows. File Explorer → `View > Show > File name extensions`. Without this you cannot tell `manifest.json` from `manifest.json.txt`, and that single confusion has ended more add-ons than any other.

Decide about experiments early. Some features (certain block and item components, some script APIs) require an **experimental toggle** on the world. If your add-on needs one, every player also needs to enable it, and worlds are permanently flagged. Check the documentation for anything you use and prefer non-experimental features where possible.

Version your pack from day one. Set `"version": [1, 0, 0]` now and increase it every time you share the pack. Minecraft uses this number to decide whether an imported pack replaces an existing copy; if the version does not increase, players may keep the old one.

4 Creating mobs textures and body

This section is about building the *appearance* of a mob: its 3D model (the "geometry" or "geo"), its texture, and the settings that tie the two together. Animations and behaviour come later — right now you are making a statue.

Everything here happens in **Blockbench**.

4.1 Understanding Bedrock geometry

Before you build anything, understand what a Bedrock model actually is, because the rules are strict and non-obvious.

A model is a tree of bones. Bones contain cubes. A cube is always a rectangular box — there are no spheres, no cylinders, no triangles. Every shape in Minecraft is boxes, and skill comes from choosing boxes well.

Units. Model space is measured in "pixels" where **16 pixels = 1 block**. A player is 32 pixels tall (2 blocks). Coordinates run: **X** = left/right, **Y** = up/down (up is positive), **Z** = forward/back. The model's origin `[0, 0, 0]` is where the mob's feet meet the ground, in the centre of its footprint.

Bones are the important part. A bone is a named group with a **pivot** — the point it rotates around. Animations move bones, never cubes. If you put every cube in one bone, your mob can only move as a single rigid lump. A walking mob needs at least a body bone, a head bone and one bone per leg, each with its pivot at the joint.

Bones can be **parented**: if `head` is a child of `body`, rotating the body carries the head with it. Vanilla mobs use this heavily. A useful default hierarchy for a humanoid:

TEXT

```
root
├─ body
│   ├── head
│   │   └─ hat (optional overlay layer)
│   ├── arm_left
│   ├── arm_right
│   ├── leg_left
│   └─ leg_right
```

Some creators add a `root` bone containing everything, so the whole mob can be tilted or bobbed in one keyframe. It costs nothing and is often handy.

Bone names become part of your animation files. Renaming a bone after you have animated it breaks every animation that referenced it, silently. Name bones properly the first time: lowercase, underscores, descriptive.

⚠ **Names must be lowercase with underscores**

`leg_left` — good. `Leg Left`, `LegLeft`, `leg-left` — bad. Capitals and spaces work *sometimes* and fail in ways that are very hard to diagnose. Do not risk it.

4.2 Setting up the Blockbench project

1. Open Blockbench → **New** → **Bedrock Model**.
2. Immediately open **Project Settings** (`File > Project` , or the wrench icon in the toolbar).
3. Set these fields:

Field	What to put	Why
Model Identifier	<code>geometry.frost_wraith</code>	This exact string goes in your entity's client file. Convention is <code>geometry.<mob_name></code> .
Texture Width	<code>64</code>	The width of the texture image in pixels.
Texture Height	<code>64</code>	The height. Vanilla uses 64×64 for most mobs, 64×32 for older ones.
Visible Bounds Width	<code>2</code>	How wide (in blocks) the box is that the game uses to decide whether the mob is on screen.
Visible Bounds Height	<code>2.5</code>	How tall that box is.
Visible Bounds Offset	<code>[0, 1, 0]</code>	Where the box is centred relative to the model origin.

⚠ **Visible bounds are not optional**

If the visible bounds are smaller than your model, the mob will **disappear** when its centre goes slightly off screen, and parts of it will clip out of existence when you turn your head. Always set the bounds a little larger than the model. A common symptom of wrong bounds is "my mob flickers when I walk past it".

1. Set the **texture size** before you build. Changing it later remaps all your UVs and ruins the texture alignment.

4.3 Blocking out the body

Work large to small: torso first, then head, then limbs, then details.

1. **Create the root bone.** In the Outliner, click the **new group / folder** icon. Name it `root`. Set its **Pivot** to `[0, 0, 0]`.
2. **Create the body bone** inside `root`. Name it `body`. Set the pivot at the model's waist, e.g. `[0, 12, 0]`.
3. **Add a cube** (the cube icon in the Outliner header, or `Edit > Add Cube`). With the cube selected, use the **Element panel** on the right to type exact numbers rather than dragging:
 1. **From:** `[-4, 12, -2]`
 2. **To:** `[4, 24, 2]`

That is an 8-wide, 12-tall, 4-deep torso whose bottom sits at y=12.

1. **Rename the cube** — Blockbench names them `cube`, `cube2` ... Rename to `torso`. Cube names do not affect Minecraft, but they keep you sane.

Repeat for each part. Useful numbers for a humanoid the size of a player:

Part	From	To	Bone pivot
Head	<code>[-4, 24, -4]</code>	<code>[4, 32, 4]</code>	<code>[0, 24, 0]</code>
Torso	<code>[-4, 12, -2]</code>	<code>[4, 24, 2]</code>	<code>[0, 24, 0]</code>
Right arm	<code>[-8, 12, -2]</code>	<code>[-4, 24, 2]</code>	<code>[-5, 22, 0]</code>
Left arm	<code>[4, 12, -2]</code>	<code>[8, 24, 2]</code>	<code>[5, 22, 0]</code>
Right leg	<code>[-4, 0, -2]</code>	<code>[0, 12, 2]</code>	<code>[-2, 12, 0]</code>
Left leg	<code>[0, 0, -2]</code>	<code>[4, 12, 2]</code>	<code>[2, 12, 0]</code>

Notice the pivots: an arm pivots at the **shoulder**, not at its centre, and a leg pivots at the **hip**. Getting pivots right is what makes an animation look like a limb swinging instead of a plank spinning.

★ Snapping and precision

In `File > Preferences > Settings`, set **Snapping** to `1` while blocking out, so cubes land on whole pixels. Drop to `0.25` or `0.5` only for small details. Off-grid cubes cause texture seams and z-fighting.

4.3.1 Techniques for non-humanoid shapes

- **Rotated cubes.** A cube can be rotated on one axis (in the Element panel's Rotation fields). In Bedrock, cube rotation is free-form — unlike Java's 22.5° limitation — so you can angle wings, horns and tails to any degree. Rotate the *cube* for a fixed angle; rotate the *bone* for something you will animate.

- **Inflate.** The **Inflate** field grows a cube outward by a fraction of a pixel in every direction without changing its UV. Use it to make an overlay layer (armour, fur, a glowing shell) sit just outside the base layer — typically `0.25` or `0.5`. This is how vanilla does the player's "jacket" layer.
- **Overlapping cubes cause z-fighting** — a flickering stripe where two faces sit at exactly the same depth. Fix it by offsetting one cube by a fraction (0.01 is enough) or by using **Inflate**.
- **Flat planes.** A cube with zero thickness on one axis becomes a single plane, useful for wings, fins, ears and leaves. Both faces render, but a zero-thickness cube can flicker; give it 0.01 depth if it does.
- **Mirroring.** Build one arm, then `Edit > Duplicate` and use `Transform > Flip > X` to make the other. Blockbench's **Mirror UV** toggle (in the UV panel) makes the mirrored cube read its texture flipped, so both limbs can share the same texture region — exactly like vanilla arms.

4.3.2 Keeping the poly count sane

Every cube is 12 triangles. A vanilla mob is typically 6–20 cubes. You can go to a hundred, but a mob that spawns in groups of six with 300 cubes each will cost frame rate on phones and consoles. Detail belongs in the texture, not the geometry — a two-pixel notch painted on a face costs nothing and reads the same from three blocks away.

4.4 UV mapping

The **UV** is the map from each cube face to a rectangle on the texture image. Bedrock supports two modes, and the choice matters.

Box UV (the default, and what vanilla uses). Blockbench automatically lays out all six faces of a cube in the classic unfolded-cross arrangement, and you position the whole cross with a single X/Y coordinate. Simple, compact, and required if you want the "skin-like" layout. Downsides: you cannot give two faces different regions, and mirrored cubes need the Mirror flag.

Per-face UV. Each face gets its own rectangle, placed independently. This lets you reuse one patch of texture on many faces (huge texture savings) and lets a single cube have completely different art on each side. Enable it in **Project Settings** (`Box UV` toggle off) or per-project when creating the model.

① Which should you use?

Use **Box UV** for organic creatures that you will texture like a skin. Use **per-face UV** for mechanical things, blocks, and anything where you want to reuse texture patches. You must choose per project — mixing is not possible — and switching later re-lays out everything.

Doing the UV layout

1. Switch to the **UV panel** on the right.
2. With a cube selected, drag its UV box to an empty area of the texture. Blockbench shows occupied areas, so you can avoid overlaps.

3. Use the panel's **auto UV** / "**Optimise UV**" buttons to pack everything automatically — a good starting point that you then tidy by hand.
4. **Do not let UVs overlap** unless you deliberately want two cubes to share art. Overlapping UVs mean painting one cube secretly repaints another.
5. Keep everything **inside** the texture bounds. A UV that runs off the edge samples garbage or wraps around.

Texture size choices. 64×64 is the standard and plenty for a mob. Use 128×128 if you have many cubes or want finer detail. Always use **powers of two** (16, 32, 64, 128, 256) — non-power-of-two textures work in Bedrock but can behave badly on some devices and with mipmapping.

4.5 Creating the texture

You have two routes. Most people use both.

4.5.1 Painting in Blockbench

1. In the **Textures** panel (bottom-right), click + → **Create Texture**. Set the size to match your project's texture size (64×64) and give it a name like `frost_wraith`.
2. Switch to **Paint** mode (top tabs).
3. Select the **Brush**, set **Size: 1**, **Opacity: 100%**, **Softness: 0**. Softness above zero produces anti-aliased, blurry pixels which look wrong in Minecraft.
4. Paint directly on the 3D model. This is Blockbench's superpower — you can see exactly which pixel lands where.
5. Double-click the texture in the Textures panel to open the flat **Texture Editor** for precise work, and use the two views together.
6. Right-click the texture → **Save As** to write the PNG.

Useful painting tools: **Paint Bucket** (fill a face — hold `Ctrl` to limit to the selected face), **Colour Picker** (`Alt` + click), **Gradient**, **Draw Shape**, and **Mirror Painting** (paints symmetrically on X, ideal for faces and bodies).

4.5.2 Painting in Paint.NET

1. In Blockbench, right-click the texture → **Save As** → save the (blank or roughed-in) PNG into your resource pack.
2. Open that PNG in Paint.NET.
3. `View > Pixel Grid` on, zoom to 800% or more.
4. Use the **Pencil** tool (not Paintbrush) so pixels are hard-edged.
5. Work in **layers**: base colours on one, shading on another, details on a third. Flatten only when saving.
6. Save as PNG. Blockbench detects the file changed and reloads it live — leave both apps open and you get a very fast loop.

4.5.3 Making a texture look good

- **Use a limited palette.** Three to five shades per material — a base, a shadow, a highlight, and maybe an accent. More colours look noisier, not richer.
- **Shade from a consistent light direction.** Minecraft's own art is lit from above, so top faces are brightest, front/side faces mid, bottom faces darkest. Bedrock already darkens faces by direction automatically, so do not over-shade or the mob turns muddy.
- **Add noise sparingly.** A few scattered darker pixels break up flat areas and read as texture. Whole-surface noise reads as static.
- **Draw the silhouette detail into the texture.** Straps, seams, teeth, eyes — all cheaper as pixels than as cubes.
- **Eyes are everything.** Two pixels of contrast in the right place turns a box into a creature. Vanilla mobs are almost all defined by their eyes.
- **Check it in game early.** Textures always look different at real scale under real lighting. Do a rough pass, look at it in game, then refine.

4.5.4 Transparency and special texture behaviour

- Areas left fully transparent (alpha 0) are invisible **only if** the entity's material supports it. The default `entity` material is opaque and will render transparent pixels as black. To use transparency you must set the material to `entity_alphatest` (hard-edged cut-out, e.g. holes and leaf-like shapes) or `entity_alphablend` (soft translucency, e.g. ghosts). Materials are chosen in the **render controller** — see the Coding section.
- **Emissive / glowing parts** need the `entity_emissive` or `entity_emissive_alpha` material. With those, the **alpha channel controls glow**: fully opaque pixels render normally, and pixels with alpha below 255 glow in the dark. This is counter-intuitive and catches everyone out — with an emissive material, alpha is *not* transparency.
- **Overlay textures** (like a spider's glowing eyes or a creeper's charged aura) are a second texture drawn on top by a second pass in the render controller.

4.6 Exporting the model

1. `File > Save` your `.bbmodel` first. Keep these somewhere outside the pack — they are your editable sources, and Minecraft does not read them.
2. `File > Export > Bedrock Geometry`.
3. Save it into your resource pack as:

TEXT

```
frostpack_RP/models/entity/frost_wraith.geo.json
```

Open the exported file and check the top:

JSON

```
{
  "format_version": "1.16.0",
  "minecraft:geometry": [
    {
      "description": {
        "identifier": "geometry.frost_wraith",
        "texture_width": 64,
        "texture_height": 64,
        "visible_bounds_width": 2,
        "visible_bounds_height": 2.5,
        "visible_bounds_offset": [0, 1, 0]
      },
      "bones": [ ... ]
    }
  ]
}
```

The `identifier` here is the string your entity file must reference. If they do not match exactly, the mob renders as nothing (or as the missing-model shape).

1. Save the texture into:

TEXT

```
frostpack_RP/textures/entity/frost_wraith.png
```

◆ Export is not save

`File > Save` writes a `.bbmodel`. Minecraft cannot read `.bbmodel`. You must **Export > Bedrock Geometry** every single time you change the model. Forgetting this is the most common "why has my change done nothing" in the whole workflow. Blockbench remembers the export path, so after the first time it is `Ctrl+E`.

4.7 Other information for mobs

Model scale and hitboxes are separate. The size of your geometry has no effect on the mob's collision box or how much space it needs to spawn. The hitbox is set in the behaviour pack with `minecraft:collision_box`, and how tall the model appears can be adjusted with `minecraft:scale` or in the render controller. Set the collision box to match the model roughly, or players will hit empty air.

Attachment points. If your mob should hold an item or wear armour, the bones must be named the way the game expects for the built-in attachable system, or you must write your own animation to position a held-item bone. For a mob that only holds items via `minecraft:behavior.pickup_items`, the item is rendered by the render controller's item slot logic — see the vanilla zombie's client entity file for the pattern.

Reusing vanilla models. You can point your entity at a vanilla geometry (`geometry.zombie`, `geometry.creeper`) and supply only a new texture. This is the fastest possible custom mob and is completely legitimate for prototyping — get the behaviour working with a vanilla model, then swap in your own.

Multiple textures on one mob. A mob can have several textures defined and pick between them at runtime (variants, damage states, team colours). Define each in the client entity file's `textures` block and choose between them in the render controller with a Molang expression on `variant` or `mark_variant`.

4.8 Extra tips for mobs

- **Build in the T-pose.** Arms out, legs straight, facing +Z. Animations assume a neutral rest pose, and a model built mid-pose is miserable to animate.
- **Face the right way.** In Bedrock, a mob's *front* faces **positive Z** in Blockbench's default view. If your mob walks backwards in game, your model is built facing the wrong direction. Fix it by rotating the root bone 180° — or better, rebuild it facing the right way.
- **Symmetry.** Build the right half, then duplicate and mirror. Use Blockbench's `Transform > Flip > X` on a selected bone to mirror a whole limb hierarchy.
- **Keep a reference.** Drop a vanilla model into Blockbench (`File > Open Model` on a vanilla `.geo.json`) alongside yours to compare scale. Mobs that are subtly too big or too small look wrong and you cannot tell why.
- **Test the silhouette.** Turn the texture off and look at the black shape. If you cannot tell what it is from the silhouette alone, no texture will save it.
- **Save incrementally.** `frost_wraith_v1.bbmodel`, `_v2`, `_v3`. Blockbench has no version history.
- **Do not animate yet.** It is tempting, but a model whose bone structure changes after animating means redoing the animation. Lock the skeleton down first.

5 Creating item textures and layout

Items are much simpler than mobs, and almost all of the work is a single small PNG. This section covers drawing item icons, the special case of 3D items, and the layout files that connect an icon to an item identifier. Item *behaviour* (what happens when you use it) is in the Coding section.

5.1 How items differ from mobs

Read this first — it will save you assuming the wrong thing.

- **Unlike mobs, most items have no 3D model at all.** An item is normally a flat PNG that Minecraft renders as a sprite in the inventory and extrudes into a thin 3D slab when held. You do not build geometry, you do not set bones, and you do not export from Blockbench.
- **Unlike mobs, item textures are tiny.** 16×16 is the standard. 32×32 is acceptable; anything larger looks out of place next to vanilla items and wastes memory.
- **Unlike mobs, items are registered in a lookup file.** A mob's texture path is written directly into its client entity file. An item's texture is registered in `textures/item_texture.json` under a short key, and the item's behaviour file refers to that key — not to the file path. This indirection catches people out constantly.
- **Unlike mobs, there is no render controller for items.** No materials to choose, no Molang, no variant switching in the same way. What you draw is what you get.

5.2 Item texture rules

Property	Value
Format	PNG
Size	16×16 (standard); 32×32 acceptable
Colour	Full RGB
Transparency	Fully supported — alpha 0 is genuinely invisible
Colour depth	32-bit RGBA
Interpolation	None — nearest-neighbour only

Transparency on items works without any material setup at all, which is a pleasant difference from mobs.

5.3 Drawing an item icon

Work in Paint.NET (or Blockbench's texture editor, or GIMP).

1. `File > New` → **Width 16, Height 16**.
2. Delete the background layer's white fill (`Ctrl+A`, `Delete`) so you are drawing on transparency.
3. `View > Pixel Grid` on. Zoom to 1600%.
4. Use the **Pencil** tool at size 1.
5. `File > Save As` → PNG into `frostpack_RP/textures/items/frost_shard.png`.

5.3.1 Making an icon read at 16×16

At the size an item appears in the hotbar, you have perhaps 12 usable pixels across. Design accordingly:

- **Silhouette first.** Draw the shape in one flat colour and check you can recognise it. If not, exaggerate — a sword blade wider than realistic, a bottle rounder than realistic.
- **Diagonal, not straight.** Vanilla tools and weapons run corner-to-corner. This gives the longest possible line inside a 16×16 box and instantly reads as "Minecraft item".
- **Outline.** Most vanilla items have a darker outline of the item's own hue (not black). It separates the icon from any inventory background.
- **Three-shade rule.** Base colour, one shade darker for the shadowed side, one lighter for the lit edge. Light from the top-left, as vanilla does.
- **Avoid pure black and pure white.** They look harsh; use a very dark tint of the item's colour instead.
- **Keep a one-pixel margin** at the edges where possible, so the icon does not collide with the item-count number or durability bar drawn in the bottom-right.
- **Consistency across your set.** If you are making five items, draw them together in one file first so their palettes, outlines and angles match, then cut them into separate PNGs.

★ Reuse the vanilla palette

Open a vanilla item PNG from `bedrock-samples/resource_pack/textures/items/` in Paint.NET and eyedropper its colours. Matching vanilla's palette is the quickest way to make custom items look like they belong in the game.

5.3.2 Layered / tinted items

Items can be tinted by the game (like leather armour or potions) using multiple texture layers. This is an advanced case; the simple version is that you draw a **greyscale** base texture and the game multiplies a colour over it. For most add-ons, drawing each colour variant as its own PNG is simpler and more predictable.

5.4 Registering the texture: item_texture.json

This is the layout file that gives your PNG a short name.

Create `frostpack_RP/textures/item_texture.json`:

JSON

```
{
  "resource_pack_name": "frostpack",
  "texture_name": "atlas.items",
  "texture_data": {
    "frost_shard": {
      "textures": "textures/items/frost_shard"
    },
    "frost_blade": {
      "textures": "textures/items/frost_blade"
    }
  }
}
```

Points to be precise about:

- `texture_name` must be exactly `atlas.items`. This tells the game these go into the item atlas.
- The keys inside `texture_data` (`frost_shard`, `frost_blade`) are the **short names** you will use in the item's behaviour file, in its `minecraft:icon` component. They are yours to choose, but keeping them identical to the file name avoids confusion.
- The path in `textures` is **relative to the resource pack root** and has **no `.png` extension**.
`textures/items/frost_shard` — not `frost_shard.png`, not `/textures/...`, not `frostpack_RP/textures/...`.
- `resource_pack_name` is informational; put your pack's short name.
- One `item_texture.json` per resource pack. Add every item to this one file — do not make several.

⚠ Forward slashes only

Windows shows paths with backslashes (`textures\items\frost_shard`). Minecraft requires **forward slashes**. A backslash in a JSON string is also an escape character, so `"textures\items\..."` is not even valid JSON. Always type `/`.

5.5 Item names: the language file

Add the display name to `frostpack_RP/texts/en_US.lang`:

TEXT

```
item.frostpack:frost_shard=Frost Shard  
item.frostpack:frost_blade=Frost Blade
```

The key format is `item.<namespace>:<item_name>` — and, depending on your item's `format_version`, may need a trailing `.name`:

TEXT

```
item.frostpack:frost_shard.name=Frost Shard
```

If your item shows its raw identifier in game, try the other form. Both appear in the wild because the requirement changed between item format versions; the version without `.name` is correct for modern data-driven items.

① Alternatively, skip the lang file

Modern item formats support `minecraft:display_name` as a component, taking either a literal string or a translation key. Using the lang file is still better for anything you intend to translate, and it is the only way to name blocks and entities consistently.

5.6 Attachables: items with real 3D models

Sometimes a flat sprite is not enough — a custom shield, a backpack, a gun, a hat. For these you need an **attachable**: a model that renders on the player's body when the item is held or worn.

This is the one place where item creation looks like mob creation.

1. In Blockbench, create a **Bedrock Model** as you would for a mob, with identifier `geometry.frost_blade`.
2. Build the model. The origin should be positioned so the item sits correctly in a hand — build it around roughly `[0, 0, 0]` and expect to adjust with animation offsets.
3. Export to `frostpack_RP/models/entity/frost_blade.geo.json` (attachable models live under `models/entity/`, not `models/item/`).
4. Create `frostpack_RP/attachables/frost_blade.attachable.json`:

```

{
  "format_version": "1.10.0",
  "minecraft:attachable": {
    "description": {
      "identifier": "frostpack:frost_blade",
      "materials": {
        "default": "entity_alphatest",
        "enchanted": "entity_alphatest_glint"
      },
      "textures": {
        "default": "textures/entity/frost_blade",
        "enchanted": "textures/misc/enchanted_item_glint"
      },
      "geometry": {
        "default": "geometry.frost_blade"
      },
      "scripts": {
        "pre_animation": [
          "v.main_hand = c.item_slot == 'main_hand';",
          "v.off_hand = c.item_slot == 'off_hand';",
          "v.head = c.item_slot == 'head';"
        ],
        "animate": [
          { "first_person_main_hand": "v.main_hand && c.is_first_person" },
          { "third_person_main_hand": "v.main_hand && !c.is_first_person" },
          { "third_person_head": "v.head && !c.is_first_person" }
        ]
      },
      "animations": {
        "first_person_main_hand": "animation.frost_blade.first_person",
        "third_person_main_hand": "animation.frost_blade.third_person",
        "third_person_head": "animation.frost_blade.head"
      },
      "render_controllers": [ "controller.render.item_default" ]
    }
  }
}

```

The `identifier` of an attachable must **exactly match the item's identifier**. That is how the game links them; there is no other reference.

1. Write the positioning animations in `frostpack_RP/animations/frost_blade.animation.json`. These are ordinary animations whose only job is to set a fixed position and rotation:

JSON

```
{
  "format_version": "1.8.0",
  "animations": {
    "animation.frost_blade.third_person": {
      "loop": true,
      "bones": {
        "root": {
          "position": [0, 4, -2],
          "rotation": [0, -90, 0],
          "scale": 0.6
        }
      }
    }
  }
}
```

Getting these numbers right is pure trial and error. Change, re-enter world, look, adjust. Copy the vanilla `shield.attachable.json` and its animation as a starting point — the offsets are already close to correct.

① Attachables are for wearing and holding only

An attachable does not change what the item looks like in the **inventory**. The inventory always shows the flat icon from `minecraft:icon`. So a 3D item still needs a 2D icon drawn for it.

5.7 Other information for items

Spawn eggs. If your mob should have a spawn egg, you do not create an item for it — the `minecraft:spawn_egg` description property on the *entity* creates one automatically. You can either give it two tint colours (`base_color` and `overlay_color`) for the vanilla speckled-egg look, or point it at a custom texture. See the Coding section.

Item groups / creative menu placement. Where your item appears in the Creative inventory is set by the `minecraft:creative_category` (older) or the `menu_category` block in the item description (current). You choose a **category** (`construction` , `nature` , `equipment` , `items`) and optionally a **group** (`itemGroup.name.sword` etc.) to nest it beside vanilla equivalents.

Durability bars and stack sizes are behaviour, not art — `minecraft:durability` and `minecraft:max_stack_size` components. Note that an item with durability cannot stack.

Item texture resolution mismatch. All items in the atlas get packed together. Mixing 16×16 and 64×64 items works, but the atlas grows to accommodate the largest, so a single oversized icon can quietly quadruple your texture memory.

5.8 Extra tips for items

- **Draw a set on one canvas.** Make a 64×64 canvas, draw four 16×16 items in a grid, keep them consistent, then use `Image > Crop to Selection` to cut each one out and save separately.
- **Test in the hotbar, not the file.** An icon that looks great at 1600% zoom often turns to mud at real size. Get it in game early.
- **Steal your own shading.** Once you have one item you like, build the rest from its palette and outline style.
- **Keep the source file.** Save a layered `.pdn` (Paint.NET) or `.xcf` (GIMP) alongside the flat PNGs so you can revise shading later.
- **Name files after identifiers.** `frost_shard.png` for `frostpack:frost_shard`. When you have thirty items, matching names are the only thing keeping the project navigable.

6 Creating block textures shape/geo

Blocks sit between mobs and items in complexity. Like items, they can be made with nothing but flat PNGs. Like mobs, they can also have full custom geometry. This section covers both routes, the texture layout file, and what makes blocks different from everything else.

6.1 How blocks differ from mobs and items

- **Unlike items, a block needs up to six textures** — one per face — and they are assigned by face name (`up` , `down` , `north` , `south` , `east` , `west`) or as a single texture used on all six.
- **Unlike items, block textures tile.** A block sits next to other blocks, so the left edge of the texture meets the right edge of the neighbour. Textures that look great in isolation can produce obvious repeating patterns across a wall. Design for tiling.
- **Unlike mobs, a block with custom geometry does not need bones or animation.** You still build it in Blockbench, but a single bone containing all the cubes is fine. Blocks do not animate through the animation system at all.
- **Unlike mobs, custom block geometry is constrained.** A block model should stay within the $16 \times 16 \times 16$ pixel cube (from `[-8, 0, -8]` to `[8, 16, 8]` in Blockbench's Bedrock Block format). Geometry *can* exceed one block — up to `[-16,-16,-16]` to `[32,32,32]` in current formats — but anything outside the base cube will not be lit, culled or collided with the way you expect, and neighbouring blocks will not know it is there.
- **Unlike both, blocks have states and permutations.** A single block definition can have many variants (rotation, on/off, growth stage) chosen by data values. This is enormously powerful and is covered in the Coding section.
- **Unlike both, blocks are registered in two texture files:** `terrain_texture.json` (the short-name lookup, equivalent to `item_texture.json`) and optionally `blocks.json` (an older mapping file, still used for some vanilla-style behaviour).

6.2 Block texture rules

Property	Value
Format	PNG
Size	16×16 standard; 32×32 or 64×64 for high-detail packs
Transparency	Supported, but requires the right material instance render method
Tiling	Must tile seamlessly with itself unless the block is always isolated

6.3 Drawing block textures

1. New 16×16 image, transparent background.
2. Fill with a base colour.
3. Add noise: scatter a handful of pixels one shade darker and one shade lighter. This is essentially how vanilla stone, dirt and wood are made — a base plus controlled noise.
4. Add structure: plank lines, brick courses, cracks, grain. Keep them from touching the edges in a way that breaks tiling.
5. Check the tiling: in Paint.NET, `Image > Canvas Size` to 48×48 with anchor top-left, then copy-paste the tile nine times into a grid. Look for a visible "seam" line or an obvious repeated blob. Adjust and re-check.
6. Save as PNG into `frostpack_RP/textures/blocks/frost_lantern.png`.

Texture design advice specific to blocks

- **Contrast low, variation high.** A block is seen hundreds of times in a wall. High contrast becomes visual noise at scale. Vanilla stone spans a narrow range of greys.
- **No strong directional light.** Unlike items, a block face can be seen from any side, and the game already shades faces by direction. Bake in minimal shading.
- **Avoid a single distinctive pixel.** One bright pixel in a texture becomes a polka-dot pattern across a wall. This is the classic beginner tiling mistake.
- **Keep edges "busy but neutral"** so the join between two copies is not obvious.

6.4 Registering block textures: `terrain_texture.json`

Create `frostpack_RP/textures/terrain_texture.json`:

JSON

```
{
  "resource_pack_name": "frostopack",
  "texture_name": "atlas.terrain",
  "padding": 8,
  "num_mip_levels": 4,
  "texture_data": {
    "frost_lantern": {
      "textures": "textures/blocks/frost_lantern"
    },
    "frost_bricks_top": {
      "textures": "textures/blocks/frost_bricks_top"
    },
    "frost_bricks_side": {
      "textures": "textures/blocks/frost_bricks_side"
    }
  }
}
```

- `texture_name` must be exactly `atlas.terrain`.
- `padding` and `num_mip_levels` should match vanilla's values (`8` and `4`) to avoid texture-bleeding artefacts at distance.
- As with items: paths are relative to the pack root, use forward slashes, and have **no** `.png`.
- The keys (`frost_lantern`, `frost_bricks_top`) are the short names used in the block's `minecraft:material_instances` component.

A texture entry can also be a **list**, which gives the block a random variant per placement:

JSON

```
"frost_bricks": {
  "textures": [
    "textures/blocks/frost_bricks_a",
    "textures/blocks/frost_bricks_b",
    "textures/blocks/frost_bricks_c"
  ]
}
```

6.5 Assigning textures to faces

This happens in the behaviour pack, in the block's `minecraft:material_instances` component. It is included here because it is really part of "how the block looks".

Same texture on all six faces:

JSON

```
"minecraft:material_instances": {
  "*": {
    "texture": "frost_lantern",
    "render_method": "opaque"
  }
}
```

Different textures per face:

JSON

```
"minecraft:material_instances": {
  "*": {
    "texture": "frost_bricks_side",
    "render_method": "opaque"
  },
  "up": {
    "texture": "frost_bricks_top",
    "render_method": "opaque"
  },
  "down": {
    "texture": "frost_bricks_top",
    "render_method": "opaque"
  }
}
```

The `*` entry is the default for any face not named. Valid face names are `up`, `down`, `north`, `south`, `east`, `west`, plus `*`.

6.5.1 Render methods — get this right

The `render_method` decides how transparency and lighting are handled. Choosing wrong is the most common block-appearance bug.

Value	Use for	Notes
<code>opaque</code>	Solid blocks with no transparent pixels	Fastest. Transparent pixels render black .
<code>alpha_test</code>	Hard-edged cut-outs: leaves, bars, glass panes, plants	Pixels are either fully visible or fully invisible. No partial transparency.
<code>alpha_test_singled_sided</code>	Cut-outs where you do not want back faces drawn	Newer variant; better performance for flat plants.
<code>blend</code>	Real translucency: stained glass, ice, water-like blocks	Most expensive; can sort incorrectly when two blend blocks overlap.
<code>double_sided</code>	Flat geometry that must be visible from both sides	Combine with the block being non-solid.

Additional per-instance properties worth knowing:

- `"ambient_occlusion": false` — disables the soft corner shading. Use on glowing or flat blocks.
- `"face_dimming": false` — stops the game darkening side and bottom faces. Essential for blocks that should look uniformly lit, such as lamps.
- `"tint_method"` — applies biome tinting (`grass`, `foliage`, `water`, `none`).

JSON

```
"minecraft:material_instances": {
  "*": {
    "texture": "frost_lantern",
    "render_method": "alpha_test",
    "face_dimming": false,
    "ambient_occlusion": false
  }
}
```

6.6 Custom block geometry

If your block is not a full cube — a lantern, a chair, a pipe, a slab with a custom profile — you build a model in Blockbench.

1. Blockbench → **New** → **Bedrock Block**. (If that format is unavailable, use **Bedrock Model** and keep the geometry inside the block bounds.)
2. Set **Project Settings**: identifier `geometry.frost_lantern`, texture size to match your texture (e.g. 16×16, or 32×32 if you use a larger sheet).

3. Build with cubes inside the block volume. In the Bedrock Block format, the block occupies `[-8, 0, -8]` to `[8, 16, 8]`, where `[0,0,0]` is the centre of the block's floor.
4. Use **per-face UV** for blocks — it lets you reuse a small texture across many faces, which is essential when a complex model must fit on one 16×16 sheet. If your model needs more room, use a larger texture (32×32 or 64×64) and register it in `terrain_texture.json` as normal.
5. Name faces' material instances if you want different render methods on different parts: in Blockbench, each face's UV panel has a **material instance** field. Set it to a name like `glass`, then define that name in `material_instances` alongside `*`.
6. `File > Export > Bedrock Geometry` → save to `frostpack_RP/models/blocks/frost_lantern.geo.json`.

❗ Where do block models live?

Convention is `models/blocks/` for blocks and `models/entity/` for mobs and attachables. Bedrock actually scans the whole `models/` tree and finds geometry by identifier, so the folder is for your benefit. Stick to the convention anyway.

Then reference the geometry in the block's behaviour file:

JSON

```
"minecraft:geometry": {
  "identifier": "geometry.frost_lantern",
  "bone_visibility": {
    "handle": true
  }
}
```

`bone_visibility` lets you show or hide named bones per permutation — the standard way to make one block model serve several visual states without separate models.

⚠ Custom geometry means custom collision

As soon as you use `minecraft:geometry`, the block no longer gets an automatic full-cube collision box or selection box. You must supply `minecraft:collision_box` and `minecraft:selection_box` yourself, or the block will be walk-through-able and unclickable in confusing ways. Both take an `origin` and a `size` in the same pixel units as the model.

JSON

```
"minecraft:collision_box": {
  "origin": [-4, 0, -4],
  "size": [8, 10, 8]
},
"minecraft:selection_box": {
  "origin": [-4, 0, -4],
  "size": [8, 10, 8]
}
```

6.7 The block's inventory icon

A custom-geometry block automatically renders its model in the inventory and in the hand. If it renders as a plain cube or as nothing:

- Check the geometry identifier matches exactly.
- Check `material_instances` covers the faces your model uses.
- If you want a completely different look in the inventory, you can define an item-style icon via the block's `menu_category` and a matching entry — but in most cases the automatic render is what you want.

6.8 Block names: the language file

TEXT

```
tile.frostpack:frost_lantern.name=Frost Lantern
```

Note the prefix is `tile.` for blocks, not `block.` — a leftover from Minecraft's history that catches everyone out. Some newer block format versions accept `block.frostpack:frost_lantern` instead; if one does not work, try the other.

6.9 Other information for blocks

The block atlas is shared. All block textures across all active resource packs are packed into one atlas. A very large custom block texture forces the whole atlas larger. Keep block textures at 16×16 unless you have a reason.

Blocks and light. `minecraft:light_emission` (0–15) makes the block glow. `minecraft:light_dampening` (0–15) controls how much light it blocks. A glass-like block should have low dampening; forgetting this makes your "glass" cast solid shadows.

Flipbook (animated) textures. A block texture can animate. Create a tall PNG that is a vertical strip of frames (e.g. 16×64 for four frames) and register it in `frostpack_RP/textures/flipbook_textures.json`:

JSON

```
[
  {
    "flipbook_texture": "textures/blocks/frost_lantern",
    "atlas_tile": "frost_lantern",
    "ticks_per_frame": 4
  }
]
```

`atlas_tile` is the short name from `terrain_texture.json`. Add `"blend_frames": true` for a smooth cross-fade, or `false` for a hard frame switch.

Sounds. Blocks get their step/break/place sounds from the `minecraft:sound` value inside the description's `menu_category`-adjacent properties, or via `blocks.json`. The simplest route is the `"sound"` property in the block's description; valid values are vanilla sound group names like `stone`, `wood`, `grass`, `glass`, `metal`.

6.10 Extra tips for blocks

- **Prototype with vanilla textures.** Point your block at `"texture": "stone"` to test its shape and behaviour before the art is finished. Vanilla short names are all available in the atlas.
- **Build a test wall.** Place a 5×5 wall of your block and step back. Tiling problems are invisible on one block and obvious on twenty-five.
- **Match the vanilla brightness range.** Custom blocks often look "too saturated" or "too dark" next to vanilla. Place yours next to stone and cobblestone and compare.
- **Slabs, stairs and fences are not free.** Bedrock has no automatic "make a stair version" — each is a separate block with its own geometry, permutations and recipes. Budget for that.
- **Keep top/side/bottom in separate files** even if they are currently identical. Splitting them later means editing the material instances everywhere; having them separate from the start costs nothing.

7 Common mistakes byte one

This byte covers everything that goes wrong during **setup and art** — the sections from "First steps" through "Creating block textures". If something is broken and you have not written any behaviour JSON yet, the answer is almost certainly here.

Use it as a checklist: read the headings, and stop at the one that matches your symptom.

7.1 Setup and manifest mistakes

The pack does not appear in the game's pack list at all.

- `manifest.json` is not in the pack's root folder. It must sit directly inside `frostpack_BP/`, not inside a sub-folder. Unzipping an archive often creates a doubled folder:
`frostpack_BP/frostpack_BP/manifest.json`.
- The file is actually called `manifest.json.txt`. Windows hides extensions by default. Turn on `View > Show > File name extensions` in File Explorer and check.
- The pack folder is in `behavior_packs` instead of `development_behavior_packs`, or vice versa.
- `min_engine_version` is higher than your game version. The game silently hides packs that claim to need a newer engine.

Two packs conflict, or only one of two loads.

- Duplicate UUIDs. You copied a manifest from another pack, or duplicated your own pack folder without generating new UUIDs. All four UUIDs across your two packs must be unique.
- The header UUID and module UUID within a single manifest are the same. They must differ.

JSON errors in the manifest.

- `"format_version": "2"` — a string. It must be the number `2`, with no quotes.
- `"version": "1.0.0"` — a string. It must be the array `[1, 0, 0]`.
- A trailing comma after the last entry in an object or array. JSON forbids this; JavaScript allows it, so it looks fine to anyone who has written JS.
- Smart quotes. If you drafted JSON in Word or Notes, `"` may have become `"` and `"`. Minecraft cannot parse those. Always write JSON in a code editor.
- The file is saved as UTF-8 **with BOM**. VS Code shows the encoding in the status bar; click it and choose *Save with Encoding* → *UTF-8* (not "UTF-8 with BOM").

7.2 Naming and identifier mistakes

- **Capital letters or spaces** in a file name, folder name, or identifier. `Frost_Wraith.png` and `frost wraith` will fail, sometimes only on some devices, which is worse than failing everywhere.
- **Hyphens in identifiers.** `frost-wraith` is invalid. Use `frost_wraith`.
- **Missing namespace.** `frost_wraith` on its own is not a valid identifier; it must be `frostpack:frost_wraith`.
- **Using `minecraft:` as your namespace.** Reserved. It will either be rejected or overwrite vanilla content.
- **Namespace mismatch between files.** `frostpack:frost_wraith` in the behaviour pack and `frost_pack:frost_wraith` in the resource pack. Use project-wide search (`Ctrl+Shift+F`) on the identifier — every file that should mention it will appear, and any that spells it differently will not.
- **Renaming a bone after animating.** The animation still refers to the old bone name and silently does nothing.

7.3 Blockbench and geometry mistakes

The mob is invisible in game.

- You saved the `.bbmodel` but never did `File > Export > Bedrock Geometry`. Minecraft cannot read `.bbmodel`. This is the single most common art-stage mistake.
- The geometry `identifier` in the exported file does not match the string in the client entity file. Open the `.geo.json` and compare character by character — `geometry.frost_wraith` vs `geometry.frostwraith`.
- The model was exported in the wrong format — a Java model, or "Generic". Check the top of the file says `"minecraft:geometry"`.
- The `.geo.json` is in the behaviour pack instead of the resource pack.

The mob flickers, or vanishes when I look away slightly.

- Visible bounds are too small. Set `visible_bounds_width` and `visible_bounds_height` larger than the model, and set `visible_bounds_offset` so the box actually surrounds it.

Parts of the model flicker against each other (z-fighting).

- Two cubes occupy the exact same plane. Offset one by 0.01, or use the **Inflate** field on one of them.

The mob walks backwards / faces the wrong way.

- The model was built facing $-Z$. Bedrock mobs face $+Z$. Rotate the root bone 180° on Y, or rebuild.

Limbs rotate from the wrong place.

- Bone pivots are at the centre of the limb rather than at the joint. An arm must pivot at the shoulder. Fix the bone's Pivot values, not the cube's.

The whole model moves when I animate one bone / nothing moves.

- The bone hierarchy is wrong: cubes are all in one bone, or bones are not parented as intended. Check the Outliner tree, not the viewport.

The model is the wrong size in game.

- Model units are 16 per block. A "tall" mob at 40 units is 2.5 blocks. If it looks tiny, you probably built it at real-world scale in your head. Compare against a vanilla model opened side by side.

7.4 UV and texture-mapping mistakes

- **Texture size changed after UV mapping.** Changing texture width/height in Project Settings remaps everything. Set the size first.
- **Overlapping UVs.** Painting one cube changes another. Blockbench's UV panel shades occupied areas — use it.
- **UVs outside the texture bounds.** Faces sample garbage, or render as a stretched single colour.
- **Mixing Box UV and per-face UV expectations.** You cannot have both in one model. Decide before you start.
- **Mirrored limbs render their texture backwards.** Use the **Mirror UV** toggle on the mirrored cube.
- **Non-power-of-two texture sizes.** 50×70 will "work" and then look wrong on some devices. Use 16/32/64/128/256.

7.5 Texture image mistakes

- **Saved as JPG.** JPG has no transparency and adds compression artefacts to pixel art. Always PNG.
- **Saved without transparency.** In Paint.NET, an image flattened onto a white background has white pixels, not transparent ones. Delete the background layer's fill before drawing.
- **Resized with smoothing.** `Image > Resize` defaults to bicubic, which blurs pixel art beyond repair. Always set **Nearest Neighbor**.
- **Transparent pixels render as black on a mob.** The entity material is `entity` (opaque). Change it to `entity_alphatest` in the render controller.
- **An emissive mob is transparent instead of glowing.** With emissive materials, the alpha channel controls *glow*, not transparency. Fully-opaque pixels are normal; lower alpha glows. Make sure your "solid" areas are alpha 255.
- **The texture looks blurry in game.** Something upscaled it, or the file is not the size the model expects. Check `texture_width` / `texture_height` in the geometry match the PNG's actual dimensions.
- **File in the wrong folder.** Entity textures go in `textures/entity/`, items in `textures/items/`, blocks in `textures/blocks/`. The paths in your JSON must match exactly.

7.6 Item texture mistakes

- The item shows the purple/black missing-texture checker. The short name in `minecraft:icon` does not exist in `item_texture.json`, or the path inside `item_texture.json` is wrong.
- `.png` left on the end of the path in `item_texture.json`. Remove it — the game adds it.
- Backslashes in the path. Must be forward slashes. A backslash is also a JSON escape character, so the file may not even parse.
- `texture_name` is not `atlas.items`. It must be exactly that.
- Several `item_texture.json` files. Only one per pack; put all items in it.
- The icon has no margin and collides with the stack-count number. Leave the bottom-right corner comparatively clear.
- Icon drawn at 64×64. It will work, but it looks alien beside vanilla items and bloats the atlas.

7.7 Block texture mistakes

- `texture_name` is not `atlas.terrain` in `terrain_texture.json`.
- Missing `padding` / `num_mip_levels`. Without them you get thin bright lines between blocks at distance ("texture bleeding"). Use `8` and `4`.
- Obvious repeating pattern across a wall. One high-contrast pixel or an off-centre blob in the texture. Test with a 3×3 tiled preview before you commit.
- Transparent parts of the block are black. `render_method` is `opaque`. Change to `alpha_test` (hard edges) or `blend` (translucent).
- A "glass" block casts a full shadow. `minecraft:light_dampening` is at its default 15. Set it to 0.
- A lamp block has dark sides. `face_dimming` is on. Set `"face_dimming": false`.
- Custom-geometry block has no collision or cannot be selected. Using `minecraft:geometry` removes the default boxes. Add `minecraft:collision_box` and `minecraft:selection_box`.
- Custom block geometry extends past the block and looks wrong. Geometry outside the 16×16×16 volume is not lit or culled correctly and neighbours do not account for it. Keep it inside unless you genuinely need otherwise.
- The block model is offset by half a block. In the Bedrock Block format the origin is the centre of the block's floor, so a cube from `[0,0,0]` to `[16,16,16]` is shifted. Build from `[-8, 0, -8]` to `[8, 16, 8]`.

7.8 Language file mistakes

- The name shows as `item.frostpack:frost_shard` — the lang key is missing, misspelt, or in the wrong pack. Language files go in the resource pack, at `texts/en_US.lang`.
- Wrong prefix. Items use `item.`, blocks use `tile.`, entities use `entity.`. Blocks using `block.` is the classic error.

- **.name suffix present or missing.** Which is correct depends on the format version of the thing being named. If one does not work, try the other — there is no shame in it, everyone does.
- **The file is `en_US.lang.txt`.** Extensions again.
- **JSON syntax in a `.lang` file.** It is a plain `key=Value` text format. No braces, no quotes, no commas.
- **`texts/languages.json` missing.** Some setups need a `texts/languages.json` containing `["en_US"]` for the game to look at your lang files at all. Add it if names are not appearing and everything else looks right.

7.9 Workflow mistakes

- **Editing files while the world is open and expecting changes to appear.** Resource pack changes (models, textures, animations) require leaving the world and re-entering, and sometimes a full game restart.
- **Editing in two places.** You have a source copy and a copy in `development_resource_packs`, and you edited the wrong one. Pick one location and work only there.
- **Not turning on the content log.** `Settings > Creator > Enable Content Log GUI` plus verbose level. Without it you are guessing; with it, the game names the broken file.
- **Changing ten things then testing.** Change one thing, test, repeat.
- **No backups.** Commit to Git, or at minimum keep dated zips. You will break something badly at some point.

8 Creating folders

Bedrock finds most of your content by **scanning known folders**. There is no master index file listing your entities — you drop a JSON in `entities/` and the game picks it up. That makes the folder structure part of the code, and a file in the wrong folder is simply a file that does not exist as far as Minecraft is concerned.

This section is the complete map. Create only the folders you actually need; empty folders are harmless but clutter the project.

8.1 The behaviour pack structure

TEXT

```
frostpack_BP/  
├─ manifest.json  
├─ pack_icon.png  
├─ animation_controllers/  
├─ animations/  
├─ biomes/  
├─ blocks/  
├─ dialogue/  
├─ entities/  
├─ feature_rules/  
├─ features/  
├─ functions/  
├─ items/  
├─ loot_tables/  
│   └─ entities/  
│       └─ blocks/  
├─ recipes/  
├─ scripts/  
├─ spawn_rules/  
├─ structures/  
├─ trading/  
└─ texts/
```


Folder	Contents	Needed when
<code>animation_controllers/</code>	Behaviour-side animation controllers (<code>.json</code>) that run commands and drive server-side state	You want a mob to run commands, or to sequence behaviour states
<code>animations/</code>	Behaviour-side animations — timed lists of commands, not visual movement	You want timed command sequences on an entity
<code>biomes/</code>	Custom or modified biome definitions	You are changing world generation
<code>blocks/</code>	One <code>.json</code> per custom block	Any custom block
<code>dialogue/</code>	NPC dialogue trees	Using the NPC entity
<code>entities/</code>	One <code>.json</code> per custom mob — the "server entity" file with all AI, health, components	Any custom mob
<code>feature_rules/</code>	Rules deciding where features are placed in the world	Custom world generation (ore, trees, structures)
<code>features/</code>	The features themselves (ore blobs, trees, scatter patterns)	As above
<code>functions/</code>	<code>.mcfuction</code> files — lists of commands	Command-driven logic, <code>/function</code> calls, tick loops
<code>items/</code>	One <code>.json</code> per custom item	Any custom item
<code>loot_tables/</code>	What mobs and blocks drop, and chest contents	Anything that drops something
<code>recipes/</code>	Crafting, furnace, smithing, brewing recipes	Any craftable item or block
<code>scripts/</code>	JavaScript files using the Script API	Advanced logic beyond JSON
<code>spawn_rules/</code>	Where and when your mobs spawn naturally	Any mob that should spawn on its own
<code>structures/</code>	<code>.mcstructure</code> files to place in the world	Custom structures
<code>trading/</code>	Villager-style trade tables	Any trading entity
<code>texts/</code>	<code>en_US.lang</code> for behaviour-pack-side text (rare)	Rarely — most text lives in the RP

Two extra behaviour-pack files that sit in the root when needed:

- `functions/tick.json` — a manifest listing functions to run every tick:

JSON

```
{
  "values": [
    "frostpack/tick"
  ]
}
```

8.2 The resource pack structure

TEXT

```
frostpack_RP/
├─ manifest.json
├─ pack_icon.png
├─ animation_controllers/
├─ animations/
├─ attachables/
├─ entity/
├─ fogs/
├─ materials/
├─ models/
│   ├─ blocks/
│   └─ entity/
├─ particles/
├─ render_controllers/
├─ sounds/
├─ texts/
│   ├─ en_US.lang
│   └─ languages.json
├─ textures/
│   ├─ blocks/
│   ├─ entity/
│   ├─ items/
│   ├─ item_texture.json
│   ├─ terrain_texture.json
│   └─ flipbook_textures.json
├─ ui/
├─ blocks.json
├─ sounds.json
└─ sound_definitions.json
```

Folder / file	Contents	Needed when
<code>animation_controllers/</code>	Client animation controllers — decide which animation plays when	Any mob with more than one animation
<code>animations/</code>	Visual animations (<code>.animation.json</code>)	Any animated mob or attachable
<code>attachables/</code>	<code>.attachable.json</code> for 3D held/worn items	3D items
<code>entity/</code>	Client entity files (<code>.entity.json</code>) — links model, texture, animations, render controllers	Any custom mob
<code>fogs/</code>	Custom fog settings	Atmosphere work
<code>materials/</code>	Custom shader materials	Advanced; rarely needed
<code>models/blocks/</code>	Custom block geometry	Custom-shaped blocks
<code>models/entity/</code>	Mob and attachable geometry	Custom mobs and 3D items
<code>particles/</code>	Particle effect definitions	Custom particles
<code>render_controllers/</code>	Which geometry/texture/material to use, and when	Any custom mob
<code>sounds/</code>	The actual <code>.ogg</code> audio files	Custom sounds
<code>texts/en_US.lang</code>	All display names	Always
<code>texts/languages.json</code>	The list of languages your pack provides	Always — a one-line file
<code>textures/blocks/</code>	Block PNGs	Custom blocks
<code>textures/entity/</code>	Mob and attachable PNGs	Custom mobs
<code>textures/items/</code>	Item icon PNGs	Custom items
<code>textures/item_texture.json</code>	Item short-name lookup	Custom items
<code>textures/terrain_texture.json</code>	Block short-name lookup	Custom blocks
<code>textures/flipbook_textures.json</code>	Animated block textures	Animated blocks
<code>ui/</code>	Custom user-interface JSON	Advanced UI work

Folder / file	Contents	Needed when
<code>blocks.json</code>	Legacy block sound / texture mapping	Sometimes needed for block sounds
<code>sounds.json</code>	Which sounds play for which events on which entities	Custom mob sounds
<code>sound_definitions.json</code>	Maps sound names to <code>.ogg</code> files	Custom sounds

❗ `entity/` versus `entities/`

The **resource pack** folder is `entity/` (singular). The **behaviour pack** folder is `entities/` (plural). This is not a typo in the guide — it is genuinely inconsistent in Minecraft, and putting a client entity file in `entities/` is a very common mistake that produces no error message at all.

8.3 `texts/languages.json`

A tiny but frequently-missing file. Create `frostpack_RP/texts/languages.json`:

```
JSON
[
  "en_US"
]
```

Add more language codes as you translate. If your names are not appearing anywhere and the lang file looks correct, this file is usually why.

8.4 Which folders do you actually need?

For the running example — one mob, one item, one block — the minimum is:

Behaviour pack:

TEXT

```
frostpack_BP/  
├─ manifest.json  
├─ pack_icon.png  
├─ blocks/frost_lantern.json  
├─ entities/frost_wraith.json  
├─ items/frost_shard.json  
├─ loot_tables/entities/frost_wraith.json  
├─ recipes/frost_lantern.json  
└─ spawn_rules/frost_wraith.json
```

Resource pack:

TEXT

```
frostpack_RP/  
├─ manifest.json  
├─ pack_icon.png  
├─ animation_controllers/frost_wraith.animation_controllers.json  
├─ animations/frost_wraith.animation.json  
├─ entity/frost_wraith.entity.json  
├─ models/entity/frost_wraith.geo.json  
├─ models/blocks/frost_lantern.geo.json  
├─ render_controllers/frost_wraith.render_controllers.json  
├─ textures/entity/frost_wraith.png  
├─ textures/items/frost_shard.png  
├─ textures/blocks/frost_lantern.png  
├─ textures/item_texture.json  
├─ textures/terrain_texture.json  
└─ texts/  
    ├─ en_US.lang  
    └─ languages.json
```

That is a complete, advanced add-on's skeleton. Everything else on the big list is an extension of it.

8.5 Naming conventions inside folders

Bedrock finds files by scanning, so the file names themselves rarely matter — but conventions keep tools and humans working:

File type	Convention	Example
Server entity	<code><name>.json</code>	<code>entities/frost_wraith.json</code>
Client entity	<code><name>.entity.json</code>	<code>entity/frost_wraith.entity.json</code>
Geometry	<code><name>.geo.json</code>	<code>models/entity/frost_wraith.geo.json</code>
Animation	<code><name>.animation.json</code>	<code>animations/frost_wraith.animation.json</code>
Animation controller	<code><name>.animation_controllers.json</code>	<code>animation_controllers/frost_wraith.animation_controllers.json</code>
Render controller	<code><name>.render_controllers.json</code>	<code>render_controllers/frost_wraith.render_controllers.json</code>
Attachable	<code><name>.attachable.json</code>	<code>attachables/frost_blade.attachable.json</code>
Particle	<code><name>.particle.json</code>	<code>particles/frost_breath.particle.json</code>
Function	<code><name>.mcfuction</code>	<code>functions/frostpack/tick.mcfuction</code>
Loot table	<code><name>.json</code>	<code>loot_tables/entities/frost_wraith.json</code>

Sub-folders are allowed and encouraged for large packs. `entities/hostile/frost_wraith.json` works fine — the game scans recursively. The one exception is that **loot table** and **function** paths are referenced by path, so if you move `loot_tables/entities/frost_wraith.json`, you must update the reference in the entity file, and if you move a function, you must update every `/function` call.

8.6 Creating the folders quickly

In File Explorer, right-click → `New > Folder` repeatedly is fine. Faster options:

In VS Code: in the Explorer sidebar, click the **New Folder** icon, and type a path with slashes — `loot_tables/entities` — and it creates both levels at once. Same with **New File**: typing `entity/frost_wraith.entity.json` creates the folder and the file together.

In PowerShell, from inside your pack folder:

TEXT

```
mkdir animation_controllers, animations, entity, models\entity, models\blocks,
particles, render_controllers, textures\entity, textures\items, textures\blocks,
texts, sounds
```

In bridge. v2, the folders are created automatically when you create a file of that type — one of the strongest reasons to use it.

◆ Test the structure before you fill it

Create the folders, put your manifest and pack icon in place, and load the packs in game. An empty structure that loads cleanly is a known-good baseline. Every problem after that point is something you added.

9 Importing the textures

Your art was made in Blockbench and Paint.NET, which are separate applications. This section is about getting the models, textures and geometry into the resource pack correctly and wiring them together so Minecraft can find them. It is the bridge between "I have made art" and "the game shows my art".

Do this after the folders exist and before you write behaviour code, so you can see your mob standing there (motionless, stupid, but visible) while you code its brain.

9.1 What has to be imported

For a mob:

1. The **geometry** (`.geo.json`) — exported from Blockbench.
2. The **texture** (`.png`).
3. A **client entity file** (`.entity.json`) that names both.
4. A **render controller** that says how to draw it.

For an item: just the PNG plus an entry in `item_texture.json`.

For a block: the PNGs plus entries in `terrain_texture.json`, and if it has custom geometry, the `.geo.json`.

Items and blocks were covered in their own sections. This section focuses on the mob chain, which is the one with four links.

9.2 Exporting from Blockbench into the pack

Set the export target once. In Blockbench, `File > Export > Bedrock Geometry` opens a save dialog. Navigate to `frostpack_RP/models/entity/` and save as `frost_wraith.geo.json`. From then on, `Ctrl+E` (or `File > Export > Bedrock Geometry` again) reuses the path, so re-exporting after a model change takes one keystroke.

Export the texture separately. In the Textures panel, right-click your texture → **Save As** → `frostpack_RP/textures/entity/frost_wraith.png`. Once saved to a path, Blockbench's **Save** on that texture overwrites it in place.

Keep the `.bbmodel` outside the pack. Put your working files in something like `frostpack_sources/`. Shipping `.bbmodel` files in the pack does nothing except make the download larger.

★ Work in place

Point Blockbench's texture and geometry saves directly at the resource pack inside

`development_resource_packs`. Then your edit loop is: change model → `Ctrl+E` → leave world → re-enter → look. No copying, no forgetting.

9.3 Checking the exported geometry

Open `frost_wraith.geo.json` in VS Code. You are looking for four things:

JSON

```
{
  "format_version": "1.16.0",
  "minecraft:geometry": [
    {
      "description": {
        "identifier": "geometry.frost_wraith",
        "texture_width": 64,
        "texture_height": 64,
        "visible_bounds_width": 2,
        "visible_bounds_height": 2.5,
        "visible_bounds_offset": [0, 1, 0]
      },
      "bones": [
        {
          "name": "root",
          "pivot": [0, 0, 0]
        },
        {
          "name": "body",
          "parent": "root",
          "pivot": [0, 12, 0],
          "cubes": [
            { "origin": [-4, 12, -2], "size": [8, 12, 4], "uv": [16, 16] }
          ]
        }
      ]
    }
  ]
}
```

1. **format_version** is a recognised value (`1.12.0` or `1.16.0` are both current and fine; `1.16.0` supports per-face UV and more).
2. **identifier** is exactly what you expect. You will type this string again in the client entity file.

3. `texture_width` / `texture_height` match the actual PNG's dimensions. If they disagree, the texture will be scaled and misaligned.
4. The **bone tree** has the parents you intended.

9.4 Writing the client entity file

This is the file that ties everything together on the visual side. Create

`frostpack_RP/entity/frost_wraith.entity.json`:

```
JSON

{
  "format_version": "1.10.0",
  "minecraft:client_entity": {
    "description": {
      "identifier": "frostpack:frost_wraith",
      "materials": {
        "default": "entity_alphatest"
      },
      "textures": {
        "default": "textures/entity/frost_wraith"
      },
      "geometry": {
        "default": "geometry.frost_wraith"
      },
      "animations": {
        "idle": "animation.frost_wraith.idle",
        "walk": "animation.frost_wraith.walk",
        "attack": "animation.frost_wraith.attack"
      },
      "scripts": {
        "animate": [
          "idle",
          { "walk": "q.modified_move_speed > 0.1" }
        ]
      },
      "render_controllers": [
        "controller.render.frost_wraith"
      ],
      "spawn_egg": {
        "base_color": "#8ecae6",
        "overlay_color": "#023047"
      }
    }
  }
}
```

Field by field:

- **identifier** — must exactly match the identifier in the *behaviour* pack's entity file. This is the only link between them.
- **materials** — a map of short names to Minecraft material names. **default** is the one your render controller will usually ask for. Common values:
 - **entity** — fully opaque
 - **entity_alphatest** — hard-edged transparency (most custom mobs want this)
 - **entity_alphablend** — soft/partial transparency (ghosts)
 - **entity_emissive** — glowing; alpha channel controls glow, not transparency
 - **entity_emissive_alpha** — both transparency and glow
- **textures** — short names to texture paths. No **.png**, **forward slashes**, **relative to the pack root**.
- **geometry** — short names to geometry identifiers. The value is **geometry.something**, **not** a file path.
- **animations** — short names to animation identifiers (**animation.<mob>.<name>**). Again identifiers, not paths.
- **scripts.animate** — the list of animations (or animation controllers) that run. A bare string always plays; an object **{"name": "molang_condition"}** plays only when the condition is true.
- **render_controllers** — the controller identifiers to run.
- **spawn_egg** — optional. Either two tint colours as above, or **{"texture": "frost_wraith_spawn_egg"}** pointing at an entry in **item_texture.json**.

⚠ Short name versus identifier versus path

Three different kinds of string appear in this file and confusing them is the number-one cause of an invisible mob:

- **textures** values are **file paths** (**textures/entity/frost_wraith**)
- **geometry** values are **identifiers** (**geometry.frost_wraith**)
- **animations** values are **identifiers** (**animation.frost_wraith.idle**)
- **materials** values are **built-in material names** (**entity_alphatest**)

9.5 Writing the render controller

The render controller decides, at draw time, which geometry, texture and material to use. For a simple mob it is boilerplate; for a variant mob it becomes powerful.

Create **frostpack_RP/render_controllers/frost_wraith.render_controllers.json**:

JSON

```
{
  "format_version": "1.8.0",
  "render_controllers": {
    "controller.render.frost_wraith": {
      "geometry": "Geometry.default",
      "materials": [
        { "*" : "Material.default" }
      ],
      "textures": [
        "Texture.default"
      ]
    }
  }
}
```

The `Geometry.default`, `Material.default` and `Texture.default` strings refer to the **short names** you defined in the client entity file. The capitalisation of `Geometry` / `Material` / `Texture` is required.

A variant example — a mob with three colour variants selected by the `variant` property:

JSON

```
{
  "format_version": "1.8.0",
  "render_controllers": {
    "controller.render.frost_wraith": {
      "arrays": {
        "textures": {
          "Array.skins": [
            "Texture.default",
            "Texture.blue",
            "Texture.deep"
          ]
        }
      },
      "geometry": "Geometry.default",
      "materials": [
        { "*" : "Material.default" }
      ],
      "textures": [
        "Array.skins[q.variant]"
      ]
    }
  }
}
```

With `Texture.blue` and `Texture.deep` added to the client entity's `textures` map, and the behaviour pack setting `variant` to 0, 1 or 2 on spawn.

Emissive parts are done with a second material entry matched by bone name:

JSON

```
"materials": [  
  { "*": "Material.default" },  
  { "eyes": "Material.glow" }  
]
```

Here `eyes` is a **bone name** from your geometry, and `Material.glow` maps to `entity_emissive_alpha` in the client entity file. Order matters — later entries override earlier ones, and `*` should come first.

9.6 Importing sounds

Sounds are `.ogg` files (Vorbis). Convert from MP3/WAV with Audacity (free) via `File > Export > Export as OGG`.

1. Put the file in `frostpack_RP/sounds/mob/frost_wraith/idle.ogg`.
2. Register it in `frostpack_RP/sounds/sound_definitions.json`:

JSON

```
{  
  "format_version": "1.14.0",  
  "sound_definitions": {  
    "mob.frost_wraith.idle": {  
      "category": "neutral",  
      "sounds": [  
        { "name": "sounds/mob/frost_wraith/idle", "volume": 0.8 }  
      ]  
    }  
  }  
}
```

1. Attach it to the entity in `frostpack_RP/sounds.json`:

JSON

```
{
  "entity_sounds": {
    "entities": {
      "frost_wraith": {
        "volume": 1.0,
        "pitch": [0.8, 1.2],
        "events": {
          "ambient": "mob.frost_wraith.idle",
          "hurt": "mob.frost_wraith.hurt",
          "death": "mob.frost_wraith.death"
        }
      }
    }
  }
}
```

Note the entity key here is the **name without the namespace**. Mono `.ogg` files are required for positional 3D audio — a stereo file will play at constant volume regardless of distance.

9.7 Verifying the import

Load the world and run:

TEXT

```
/summon frostpack:frost_wraith
```

❗ This will fail until the behaviour pack entity file exists

The client entity file describes how to draw a mob; the behaviour pack file is what makes the mob *exist*. If you have not written it yet, jump to the Coding section, create the minimal server entity, and come back. Many people write both together for this reason.

Once it summons, check:

- The model appears with the right shape.
- The texture is applied and aligned.
- Nothing is a purple/black checker (missing texture) or a plain white box (missing geometry).
- Turning your head does not make it vanish (visible bounds).

9.8 Re-importing after changes

The loop, in order:

1. Change the model in Blockbench → `Ctrl+E` to re-export geometry.
2. Change the texture → right-click → **Save**.
3. In Minecraft: **leave the world**, then **re-enter**.
4. If the change still does not show, **fully quit and relaunch Minecraft**.

`/reload` refreshes behaviour pack data only. It does **not** refresh models, textures, animations or render controllers. Bedrock caches resource pack contents aggressively, and the only guaranteed refresh is a restart.

◆ Texture caching

If a texture stubbornly refuses to update even after a restart, change the file name (`frost_wraith_2.png`), update the client entity file, and test. If the new name works, it was a cache. Changing it back afterwards is usually fine.

10 Coding

This is the largest section of the guide, because this is where an add-on stops being a folder of pictures and starts being content. Everything here is written in JSON — no programming language required — except the optional Script API at the end, which uses JavaScript.

Read the first three sub-sections in order. After that, jump to whichever sub-section covers the thing you are building.

10.1 Where to put the code

Every kind of "code" in Bedrock has exactly one home. If a file is in the wrong pack or the wrong folder, it does not error — it simply does not exist.

What you are writing	Pack	Folder	File name
A mob's stats, AI and behaviour	Behaviour	entities/	frost_wraith.json
How that mob is drawn	Resource	entity/	frost_wraith.entity.json
Where the mob spawns	Behaviour	spawn_rules/	frost_wraith.json
What the mob drops	Behaviour	loot_tables/entities/	frost_wraith.json
The mob's trades	Behaviour	trading/	frost_wraith.json
An item's properties	Behaviour	items/	frost_shard.json
A block's properties	Behaviour	blocks/	frost_lantern.json
A crafting recipe	Behaviour	recipes/	frost_lantern.json
Visual animation	Resource	animations/	frost_wraith.animation.json
Which animation plays when (visual)	Resource	animation_controllers/	frost_wraith.animation_controllers.json
Which model/texture/material to draw	Resource	render_controllers/	frost_wraith.render_controllers.json
Timed command sequences on a mob	Behaviour	animations/	frost_wraith.animation.json
Behaviour state machines	Behaviour	animation_controllers/	frost_wraith.animation_controllers.json
Lists of commands	Behaviour	functions/	frostpack/summon_storm.mcfun ction
Particle effects	Resource	particles/	frost_breath.particle.json
JavaScript	Behaviour	scripts/	main.js

⚠ The two `animations` folders are different things

`BP/animations/` holds **behaviour animations** — timelines that run `/commands` and fire events.

`RP/animations/` holds **visual animations** — timelines that move bones. They have similar file formats and completely different purposes, and putting one in the other's folder does nothing at all.

10.2 JSON in ninety seconds

Every file you write is JSON. The whole language is six ideas:

JSON

```
{
  "a_string": "hello",
  "a_number": 42,
  "a_decimal": 3.5,
  "a_boolean": true,
  "a_list": [1, 2, 3],
  "an_object": {
    "nested": "value"
  }
}
```

The rules that break people:

- **Every key is in double quotes.** `{name: "x"}` is invalid; `{"name": "x"}` is valid.
- **Commas separate entries, and there is never a comma before a closing `}` or `]`.** Trailing commas are the single most common JSON error.
- **Brackets must balance.** `{` pairs with `}`, `[` pairs with `]`. VS Code highlights the matching bracket when your cursor is on one; use it.
- **"2" is not 2.** Some Minecraft fields want a string, some want a number, and they are strict about it.
- **No comments** in strict JSON. Bedrock tolerates `//` in most files, but not all — avoid them in anything you ship.
- **Escape backslashes and quotes** inside strings. In practice: never use backslashes; use forward slashes for paths.

If VS Code underlines something in red, fix it before testing. `Alt+Shift+F` reformats the document, and if the indentation comes out strange, you have a bracket in the wrong place.

10.3 Molang: Minecraft's little expression language

Molang is a tiny maths-and-logic language used anywhere Minecraft needs a value that changes at runtime — animation values, render controller choices, particle behaviour, filter conditions. You will meet it constantly, so learn it now.

A Molang expression is written as a **string** in JSON:

JSON

```
"rotation": ["math.sin(query.anim_time * 90) * 10", 0, 0]
```

Variable prefixes:

Prefix	Short	Meaning
<code>query.</code>	<code>q.</code>	Ask the game something (read-only)
<code>variable.</code>	<code>v.</code>	Your own variable, stored per entity
<code>temp.</code>	<code>t.</code>	A scratch variable within one expression
<code>context.</code>	<code>c.</code>	Contextual info supplied by the system running the expression
<code>math.</code>	—	The maths library

Queries you will actually use:

- `q.anim_time` — seconds since the current animation started
- `q.life_time` — seconds since the entity spawned
- `q.modified_move_speed` — how fast the entity is moving (0 when still)
- `q.ground_speed` — horizontal speed in blocks per second
- `q.is_on_ground`, `q.is_in_water`, `q.is_sneaking`, `q.is_sleeping`, `q.is_baby`, `q.is_angry`, `q.is_using_item`
- `q.health`, `q.max_health`
- `q.variant`, `q.mark_variant`, `q.skin_id` — three independent integer properties you can set from the behaviour pack and read here
- `q.property('frostack:charge')` — read a custom entity property (the modern replacement for variant hacks)
- `q.body_x_rotation`, `q.head_x_rotation`, `q.target_x_rotation` — angles, useful for look-at animations
- `q.time_of_day`, `q.moon_phase`
- `q.block_property('frostack:lit')` — read a block state, in block contexts

Maths functions: `math.sin`, `math.cos` (in degrees, not radians), `math.abs`, `math.clamp(v, min, max)`, `math.lerp(a, b, t)`, `math.random(a, b)`, `math.floor`, `math.ceil`, `math.round`, `math.mod`, `math.pow`, `math.sqrt`, `math.min`, `math.max`, `math.die_roll(n, low, high)`.

Operators: `+` `-` `*` `/` `( )`, comparisons `<` `>` `<=` `>=` `==` `!=`, logic `&&` `||` `!`, and the ternary `condition ? value_if_true : value_if_false`. There is also the null-coalescing `??`.

Examples that do real work:

TEXT

```
math.sin(q.anim_time * 200) * 15
```

A limb swinging ± 15 degrees, cycling faster as the multiplier rises.

TEXT

```
q.modified_move_speed > 0.1
```

True while the mob is moving — the standard condition for a walk animation.

TEXT

```
q.is_in_water ? 1.0 : 0.0
```

Returns 1 in water, 0 out of it.

TEXT

```
v.wobble = math.sin(q.life_time * 90) * 2;
```

Assigns a variable. Note the semicolon — **assignments need one, plain expressions do not.**

★ Degrees, not radians

`math.sin(90)` is 1 in Molang. Every trig function takes degrees. If your animation wobbles almost imperceptibly, you probably passed radians.

⚠ Client versus server Molang

Molang in the **resource pack** (animations, render controllers, particles) runs on the client and can only see client-visible state. Molang in the **behaviour pack** (filters, some components) runs on the server. Some queries exist on only one side. If a query "does nothing", check which side you are on — this is one of the hardest bugs to diagnose because there is no error message.

10.4 Coding mobs: the server entity file

This is the heart of any add-on. Create `frostpack_BP/entities/frost_wraith.json`.

10.4.1 The overall shape

JSON

```
{
  "format_version": "1.21.0",
  "minecraft:entity": {
    "description": {
      "identifier": "frostdpack:frost_wraith",
      "is_spawnable": true,
      "is_summonable": true,
      "is_experimental": false
    },
    "component_groups": { },
    "components": { },
    "events": { }
  }
}
```

- **description** — identity and top-level flags.
- **components** — everything that is *always* true about this mob.
- **component_groups** — named bundles of components that can be added and removed at runtime. This is how a mob changes state: baby→adult, calm→angry, tamed→untamed.
- **events** — named triggers that add and remove component groups.

That four-part structure is the whole system. Components define what is true now; events swap groups to change what is true.

10.4.2 The description block

JSON

```
"description": {
  "identifier": "frostpack:frost_wraith",
  "is_spawnable": true,
  "is_summonable": true,
  "is_experimental": false,
  "properties": {
    "frostpack:charge": {
      "type": "int",
      "range": [0, 3],
      "default": 0,
      "client_sync": true
    },
    "frostpack:enraged": {
      "type": "bool",
      "default": false,
      "client_sync": true
    }
  },
  "runtime_identifier": "minecraft:zombie",
  "spawn_category": "monster"
}
```

- `is_spawnable` — a spawn egg is generated for it.
- `is_summonable` — `/summon` works on it.
- `properties` — **entity properties**, the modern way to store state on a mob. Types are `int`, `float`, `bool` and `enum`. Setting `client_sync: true` makes the value readable from resource-pack Molang via `q.property('frostpack:charge')`, which is how you drive animations from behaviour state. This replaced the old trick of abusing `variant` and `mark_variant`, and you should use it.
- `runtime_identifier` — makes your entity inherit a vanilla entity's hardcoded engine behaviour (a zombie burning in daylight, a creeper exploding, a boat being rideable). Powerful, but it also drags in behaviour you may not want, and some values cause crashes. Use sparingly and test.
- `spawn_category` — `monster`, `creature`, `ambient`, `water_creature`, `misc`. Affects mob-cap accounting.

10.4.3 Core components every mob needs

JSON

```
"components": {
  "minecraft:type_family": {
    "family": ["frost_wraith", "monster", "undead"]
  },
  "minecraft:collision_box": {
    "width": 0.8,
    "height": 2.0
  },
  "minecraft:health": {
    "value": 30,
    "max": 30
  },
  "minecraft:movement": {
    "value": 0.25
  },
  "minecraft:navigation.walk": {
    "can_path_over_water": false,
    "avoid_water": true,
    "avoid_damage_blocks": true
  },
  "minecraft:movement.basic": {},
  "minecraft:jump.static": {},
  "minecraft:physics": {},
  "minecraft:pushable": {
    "is_pushable": true,
    "is_pushable_by_piston": true
  },
  "minecraft:attack": {
    "damage": 5
  },
  "minecraft:follow_range": {
    "value": 32,
    "max": 48
  },
  "minecraft:knockback_resistance": {
    "value": 0.2
  },
  "minecraft:nameable": {},
  "minecraft:loot": {
    "table": "loot_tables/entities/frost_wraith.json"
  },
  "minecraft:despawn": {
    "despawn_from_distance": {}
  }
}
```

```
}
}
```

Notes on the ones that trip people up:

- **minecraft:physics** is mandatory. Without it, the mob has no gravity or collision and floats where it spawned. It is easy to forget because it takes no parameters.
- **type_family** is how filters find your mob. Other mobs' targeting filters check families, not identifiers. Give your mob a unique family plus the standard ones (`monster` , `undead` , `mob`) so vanilla interactions work — for example, adding `undead` makes Smite work against it.
- **Movement is three components, not one.** `minecraft:movement` (the speed value), `minecraft:movement.basic` (the movement *type* — how it physically moves), and `minecraft:navigation.walk` (the pathfinder — how it decides where to step). All three are needed. Flying mobs use `movement.fly` + `navigation.fly` or `navigation.hover`.
- **minecraft:loot** 's path is relative to the behaviour pack root and **does** include `.json` . This is inconsistent with texture paths; yes, it is annoying.
- **collision_box** is in blocks, not model pixels. A 2-block-tall model needs `"height": 2.0` .

Movement type / navigation pairings:

Mob style	Movement component	Navigation component
Walking	<code>minecraft:movement.basic</code>	<code>minecraft:navigation.walk</code>
Flying (bird)	<code>minecraft:movement.fly</code>	<code>minecraft:navigation.fly</code>
Hovering (bee, ghost)	<code>minecraft:movement.hover</code>	<code>minecraft:navigation.hover</code>
Swimming	<code>minecraft:movement.swim</code>	<code>minecraft:navigation.swim</code>
Amphibious	<code>minecraft:movement.amphibious</code>	<code>minecraft:navigation.generic</code>
Jumping (rabbit, slime)	<code>minecraft:movement.jump</code>	<code>minecraft:navigation.walk</code>
Climbing spider	<code>minecraft:movement.basic</code>	<code>minecraft:navigation.climb</code>

10.5 Behaviours (mob AI)

AI in Bedrock is a **priority list of goals**. Every tick the game walks the list from priority 0 upward and runs the first goal whose conditions are met. Lower numbers are higher priority.

JSON

```
"minecraft:behavior.float": { "priority": 0 },
"minecraft:behavior.melee_attack": {
  "priority": 2,
  "track_target": true,
  "reach_multiplier": 1.5,
  "speed_multiplier": 1.2
},
"minecraft:behavior.random_stroll": {
  "priority": 6,
  "speed_multiplier": 0.8,
  "xz_dist": 10,
  "y_dist": 7
},
"minecraft:behavior.look_at_player": {
  "priority": 7,
  "look_distance": 8,
  "probability": 0.02
},
"minecraft:behavior.random_look_around": { "priority": 8 }
```

Targeting is separate from attacking. You need one behaviour to *choose* a target and another to *act* on it:

JSON

```
"minecraft:behavior.nearest_attackable_target": {
  "priority": 1,
  "must_see": true,
  "must_see_forget_duration": 6.0,
  "reselect_targets": true,
  "within_radius": 25,
  "entity_types": [
    {
      "filters": {
        "any_of": [
          { "test": "is_family", "subject": "other", "value": "player" },
          { "test": "is_family", "subject": "other", "value": "villager" }
        ]
      },
      "max_dist": 25
    }
  ]
}
```

Without a target-selecting behaviour, `melee_attack` never fires, because there is nothing to attack. This is a very common "my mob is passive" cause.

The behaviours worth knowing (there are over a hundred; these cover most add-ons):

Behaviour	Does
<code>float</code>	Swims to the surface instead of drowning. Always give this priority 0.
<code>nearest_attackable_target</code>	Picks a target matching a filter
<code>hurt_by_target</code>	Retaliates against whatever hit it
<code>melee_attack</code>	Walks up and hits the target
<code>ranged_attack</code>	Fires projectiles at the target (needs <code>minecraft:shooter</code>)
<code>random_stroll</code>	Wanders
<code>look_at_player</code> / <code>random_look_around</code>	Idle head movement — makes a mob feel alive
<code>panic</code>	Runs away when hurt
<code>avoid_mob_type</code>	Flees specific mobs
<code>flee_sun</code> / <code>hurt_by_target</code>	Sun avoidance for undead
<code>move_towards_target</code>	Approaches without attacking
<code>stay_while_sitting</code> / <code>follow_owner</code>	Tamed pet behaviour
<code>tempt</code>	Follows a player holding a given item
<code>breed</code> / <code>mount_pathing</code> / <code>rideable</code>	Breeding and riding
<code>delayed_attack</code>	Melee with a wind-up, so an attack animation can play
<code>send_event</code>	Fires an entity event on a timer — the bridge from AI to your own logic
<code>summon_entity</code>	Spawns other entities as an attack
<code>charge_attack</code>	Rushes the target
<code>knockback_roar</code>	Area knockback
<code>nearest_prioritized_attackable_target</code>	Like <code>nearest_attackable_target</code> but with per-type priorities

★ Read the vanilla files

Every one of these is used in `bedrock-samples/behavior_pack/entities/`. When you want a mob that behaves like a vanilla one, open that mob's file and copy the goal list. The vanilla zombie, skeleton, wolf and villager between them demonstrate nearly every pattern.

Priorities in practice. A workable convention:

- `0` — `float` (never drown)
- `1` — targeting behaviours
- `2-3` — attacking / fleeing
- `4-5` — task behaviours (breed, tempt, follow owner)
- `6-7` — wander, look at player
- `8+` — idle fallbacks

10.6 Component groups and events

This is the mechanism for anything that changes. A component group is a named bundle; an event adds and removes groups.

```

"component_groups": {
  "frostpack:baby": {
    "minecraft:is_baby": {},
    "minecraft:scale": { "value": 0.5 },
    "minecraft:ageable": {
      "duration": 1200,
      "grow_up": { "event": "minecraft:ageable_grow_up", "target": "self" },
      "feed_items": ["frostpack:frost_shard"]
    }
  },
  "frostpack:adult": {
    "minecraft:experience_reward": { "on_death": "10" },
    "minecraft:scale": { "value": 1.0 },
    "minecraft:behavior.breed": { "priority": 3 }
  },
  "frostpack:enraged": {
    "minecraft:movement": { "value": 0.4 },
    "minecraft:attack": { "damage": 9 },
    "minecraft:timer": {
      "time": [10, 15],
      "looping": false,
      "time_down_event": { "event": "frostpack:calm_down" }
    }
  }
},

"events": {
  "minecraft:entity_spawned": {
    "randomize": [
      { "weight": 80, "add": { "component_groups": ["frostpack:adult"] } },
      { "weight": 20, "add": { "component_groups": ["frostpack:baby"] } }
    ]
  },
  "minecraft:ageable_grow_up": {
    "remove": { "component_groups": ["frostpack:baby"] },
    "add": { "component_groups": ["frostpack:adult"] }
  },
  "frostpack:enrage": {
    "add": { "component_groups": ["frostpack:enraged"] },
    "run_command": {
      "command": ["playsound mob.wither.spawn @a"]
    }
  },
  "frostpack:calm_down": {
    "remove": { "component_groups": ["frostpack:enraged"] }
  }
}

```

```
}  
}
```

Built-in events the game fires for you: `minecraft:entity_spawned`, `minecraft:entity_born`, `minecraft:entity_transformed`, `minecraft:on_prime` (TNT-like), `minecraft:ageable_grow_up`, `minecraft:on_tame`.

Event modifiers you can use:

- `add` / `remove` — component groups.
- `randomize` — weighted random choice between outcomes.
- `sequence` — run several outcomes in order, each optionally filtered.
- `set_property` — set an entity property (the modern state mechanism).
- `run_command` — run commands (available in current format versions).
- `trigger` — fire another event.
- `filters` — only run this branch if the filter passes.
- `emit_particle` / `play_sound` in some contexts.

Firing your own events. Four common routes:

1. `minecraft:timer` — after N seconds.
2. `minecraft:damage_sensor` — when damaged in a particular way.
3. `minecraft:behavior.send_event` — as an AI goal, so it fires when the mob is in a given situation.
4. `/event entity @e[type=frostpack:frost_wraith] frostpack:enrage` — from a command.
5. `minecraft:interact` — when a player right-clicks with a given item.

A damage sensor example — take triple damage from fire and enrage below half health:

JSON

```
"minecraft:damage_sensor": {
  "triggers": [
    {
      "cause": "fire",
      "damage_multiplier": 3.0,
      "deals_damage": true
    },
    {
      "on_damage": {
        "filters": {
          "all_of": [
            { "test": "has_component", "operator": "!=", "value":
"minecraft:movement" },
            { "test": "health", "operator": "<=", "value": 15 }
          ]
        },
        "event": "frostpack:enrage"
      },
      "deals_damage": true
    }
  ]
}
```

An interaction example — right-click with a frost shard to charge the mob:

JSON

```
"minecraft:interact": {
  "interactions": [
    {
      "on_interact": {
        "filters": {
          "all_of": [
            { "test": "is_family", "subject": "other", "value": "player" },
            { "test": "has_equipment", "subject": "other", "domain": "hand", "value":
"frostpack:frost_shard" }
          ]
        },
        "event": "frostpack:charge_up",
        "target": "self"
      },
      "use_item": true,
      "hurt_item": 0,
      "play_sounds": "beacon.power",
      "interact_text": "action.interact.charge"
    }
  ]
}
```

10.7 Filters

Filters appear everywhere — in behaviours, events, interactions, spawn rules, loot tables. They all share one syntax:

JSON

```
{
  "test": "is_family",
  "subject": "other",
  "operator": "==",
  "value": "player"
}
```

- **test** — what to check. Common: `is_family`, `has_component`, `health`, `is_underwater`, `is_in_village`, `weather`, `is_daytime`, `hourly_clock_time`, `has_equipment`, `is_riding`, `random_chance`, `distance_to_nearest_player`, `is_block`, `enum_property`, `int_property`, `bool_property`, `is_difficulty`, `is_game_rule`, `has_tag`, `is_biome`, `is_target`, `is_owner`, `on_ground`, `is_sneaking`, `is_moving`.

- **subject** — who is being tested: `self` (default), `other`, `target`, `parent`, `player`, `block`, `damager`.
- **operator** — `==`, `!=`, `<`, `<=`, `>`, `>=`. Defaults to `==`.
- **value** — what to compare against.

Combine them with `all_of` (AND), `any_of` (OR) and `none_of` (NOT):

JSON

```
"filters": {
  "all_of": [
    { "test": "is_daytime", "value": false },
    {
      "any_of": [
        { "test": "is_biome", "value": "ice_plains" },
        { "test": "is_biome", "value": "cold_taiga" }
      ]
    },
    { "test": "has_component", "operator": "!=", "value": "minecraft:is_baby" }
  ]
}
```

Filters nest arbitrarily. Note that `all_of` / `any_of` / `none_of` take an **array**, and a single filter object can be used directly without a wrapper.

10.8 Spawning

Natural spawning is defined in a separate file: `frostpack_BP/spawn_rules/frost_wraith.json`.


```

{
  "format_version": "1.8.0",
  "minecraft:spawn_rules": {
    "description": {
      "identifier": "frostpack:frost_wraith",
      "population_control": "monster"
    },
    "conditions": [
      {
        "minecraft:spawns_on_surface": {},
        "minecraft:brightness_filter": {
          "min": 0,
          "max": 7,
          "adjust_for_weather": true
        },
        "minecraft:difficulty_filter": {
          "min": "easy",
          "max": "hard"
        },
        "minecraft:weight": {
          "default": 30
        },
        "minecraft:herd": {
          "min_size": 1,
          "max_size": 3
        },
        "minecraft:biome_filter": {
          "any_of": [
            { "test": "has_biome_tag", "operator": "==", "value": "cold" },
            { "test": "has_biome_tag", "operator": "==", "value": "frozen" }
          ]
        },
        "minecraft:height_filter": {
          "min": 60,
          "max": 200
        },
        "minecraft:spawns_lava": false
      }
    ]
  }
}

```

Field notes:

- **population_control** — which mob cap this counts against: `monster`, `animal`, `water_animal`, `ambient`, `pillager`, `cat`, `villager`. Get this wrong and your mob either never spawns (cap already full) or floods the world.
- **minecraft:weight** — relative chance against other mobs eligible in the same place. Vanilla zombies are around 100; a rare mob might be 5. This is *relative*, so the number only means something compared to what else can spawn there.
- **Placement conditions** — `spawns_on_surface`, `spawns_underground`, `spawns_underwater`, `spawns_on_block_filter` (spawn only on specific blocks), `spawns_on_block_prevented_filter`.
- **minecraft:herd** — group size, and optionally an `event` to fire on the group leader.
- **minecraft:biome_filter** — uses **biome tags**, not biome names, in current formats. Useful tags: `cold`, `frozen`, `warm`, `hot`, `ocean`, `beach`, `desert`, `forest`, `jungle`, `mesa`, `mountain`, `plains`, `river`, `savanna`, `swamp`, `taiga`, `nether`, `the_end`, `overworld`, `monster`, `animal`, `spawn_zombie`, `spawn_pigs`.
- **Multiple conditions entries** = multiple independent spawn scenarios (e.g. one for surface at night, one for underground at any time).

⚠ Spawning is the hardest thing to test

Natural spawn rules can take many minutes of real play to demonstrably work, and they interact with the mob cap, the difficulty, the light level and how far the player is from the spawn point. Test with `/summon` first to prove the mob works, and only then debug spawning. To test spawn rules quickly: make a superflat world, `/gamerule doMobSpawning true`, `/kill @e[type!=player]`, set night with `/time set midnight`, stand still and wait — spawning happens 24–128 blocks from the player, so run 50 blocks away and look back.

10.9 Loot tables

`frostpack_BP/loot_tables/entities/frost_wraith.json`:

```

{
  "pools": [
    {
      "rolls": 1,
      "entries": [
        {
          "type": "item",
          "name": "frostopack:frost_shard",
          "weight": 3,
          "functions": [
            {
              "function": "set_count",
              "count": { "min": 1, "max": 3 }
            },
            {
              "function": "looting_enchant",
              "count": { "min": 0, "max": 2 }
            }
          ]
        },
        {
          "type": "item",
          "name": "minecraft:ice",
          "weight": 1
        },
        {
          "type": "empty",
          "weight": 2
        }
      ]
    },
    {
      "rolls": 1,
      "conditions": [
        { "condition": "killed_by_player" },
        { "condition": "random_chance_with_looting", "chance": 0.05,
"looting_multiplier": 0.02 }
      ],
      "entries": [
        {
          "type": "item",
          "name": "frostopack:frost_blade",
          "weight": 1
        }
      ]
    }
  ]
}

```

```
]
}
```

- **Pools roll independently.** Each pool produces its own drops, so use one pool for common drops and another for a rare bonus.
- **rolls** can be a number or `{"min": 1, "max": 3}`.
- **weight** is relative within a pool. An `"type": "empty"` entry with weight lets a pool sometimes produce nothing.
- **Common functions:** `set_count`, `set_damage`, `set_data` (for block variants), `enchant_with_levels`, `enchant_randomly`, `specific_enchants`, `looting_enchant`, `set_name`, `set_lore`, `random_block_state`, `furnace_smelt`.
- **Common conditions:** `killed_by_player`, `random_chance`, `random_chance_with_looting`, `entity_properties`, `has_mark_variant`, `match_tool`.

For a block, put the file in `loot_tables/blocks/` and reference it from the block's `minecraft:loot` component.

① Silk touch and fortune on blocks

Use `match_tool` conditions with `enchancements` to make a silk-touch pool that drops the block itself and a normal pool that drops something else. Structure it as two pools with mutually exclusive conditions.

10.10 Trade tables

`frostpack_BP/trading/frost_merchant.json` :

JSON

```
{
  "tiers": [
    {
      "groups": [
        {
          "num_to_select": 2,
          "trades": [
            {
              "wants": [
                { "item": "minecraft:emerald", "quantity": { "min": 3, "max": 5 } }
              ],
              "gives": [
                { "item": "frostpack:frost_shard", "quantity": 1 }
              ],
              "trader_exp": 2,
              "max_uses": 12,
              "reward_exp": true
            }
          ]
        }
      ]
    },
    {
      "total_exp_required": 10,
      "groups": [ ... ]
    }
  ]
}
```

Attach it to the entity with:

JSON

```
"minecraft:trade_table": {
  "display_name": "entity.frostpack:frost_merchant.name",
  "table": "trading/frost_merchant.json",
  "convert_trades_economy": true
}
```

The mob also needs `minecraft:economy_trade_table` or the trading behaviours (`minecraft:behavior.trade_with_player`, `minecraft:behavior.trade_interest`) and `minecraft:trusting` -style setup depending on the style you want. Copy the vanilla villager or wandering trader as a base.

10.11 Item properties

Items live in `frostpack_BP/items/frost_shard.json`. The modern data-driven item format looks like this:

JSON

```
{
  "format_version": "1.21.30",
  "minecraft:item": {
    "description": {
      "identifier": "frostpack:frost_shard",
      "menu_category": {
        "category": "items",
        "group": "itemGroup.name.miscFood",
        "is_hidden_in_commands": false
      }
    },
    "components": {
      "minecraft:icon": {
        "texture": "frost_shard"
      },
      "minecraft:display_name": {
        "value": "item.frostpack:frost_shard"
      },
      "minecraft:max_stack_size": 64,
      "minecraft:hand_equipped": false,
      "minecraft:glint": false,
      "minecraft:stacked_by_data": true,
      "minecraft:should_despawn": true,
      "minecraft:fuel": {
        "duration": 0
      }
    }
  }
}
```

`minecraft:icon`'s `texture` value is the **short name** from `item_texture.json`, not a path. In some format versions `minecraft:icon` accepts a bare string (`"minecraft:icon": "frost_shard"`); the object form above is current and safer.

10.11.1 Item component reference

Basic properties

Component	Purpose
<code>minecraft:icon</code>	The inventory sprite (short name from <code>item_texture.json</code>)
<code>minecraft:display_name</code>	Name, literal or translation key
<code>minecraft:max_stack_size</code>	1–64. Anything with durability must be 1
<code>minecraft:hand_equipped</code>	<code>true</code> makes it render like a tool in hand (angled) rather than flat
<code>minecraft:glint</code>	Permanent enchantment shimmer
<code>minecraft:foil</code>	Alias of glint in some versions
<code>minecraft:allow_off_hand</code>	Can be placed in the off-hand slot
<code>minecraft:can_destroy_in_creative</code>	Whether creative left-click breaks blocks with it held
<code>minecraft:stacked_by_data</code>	Whether data values keep stacks separate
<code>minecraft:should_despawn</code>	Whether the dropped item despawns

Tools and weapons

JSON

```
"minecraft:durability": {
  "max_durability": 512,
  "damage_chance": { "min": 10, "max": 50 }
},
"minecraft:damage": 8,
"minecraft:enchantable": {
  "value": 14,
  "slot": "sword"
},
"minecraft:repairable": {
  "repair_items": [
    {
      "items": ["frostpack:frost_shard"],
      "repair_amount": "context.other->query.remaining_durability + 0.05 *
context.other->query.max_durability"
    }
  ]
},
"minecraft:digger": {
  "use_efficiency": true,
  "destroy_speeds": [
    {
      "block": { "tags": "q.any_tag('stone', 'metal')" },
      "speed": 9
    }
  ]
},
"minecraft:hand_equipped": true,
"minecraft:max_stack_size": 1
```

`minecraft:enchantable` slot values include `sword`, `axe`, `pickaxe`, `shovel`, `hoe`, `bow`, `crossbow`, `armor_head`, `armor_torso`, `armor_legs`, `armor_feet`, `elytra`, `trident`, `fishing_rod`, `shears`, `flintsteel`, `all`. Higher `value` means better enchantments at the table.

Armour

JSON

```
"minecraft:wearable": {
  "slot": "slot.armor.head",
  "protection": 3
},
"minecraft:armor": {
  "protection": 3
},
"minecraft:durability": { "max_durability": 300 },
"minecraft:max_stack_size": 1
```

Custom armour also needs an **attachable** in the resource pack to render on the player, plus armour geometry. Slots are `slot.armor.head`, `slot.armor.chest`, `slot.armor.legs`, `slot.armor.feet`, `slot.weapon.offhand`.

Food

JSON

```
"minecraft:food": {
  "nutrition": 4,
  "saturation_modifier": "normal",
  "can_always_eat": true,
  "using_converts_to": "minecraft:glass_bottle"
},
"minecraft:use_modifiers": {
  "use_duration": 1.6,
  "movement_modifier": 0.35
},
"minecraft:use_animation": "eat"
```

`saturation_modifier` accepts `poor`, `low`, `normal`, `good`, `max`, `supernatural`, or a number.

`use_animation` accepts `eat`, `drink`, `bow`, `block`, `camera`, `crossbow`, `none`.

To give an effect on eating, add `minecraft:on_consume` in newer formats, or handle it with a script or an `on_use` event running a command.

Placement and projectiles

JSON

```
"minecraft:block_placer": {
  "block": "frostpack:frost_lantern",
  "use_on": ["minecraft:stone", "minecraft:dirt"]
},
"minecraft:projectile": {
  "projectile_entity": "frostpack:frost_bolt",
  "minimum_critical_power": 0.5
},
"minecraft:throwable": {
  "do_swing_animation": true,
  "launch_power_scale": 1.5,
  "max_draw_duration": 0,
  "max_launch_power": 1,
  "scale_power_by_draw_duration": false
},
"minecraft:shooter": {
  "ammunition": [
    {
      "item": "frostpack:frost_bolt",
      "use_offhand": true,
      "search_inventory": true,
      "use_in_creative": true
    }
  ],
  "max_draw_duration": 1.0,
  "charge_on_draw": true,
  "scale_power_by_draw_duration": true
}
```

Cooldowns, tags and misc

JSON

```
"minecraft:cooldown": {
  "category": "frost_magic",
  "duration": 3.5
},
"minecraft:tags": {
  "tags": ["minecraft:is_sword", "frostpack:is_frost_tool"]
},
"minecraft:fuel": {
  "duration": 40
},
"minecraft:entity_placer": {
  "entity": "frostpack:frost_wraith",
  "dispense_on": ["minecraft:snow"],
  "use_on": ["minecraft:ice"]
}
```

`minecraft:tags` is important: vanilla systems check tags like `minecraft:is_pickaxe` to decide whether a block breaks properly, and your own filters can check `q.any_tag(...)`.

10.11.2 Custom item behaviour

Data-driven components cover a lot, but "when the player uses this item, do X" needs one of:

- `minecraft:custom_components` — attaches a script-defined component to the item. This is the modern route and requires the Script API and, depending on version, an experimental toggle.
- A script event listener on `ItemUseAfterEvent` — see the Script API sub-section.
- The old command trick — give the item a cooldown and run a `tick.json` function that detects the cooldown state. Ugly, but requires no scripting.

10.12 Coding blocks

Blocks live in `frostpack_BP/blocks/frost_lantern.json` and have the most capable data-driven format of the three.

```
{
  "format_version": "1.21.30",
  "minecraft:block": {
    "description": {
      "identifier": "frostpack:frost_lantern",
      "menu_category": {
        "category": "construction",
        "group": "itemGroup.name.lantern"
      },
    },
    "states": {
      "frostpack:lit": [false, true],
      "frostpack:level": [0, 1, 2, 3]
    },
    "traits": {
      "minecraft:placement_direction": {
        "enabled_states": ["minecraft:cardinal_direction"],
        "y_rotation_offset": 180
      }
    }
  },
  "components": {
    "minecraft:geometry": {
      "identifier": "geometry.frost_lantern"
    },
    "minecraft:material_instances": {
      "*": {
        "texture": "frost_lantern",
        "render_method": "alpha_test",
        "face_dimming": false,
        "ambient_occlusion": false
      }
    },
    "minecraft:collision_box": {
      "origin": [-4, 0, -4],
      "size": [8, 10, 8]
    },
    "minecraft:selection_box": {
      "origin": [-4, 0, -4],
      "size": [8, 10, 8]
    },
    "minecraft:destructible_by_mining": {
      "seconds_to_destroy": 1.5
    },
    "minecraft:destructible_by_explosion": {
      "explosion_resistance": 5
    },
    "minecraft:friction": 0.6,
  }
}
```

```

"minecraft:light_dampening": 0,
"minecraft:light_emission": 0,
"minecraft:map_color": "#8ecae6",
"minecraft:loot": "loot_tables/blocks/frost_lantern.json",
"minecraft:flammable": {
  "catch_chance_modifier": 0,
  "destroy_chance_modifier": 0
},
"minecraft:placement_filter": {
  "conditions": [
    {
      "allowed_faces": ["up"],
      "block_filter": [
        { "tags": "!q.any_tag('minecraft:plant')" }
      ]
    }
  ]
},
"permutations": [
  {
    "condition": "q.block_state('frostpack:lit') == true",
    "components": {
      "minecraft:light_emission": 14,
      "minecraft:material_instances": {
        "*": {
          "texture": "frost_lantern_lit",
          "render_method": "alpha_test",
          "face_dimming": false,
          "ambient_occlusion": false
        }
      }
    }
  },
  {
    "condition": "q.block_state('frostpack:level') >= 2",
    "components": {
      "minecraft:geometry": {
        "identifier": "geometry.frost_lantern_tall"
      }
    }
  }
],
"events": {
  "frostpack:toggle_light": {
    "set_block_state": {
      "frostpack:lit": "!q.block_state('frostpack:lit')"
    }
  }
}

```

```
}  
}  
}
```

10.12.1 Block states

`states` define the variants a block can be in. Each state is a name (namespaced) and a list of allowed values — booleans, integers, or strings.

- Total combinations are limited (currently 65,536 permutations per block, and 16 states max), which sounds huge until you have four 4-value states multiplied together. Keep states minimal.
- Read a state in Molang with `q.block_state('frostopack:lit')`.
- Set one with a command: `/setblock ~ ~ ~ frostopack:frostop_lantern ["frostopack:lit"=true]`.
- Set one from an event with `set_block_state`.

10.12.2 Permutations

A **permutation** is a condition plus a set of components that apply when the condition is true. Permutations are evaluated in order and later matching ones override earlier ones, so put general cases first and specific cases last.

This is the mechanism for: a lamp that glows only when lit, a crop that changes model as it grows, a directional block that rotates its geometry, a block that becomes solid when powered.

10.12.3 Block traits

Traits are pre-built state bundles the game manages for you:

- `minecraft:placement_direction` — adds `minecraft:cardinal_direction` and/or `minecraft:facing_direction`, set automatically from how the player was facing when they placed it. This saves you writing four permutations and a placement event.
- `minecraft:placement_position` — adds `minecraft:block_face` and `minecraft:vertical_half`, for blocks that care which face they were placed against (like slabs and torches).

Use traits rather than rolling your own rotation states; they handle edge cases you will not think of.

10.12.4 Block components reference

Component	Purpose
<code>minecraft:geometry</code>	Custom model. Also accepts <code>bone_visibility</code>
<code>minecraft:material_instances</code>	Face textures and render methods
<code>minecraft:collision_box</code>	Physical collision. <code>false</code> = walk through
<code>minecraft:selection_box</code>	The outline you see when aiming at it
<code>minecraft:destructible_by_mining</code>	Break time in seconds, or <code>false</code> for unbreakable
<code>minecraft:destructible_by_explosion</code>	Blast resistance, or <code>false</code> for immune
<code>minecraft:friction</code>	0.0 (ice-like) to 0.9+ (slime-like). Default 0.4
<code>minecraft:light_emission</code>	0–15 light given off
<code>minecraft:light_dampening</code>	0–15 light blocked
<code>minecraft:flammable</code>	Catch and burn chances
<code>minecraft:map_color</code>	Colour on maps
<code>minecraft:loot</code>	Loot table path
<code>minecraft:placement_filter</code>	Which faces and which blocks it can be placed on
<code>minecraft:transformation</code>	Rotate/scale/translate the whole block model
<code>minecraft:crafting_table</code>	Turns the block into a crafting station with custom tags
<code>minecraft:display_name</code>	Name (or use the lang file)
<code>minecraft:redstone_conductivity</code>	Whether redstone signals pass through
<code>minecraft:item_visual</code>	Overrides the inventory appearance
<code>minecraft:liquid_detection</code>	How the block reacts to water
<code>minecraft:custom_components</code>	Attaches script-defined behaviour
<code>minecraft:tick</code>	Fires <code>minecraft:on_tick</code> -style events on an interval

10.12.5 Block events

Older block formats had built-in triggers (`minecraft:on_interact`, `minecraft:on_player_placing`, `minecraft:on_step_on`, `minecraft:on_fall_on`, `minecraft:on_player_destroyed`, `minecraft:queued_ticking`, `minecraft:random_ticking`). Depending on your `format_version`, some of these are available as components with an `event` property:

JSON

```
"minecraft:queued_ticking": {
  "looping": true,
  "interval_range": [20, 40],
  "on_tick": { "event": "frostack:pulse" }
},
"minecraft:random_ticking": {
  "on_tick": { "event": "frostack:try_grow" }
}
```

Event responses available inside a block's `events` :

- `set_block` — replace with a different block
- `set_block_state` — change a state value (Molang allowed)
- `spawn_loot` — drop a loot table
- `damage` — damage the entity that triggered it
- `decrement_stack` — consume one of the player's held item
- `play_effect` / `play_sound`
- `run_command` — run commands
- `swing` — play the player's arm swing
- `transform_item` — change the held item
- `sequence` , `randomize` , `trigger` — the same control structures as entity events

For anything more complex than these, use `minecraft:custom_components` and the Script API.

10.13 Recipes

Recipes go in `frostack_BP/recipes/` . Each type is its own format.

Shaped crafting:

JSON

```
{
  "format_version": "1.20.10",
  "minecraft:recipe_shaped": {
    "description": {
      "identifier": "frostpack:frost_lantern"
    },
    "tags": ["crafting_table"],
    "pattern": [
      " I ",
      "ISI",
      " I "
    ],
    "key": {
      "I": { "item": "minecraft:iron_ingot" },
      "S": { "item": "frostpack:frost_shard" }
    },
    "result": {
      "item": "frostpack:frost_lantern",
      "count": 1
    }
  }
}
```

- The `pattern` is 1–3 rows of 1–3 characters. A space means "must be empty".
- Each character in `key` maps to an item. Add `"data": 1` for a specific variant.
- `tags` says which crafting station: `crafting_table`, `stonecutter`, `smithing_table`, `furnace`, `blast_furnace`, `smoker`, `campfire`, `brewing_stand`.
- `priority` (optional integer) breaks ties when two recipes match; lower is preferred.

Shapeless crafting:

JSON

```
{
  "format_version": "1.20.10",
  "minecraft:recipe_shapeless": {
    "description": { "identifier": "frostpack:frost_dust" },
    "tags": ["crafting_table"],
    "ingredients": [
      { "item": "frostpack:frost_shard" },
      { "item": "minecraft:gunpowder" }
    ],
    "result": { "item": "frostpack:frost_dust", "count": 3 }
  }
}
```

Furnace:

JSON

```
{
  "format_version": "1.20.10",
  "minecraft:recipe_furnace": {
    "description": { "identifier": "frostpack:smelt_frost_shard" },
    "tags": ["furnace", "blast_furnace"],
    "input": "frostpack:frost_ore",
    "output": "frostpack:frost_shard"
  }
}
```

Brewing (`minecraft:recipe_brewing_mix` and `minecraft:recipe_brewing_container`) and **smithing** (`minecraft:recipe_smithing_transform`) follow the same shape with different fields.

① **Recipes need `/reload` only**

Recipes, loot tables and functions are behaviour pack data, so `/reload` picks up changes without leaving the world. Use it — this is the fastest iteration loop in the whole workflow.

10.14 Animations

Visual animations live in the **resource pack**. Create them in Blockbench's **Animate** mode.

10.14.1 Making an animation in Blockbench

1. Switch to the **Animate** tab.

2. In the **Animations** panel, click + and name it `animation.frost_wraith.walk`. The identifier convention is `animation.<entity>.<action>`.
3. Set the **length** (top-right of the timeline) — 1 second is typical for a walk cycle.
4. Turn **Loop** on for cyclic animations (idle, walk). Leave it off for one-shot animations (attack, death).
5. Select a bone in the Outliner, move the playhead, and change its **Rotation** / **Position** / **Scale** in the right-hand panel. A keyframe appears on the timeline.
6. For a walk cycle: at time 0 set `leg_left` rotation X to `-25` and `leg_right` to `25`; at 0.5s swap them; at 1.0s return to the time-0 values so it loops seamlessly.
7. Right-click keyframes to change **interpolation**: `linear` (constant speed), `catmullrom` (smooth ease), `step` (snap).
8. `File > Export > Bedrock Animation` → `frostpack_RP/animations/frost_wraith.animation.json`.

10.14.2 The animation file format

JSON

```
{
  "format_version": "1.8.0",
  "animations": {
    "animation.frost_wraith.walk": {
      "loop": true,
      "animation_length": 1.0,
      "bones": {
        "leg_left": {
          "rotation": {
            "0.0": [-25, 0, 0],
            "0.5": [25, 0, 0],
            "1.0": [-25, 0, 0]
          }
        },
        "leg_right": {
          "rotation": {
            "0.0": [25, 0, 0],
            "0.5": [-25, 0, 0],
            "1.0": [25, 0, 0]
          }
        }
      }
    },
    "animation.frost_wraith.idle": {
      "loop": true,
      "bones": {
        "body": {
          "position": [0, "math.sin(q.life_time * 90) * 0.5", 0]
        },
        "head": {
          "rotation": [
            "q.head_x_rotation",
            "q.head_y_rotation",
            0
          ]
        }
      }
    }
  }
}
```

Two things to notice:

- **Keyframes are timestamps as string keys.** `"0.5": [25, 0, 0]`.

- A bone channel can be a single Molang expression instead of keyframes. The `idle` example above has no keyframes at all — the body bobs continuously via `math.sin`, and the head tracks where the mob is looking via `q.head_x_rotation`. **Procedural animation like this is often better than keyframes:** it never de-syncs, costs nothing to author, and responds to game state. Vanilla uses it heavily.

Other animation properties: `"animation_length"`, `"loop": "hold_on_last_frame"` (freeze at the end instead of looping or resetting), `"anim_time_update"` (a Molang expression controlling how fast the animation's clock runs — use `"q.modified_move_speed"` to make a walk cycle speed up with the mob), `"blend_weight"`, `"override_previous_animation"`, `"start_delay"`, `"loop_delay"`.

JSON

```
"animation.frost_wraith.walk": {  
  "loop": true,  
  "anim_time_update": "q.modified_move_speed * 2.0",  
  "bones": { ... }  
}
```

That one line is what makes a vanilla walk cycle match the mob's actual speed.

10.14.3 Animation controllers (client)

An animation controller is a **state machine** deciding which animation plays. Simple mobs can skip it and use the client entity's `scripts.animate` conditions; anything with more than two or three animations needs a controller.

```
frostpack_RP/animation_controllers/frost_wraith.animation_controllers.json :
```

```
{
  "format_version": "1.10.0",
  "animation_controllers": {
    "controller.animation.frost_wraith.move": {
      "initial_state": "default",
      "states": {
        "default": {
          "animations": ["idle"],
          "transitions": [
            { "walking": "q.modified_move_speed > 0.1" },
            { "attacking": "q.is_delayed_attacking" }
          ]
        },
        "walking": {
          "animations": ["walk"],
          "blend_transition": 0.2,
          "transitions": [
            { "default": "q.modified_move_speed <= 0.1" },
            { "attacking": "q.is_delayed_attacking" }
          ]
        },
        "attacking": {
          "animations": ["attack"],
          "blend_transition": 0.1,
          "transitions": [
            { "default": "q.all_animations_finished" }
          ]
        }
      }
    }
  }
}
```

- **initial_state** — where it starts.
- **animations** — the short names from the client entity file's **animations** map.
- **transitions** — a list of `{"target_state": "molang_condition"}`. The first true condition wins, checked in order.
- **blend_transition** — seconds to cross-fade into this state. A small value (0.1–0.3) removes the snapping between animations.
- **on_entry** / **on_exit** — arrays of Molang statements or `/commands` run when entering/leaving. Client controllers can play sounds and particles here:

JSON

```
"attacking": {
  "animations": ["attack"],
  "particle_effects": [
    { "effect": "frost_burst", "locator": "hand_right" }
  ],
  "sound_effects": [
    { "effect": "frost_swing" }
  ],
  "on_entry": ["v.attack_flash = 1;"],
  "on_exit": ["v.attack_flash = 0;"]
}
```

Register the controller in the client entity file:

JSON

```
"scripts":{
  "animate": [ "controller.animation.frost_wraith.move" ]
},
"animations": {
  "idle": "animation.frost_wraith.idle",
  "walk": "animation.frost_wraith.walk",
  "attack": "animation.frost_wraith.attack",
  "controller.animation.frost_wraith.move": "controller.animation.frost_wraith.move"
}
```

Note the controller must appear in **both** the `animations` map and the `scripts.animate` list.

10.14.4 Behaviour animations and controllers

The same file formats exist in the behaviour pack, but instead of moving bones they **run commands** and fire events. This is how you get timed sequences without scripting.

`frostpack_BP/animations/frost_wraith.animation.json` :

JSON

```
{
  "format_version": "1.10.0",
  "animations": {
    "animation.frost_wraith.frost_nova": {
      "loop": false,
      "animation_length": 2.0,
      "timeline": {
        "0.0": ["/playsound beacon.activate @a ~ ~ ~"],
        "1.0": ["/particle minecraft:snowflake_particle ~ ~1 ~"],
        "1.5": ["/effect @e[r=5,type=!frostpack:frost_wraith] slowness 5 2"]
      }
    }
  }
}
```

frostpack_BP/animation_controllers/frost_wraith.animation_controllers.json :

JSON

```
{
  "format_version": "1.10.0",
  "animation_controllers": {
    "controller.animation.frost_wraith.nova": {
      "initial_state": "waiting",
      "states": {
        "waiting": {
          "transitions": [
            { "casting": "q.property('frostpack:enraged')" }
          ]
        },
        "casting": {
          "animations": ["frost_nova"],
          "transitions": [
            { "waiting": "q.all_animations_finished" }
          ]
        }
      }
    }
  }
}
```

And register both on the **server entity** with a `minecraft:behavior`-adjacent script block in `description` :

JSON

```
"description": {
  "identifier": "frostpack:frost_wraith",
  "animations": {
    "frost_nova": "animation.frost_wraith.frost_nova",
    "nova_controller": "controller.animation.frost_wraith.nova"
  },
  "scripts": {
    "animate": [ "nova_controller" ]
  }
}
```

◆ Behaviour animation controllers are the poor man's scripting

Before the Script API existed, this was how every advanced add-on did timed logic, boss phases, and multi-stage attacks. It still works, needs no experimental toggles, and runs on every platform. If you want to avoid JavaScript, learn this pattern well.

10.15 Particles

frostpack_RP/particles/frost_burst.particle.json :

```
{
  "format_version": "1.10.0",
  "particle_effect": {
    "description": {
      "identifier": "frostpack:frost_burst",
      "basic_render_parameters": {
        "material": "particles_alpha",
        "texture": "textures/particle/particles"
      }
    },
    "components": {
      "minecraft:emitter_rate_instant": { "num_particles": 30 },
      "minecraft:emitter_lifetime_once": { "active_time": 0.5 },
      "minecraft:emitter_shape_sphere": {
        "radius": 0.5,
        "direction": "outwards"
      },
      "minecraft:particle_lifetime_expression": {
        "max_lifetime": 1.2
      },
      "minecraft:particle_initial_speed": 3,
      "minecraft:particle_motion_dynamic": {
        "linear_acceleration": [0, -4, 0],
        "linear_drag_coefficient": 1.5
      },
      "minecraft:particle_appearance_billboard": {
        "size": [0.1, 0.1],
        "facing_camera_mode": "lookat_xyz",
        "uv": {
          "texture_width": 128,
          "texture_height": 128,
          "uv": [0, 0],
          "uv_size": [8, 8]
        }
      },
      "minecraft:particle_appearance_tinting": {
        "color": [0.6, 0.85, 1.0, 1.0]
      }
    }
  }
}
```

Trigger particles by:

- `/particle frostpack:frost_burst ~ ~1 ~`
- A `particle_effects` entry on an animation controller state (as shown above).

- A `minecraft:particle_effect` block in an entity's client description with a named locator.
- The `play_effect` event response in a block event.

★ Use Snowstorm

Writing particle JSON by hand is miserable. **Snowstorm** (`snowstorm.app` , by the Blockbench author) is a free browser-based particle editor with a live preview that exports Bedrock particle JSON directly. Design there, export, drop into `particles/` .

10.16 Functions and commands

A `.mcfuction` file is a plain text file of commands, one per line, with no leading slash. Comments start with `#` .

`frostpack_BP/functions/frostpack/tick.mcfuction` :

TEXT

```
# Slow anything standing near a lit frost lantern
execute as @a[tag=near_frost] run effect @s slowness 2 0 true
scoreboard players add @a frost_time 1
```

Run it with `/function frostpack/tick` , or make it run every tick by listing it in

`frostpack_BP/functions/tick.json` :

JSON

```
{
  "values": [ "frostpack/tick" ]
}
```

⚠ Tick functions run 20 times a second

A tick function with expensive selectors (`@e[r=100]`) run every tick will noticeably hurt performance on lower-end devices. Use scoreboard timers to run heavy logic every 20 ticks instead of every tick, and always constrain selectors with `type=` and a radius.

Commands you will use most in add-on work:

TEXT

```
/summon frostpack:frost_wraith ~ ~ ~  
/event entity @e[type=frostpack:frost_wraith,c=1] frostpack:enrage  
/give @s frostpack:frost_shard 8  
/setblock ~ ~ ~ frostpack:frost_lantern ["frostpack:lit"]=true]  
/tag @s add frost_touched  
/scoreboard objectives add frost_time dummy  
/execute as @e[type=frostpack:frost_wraith] at @s run particle  
minecraft:snowflake_particle ~ ~1 ~  
/testfor @e[type=frostpack:frost_wraith]  
/damage @e[type=frostpack:frost_wraith] 5 magic
```

10.17 The Script API

For logic JSON cannot express, Bedrock provides a JavaScript API. This is genuinely powerful — real event handlers, per-tick logic, dynamic properties, inventory manipulation — and it is how most large modern add-ons work.

1. Declare the module in the behaviour pack manifest:

JSON

```
"modules": [  
  {  
    "type": "data",  
    "uuid": "...",  
    "version": [1, 0, 0]  
  },  
  {  
    "type": "script",  
    "language": "javascript",  
    "uuid": "...",  
    "version": [1, 0, 0],  
    "entry": "scripts/main.js"  
  }  
],  
"dependencies": [  
  {  
    "module_name": "@minecraft/server",  
    "version": "2.0.0"  
  }  
]
```

The `version` string in `dependencies` must be a version the game actually ships. Check the current value in the official docs or in a recent vanilla sample — using a version that does not exist makes the whole pack fail to load with a script error.

2. Write `frostpack_BP/scripts/main.js` :

JAVASCRIPT

```
import { world, system, EntityDamageCause } from "@minecraft/server";

// React to a player using your item
world.afterEvents.itemUse.subscribe((event) => {
  const { source, itemStack } = event;
  if (itemStack.typeId !== "frostpack:frost_shard") return;

  const dimension = source.dimension;
  const origin = source.location;

  dimension.spawnParticle("frostpack:frost_burst", origin);
  dimension.playSound("beacon.activate", origin);

  for (const entity of dimension.getEntities({
    location: origin,
    maxDistance: 6,
    excludeTypes: ["minecraft:player"],
  })) {
    entity.addEffect("slowness", 100, { amplifier: 2 });
    entity.applyDamage(4, { cause: EntityDamageCause.magic });
  }
});

// A cheap 1-second loop
system.runInterval(() => {
  for (const player of world.getAllPlayers()) {
    const held = player.getComponent("minecraft:equippable")
      ?.getEquipment("Mainhand");
    if (held?.typeId === "frostpack:frost_blade") {
      player.addEffect("night_vision", 220, { showParticles: false });
    }
  }
}, 20);
```

Key concepts:

- `world.afterEvents.*` — things that already happened (`entityDie` , `itemUse` , `playerSpawn` , `blockBreak` , `entityHurt` , `playerInteractWithBlock`).

- `world.beforeEvents.*` — things about to happen, which you can cancel by setting `event.cancel = true`. **You cannot modify the world from a before-event handler** — you must defer with `system.run(() => { ... })`.
- `system.run`, `system.runTimeout`, `system.runInterval` — scheduling.
- `system.afterEvents.scriptEventReceive` — lets a command (`/scriptevent frostpack:do_thing`) call into your script. This is the bridge between JSON events and scripts.
- **Dynamic properties** — `entity.setDynamicProperty()` / `world.setDynamicProperty()` for persistent custom data.
- **Custom components** — register a block or item component in script with `world.beforeEvents.worldInitialize.subscribe(e => e.blockComponentRegistry.registerCustomComponent(...))`, then reference it from the block's JSON. This is the modern way to give blocks arbitrary behaviour.

⚠ Scripting caveats

- The API changes between versions, sometimes with breaking changes. Pin your dependency version and re-test on each game update.
- Some APIs require the **Beta APIs** experimental toggle. Anything in `@minecraft/server-ui`, `@minecraft/server-admin` and various newer classes may be beta.
- Scripts run in a sandbox: no file system, no network, no `setTimeout`. Use `system.runTimeout`.
- Errors go to the content log. Turn it on, or you will get silent failures.

10.18 Debugging your code

The tools, in order of how often you will use them:

1. **The content log.** `Settings > Creator > Enable Content Log GUI` + verbose. It names the file and the error. This finds 80% of problems instantly.
2. **VS Code's Problems panel** with Blockception's extension. Catches JSON syntax errors and unknown component names before the game ever sees them.
3. `/reload` for behaviour data, world re-entry for resources, full restart when in doubt.
4. **Bisecting.** Comment out (or cut into a scratch file) half your components, test, and narrow down which half breaks it. Crude but reliable.
5. **say debugging.** Put `"run_command": { "command": ["say enrage fired"] }` in an event to confirm it is actually firing. Half of all "my logic is broken" turns out to be "my event never ran".
6. `/scriptevent` + `world.sendMessage()` for scripts.
7. **The vanilla packs.** When something should be possible and is not working, find a vanilla entity that does it and diff your file against theirs.

◆ The debugging question that solves most bugs

"Is this actually running at all?" Before you debug *what* your code does, prove *that* it runs — a `say` command, a particle, a sound. Most of the time it is not running, and the cause is a filter that never passes, an event name typo, or a file in the wrong folder.

11 Common mistakes byte two

This byte covers everything that goes wrong during **coding** — folders, importing, and the whole Coding section. If your files exist and your art is fine but the add-on does not behave, the answer is here.

11.1 Folder mistakes

- Client entity file in `entities/` instead of `entity/`. The resource pack uses `entity/` (singular); the behaviour pack uses `entities/` (plural). No error is produced; the mob simply has no appearance.
- Behaviour file in the resource pack, or vice versa. Very common when copying between the two.
- Geometry in `models/` root instead of `models/entity/`. Usually still works because the game scans recursively — but if the tooling complains, move it.
- Loot table path wrong. `minecraft:loot` 's path is relative to the BP root and includes `.json`: `"loot_tables/entities/frost_wraith.json"`. Texture paths exclude the extension; loot table paths include it.
- `texts/languages.json` missing, so nothing in `en_US.lang` is read.
- Folder named `behaviour_packs` (British spelling). Minecraft uses `behavior_packs` and `development_behavior_packs` — American spelling, always.

11.2 Identifier and linking mistakes

- The BP entity identifier and the RP client entity identifier do not match. These two strings are the only link between a mob's brain and its body. Search the identifier project-wide; you should find it in both.
- Geometry identifier mismatch. `"geometry": { "default": "geometry.frost_wraith" }` must match the `identifier` inside the `.geo.json`.
- Animation identifier mismatch. `animation.frost_wraith.walk` in the animation file must match what the client entity file references.
- Render controller not listed in the client entity's `render_controllers` array, so nothing draws.
- An animation controller registered in `scripts.animate` but not in the `animations` map (or vice versa). It must be in both.
- Texture short name used where a path is expected, or a path used where a short name is expected. In the client entity file, `textures` takes paths; in an item's `minecraft:icon`, it takes a short name from `item_texture.json`.
- Using the item's identifier as its icon name. `"texture": "frostpack:frost_shard"` is wrong; it should be the short key, `"frost_shard"`.

11.3 Entity behaviour mistakes

The mob spawns but falls through the world / floats / does not move.

- Missing `minecraft:physics`. It takes no parameters and is easy to forget.
- Missing one of the movement trio: `minecraft:movement` (speed), `minecraft:movement.basic` (type), `minecraft:navigation.walk` (pathfinder). All three are required.
- `minecraft:movement` set to `0`.

The mob is passive and never attacks.

- No targeting behaviour. `melee_attack` needs a target, which comes from `nearest_attackable_target` or `hurt_by_target`.
- The target filter never passes — check the family names. Players are family `player`; villagers are `villager`.
- `must_see: true` with the mob unable to see the target.
- `within_radius` / `max_dist` too small.
- Priority conflict: a lower-numbered behaviour (higher priority) is always winning. `random_stroll` at priority 1 will suppress an attack at priority 2.

The mob attacks but deals no damage.

- Missing `minecraft:attack`.
- `reach_multiplier` too small for the mob's collision box size.

The mob drowns or gets stuck in water.

- Missing `minecraft:behavior.float` at priority 0.

Component groups do not apply.

- The group name in the event does not match the key in `component_groups` (including the namespace prefix if you used one).
- The event is never fired. Add a `run_command` with a `say` to prove it.
- You added a group but a component in `components` is still overriding it — components in `components` are always active; a group cannot remove them. To make something removable, put it in a group in the first place.
- A group is added but another group containing the same component is also active, and which one wins is unpredictable. Do not have two active groups define the same component.

Filters never pass.

- `subject` is wrong. In an interaction, the *player* is `other`, not `self`.
- Testing `is_family` against the entity's identifier rather than a family from `minecraft:type_family`.
- `operator` defaults to `==` — a `has_component` test needs `"operator": "!="` to mean "does not have".

- Nesting `all_of` inside `all_of` incorrectly, so a condition is being combined the wrong way.

Entity properties do nothing in animations.

- `client_sync` is not set to `true` in the property definition, so the resource pack cannot see the value.
- Reading it as `q.property('charge')` instead of `q.property('frostpack:charge')` — the namespace is required.

11.4 Spawn rule mistakes

- The spawn rule identifier does not match the entity identifier.
- `is_spawnable` is `false` in the entity's description.
- `population_control` is wrong, so the mob competes with (or is capped by) the wrong category.
- `minecraft:weight` too low relative to vanilla mobs. A weight of 1 against zombies at 100 means roughly a 1% share.
- Biome filter uses biome names instead of biome tags. Current formats want `has_biome_tag`.
- Testing spawning too close to yourself. Mobs spawn 24–128 blocks away. Standing still in one spot and expecting them at your feet will not work.
- Light level filter contradicts the placement. `spawns_on_surface` plus `brightness_filter 0–7` means only at night or in shadow.
- The mob cap is full of other mobs. `/kill @e[type=!player]` before testing.

11.5 Loot table mistakes

- `minecraft:loot` path missing `.json` or missing the `loot_tables/` prefix.
- Nothing drops. Loot only rolls when the mob dies "properly" — `/kill` does drop loot, but a mob removed by `/kill @e` in creative sometimes behaves differently than expected. Kill it with a weapon to be certain.
- `killed_by_player` condition on a pool means `/kill` produces nothing.
- Weights misunderstood. Weight is relative *within a pool*. Two entries at weight 1 and 3 give 25% and 75% — not 1% and 3%.
- Item identifier typo in the `name` field. The pool rolls, produces an unknown item, and drops nothing.

11.6 Item mistakes

- `format_version` too low for the components you used. Item components have been added over many versions; a component in a file with `"format_version": "1.16.0"` may simply be ignored. Use a current version.
- An item with `minecraft:durability` and `max_stack_size` above 1. Durability requires stack size 1.

- `minecraft:icon` referencing a missing `item_texture.json` key — the purple/black checker.
- The item does not appear in the creative menu. Missing or invalid `menu_category`. Note that `is_hidden_in_commands: true` also hides it from `/give` autocomplete.
- A custom tool does not mine faster. Missing `minecraft:digger`, or missing the appropriate `minecraft:tags` (`minecraft:is_pickaxe` etc.) so the game does not treat it as the right tool class.
- A weapon does no extra damage. `minecraft:damage` is the added damage value; it also needs `minecraft:hand_equipped` to look right in hand.

11.7 Block mistakes

- State names without a namespace. `"lit": [false, true]` is invalid; it must be `"frostdpack:lit"`.
- Permutation conditions written as JSON filters. Block permutation conditions are **Molang strings**, not filter objects: `"q.block_state('frostdpack:lit') == true"`.
- Permutation order. Later matching permutations override earlier ones. If a general permutation is listed after a specific one, it wins and you see the wrong result.
- Too many state combinations. Multiplying several multi-value states blows past the permutation limit and the block fails to load.
- Custom geometry with no collision or selection box, so you fall through it and cannot click it.
- Block does not appear in the creative inventory. Missing `menu_category`.
- The block breaks instantly / never breaks. Missing `minecraft:destructible_by_mining`, or set to `false`.
- Block events do nothing. The event trigger component (`minecraft:queued_ticking`, `minecraft:random_ticking`, an interact component) is missing or unavailable in your `format_version`.
- `/setblock` with states fails. The syntax is `/setblock ~ ~ ~ frostdpack:frostd_lantern ["frostdpack:lit"=true]` — square brackets, quoted state name, no space around `=`.

11.8 Recipe mistakes

- The recipe does not appear. The recipe's `description.identifier` should be unique; two recipes sharing an identifier means one is ignored.
- `tags` missing or wrong, so the recipe is not registered to any crafting station.
- Pattern rows of unequal length. All rows must have the same number of characters; pad with spaces.
- A key character used in the pattern but not defined in `key`.
- Recipes not reloading. `/reload` picks up recipes, but the recipe book may cache — try re-opening the crafting table.

11.9 Animation mistakes

- **Bone name in the animation does not exist in the geometry.** Silent no-op. This happens whenever you rename a bone after animating.
- **The animation is never listed** in the client entity's `animations` map, or never referenced in `scripts.animate` / a controller.
- **A looping animation snaps at the loop point.** The first and last keyframes are not identical.
- **A one-shot animation loops forever.** `"loop": true` left on.
- **An animation controller state never exits.** Its transition condition is never true — `q.all_animations_finished` only becomes true for non-looping animations.
- **Radians instead of degrees** in a Molang trig expression, producing an almost invisible wobble.
- **Two animations fighting.** Both are active and both write to the same bone. Use `blend_weight` or an animation controller so only one plays at a time.
- **A behaviour animation put in `RP/animations/`, or a visual animation put in `BP/animations/`.** Both do nothing.

11.10 Script API mistakes

- `dependencies` version does not exist. The whole pack fails to load. Check the exact version string against the docs.
- **Modifying the world inside a `beforeEvents` handler.** Not allowed — wrap the change in `system.run(() => { ... })`.
- **Using `setTimeout`.** Not available. Use `system.runTimeout`.
- **A beta API used without the Beta APIs experimental toggle enabled.**
- **The `entry` path in the manifest does not match the actual file.** It is relative to the pack root: `scripts/main.js`.
- **A `runInterval` at 1-tick frequency doing heavy work,** tanking the frame rate.
- **Errors not visible** because the content log is off.

11.11 General coding mistakes

- **Trailing commas and unbalanced brackets.** Watch the Problems panel.
- **Editing the file while the game holds it.** Save, then reload.
- **Assuming `/reload` refreshes textures and models.** It does not — only behaviour pack data.
- **Copying a snippet with a different `format_version` into a file with an older one.** Format versions apply per file, and components silently do nothing if the version is too low.
- **Copying vanilla files wholesale without changing identifiers,** so your entity overwrites a vanilla one.
- **Changing many things at once,** so when it breaks you cannot tell which change did it.

12 Creating the manifests

The `manifest.json` is the identity card of a pack. Minecraft reads it before anything else, and if it is wrong the pack does not exist as far as the game is concerned — no error, no entry in the pack list, nothing.

You have already made minimal manifests in the First Steps section. This section explains every field, covers the variants for different pack types, and shows how to link packs together.

12.1 What a manifest does

Four jobs:

1. **Identifies the pack** with a UUID, so the game can tell packs apart, track which worlds use which pack, and handle updates.
2. **Declares what kind of pack it is** through its modules — data (behaviour), resources, scripts, world template, skins.
3. **Declares the minimum engine version** so old game versions do not try to load a pack that needs newer features.
4. **Declares dependencies** — other packs this one needs, and script API modules it uses.

12.2 The complete behaviour pack manifest

JSON

```
{
  "format_version": 2,
  "header": {
    "name": "pack.name",
    "description": "pack.description",
    "uuid": "1c8ad0e0-6d33-4b4e-9d1a-6a1f4e30b101",
    "version": [1, 0, 0],
    "min_engine_version": [1, 21, 0]
  },
  "modules": [
    {
      "type": "data",
      "description": "Frost Pack behaviours",
      "uuid": "1c8ad0e0-6d33-4b4e-9d1a-6a1f4e30b102",
      "version": [1, 0, 0]
    }
  ],
  "dependencies": [
    {
      "uuid": "1c8ad0e0-6d33-4b4e-9d1a-6a1f4e30b201",
      "version": [1, 0, 0]
    }
  ],
  "metadata": {
    "authors": ["Your Name"],
    "license": "MIT",
    "url": "https://example.com",
    "product_type": "addon",
    "generated_with": {
      "bridge": ["2.0.0"]
    }
  }
}
```

12.2.1 Field by field

format_version — the version of the *manifest format*, not your pack. It is the number `2` for everything modern. Not a string; no quotes. Version 1 manifests are ancient and should not be used.

header.name — the display name in the pack list. Either literal text (`"Frost Pack"`) or a translation key (`"pack.name"`) that you define in `texts/en_US.lang` . Using the key lets you translate it.

header.description — the subtitle in the pack list. Same rules; conventionally `"pack.description"` .

header.uuid — the pack's unique identity. This is the value Minecraft records in a world's `world_behavior_packs.json` when a player activates your pack. **It must never change once you publish**, or every existing world will lose the pack.

header.version — `[major, minor, patch]`, all integers, in an array. Increase this every release. If a player imports an updated `.mcaddon` with the same version number, the game may keep the old copy.

header.min_engine_version — the lowest game version that can load this pack. Set it to the version you developed and tested against. Setting it too high hides your pack from players on older versions; setting it too low means it loads on versions where your components do not exist.

modules — the array declaring what the pack contains. Each module has its own UUID (different from the header and from every other module).

type	Meaning	Pack
data	Behaviour pack content	BP
resources	Resource pack content	RP
script	JavaScript	BP
world_template	A world template	Template
skin_pack	Skins	Skin pack

dependencies — two different shapes, and this is where people get confused:

- **Pack dependency** — `{ "uuid": "...", "version": [1, 0, 0] }`. Points at another pack's *header* UUID. Used to make a behaviour pack automatically pull in its resource pack.
- **Script module dependency** — `{ "module_name": "@minecraft/server", "version": "2.0.0" }`. Points at a game-provided script API module. Note the version here is a **string**, not an array.

metadata — entirely optional and never affects loading. Worth filling in: `authors`, `license`, `url`, and `product_type` (set to `"addon"` if your pack is not intended for the Marketplace).

capabilities — an optional array in the header for special permissions:

```
JSON

"capabilities": ["experimental_custom_ui", "chemistry", "raytraced", "pbr",
"script_eval"]
```

Only add what you actually need; some capabilities force experimental toggles on the world.

subpacks — optional. Lets one resource pack offer several quality tiers the player chooses between (e.g. low/medium/high texture resolution), each in its own `subpacks/<name>/` folder with a `memory_tier`.

12.3 The complete resource pack manifest

Identical structure with a different module type:

JSON

```
{
  "format_version": 2,
  "header": {
    "name": "pack.name",
    "description": "pack.description",
    "uuid": "1c8ad0e0-6d33-4b4e-9d1a-6a1f4e30b201",
    "version": [1, 0, 0],
    "min_engine_version": [1, 21, 0]
  },
  "modules": [
    {
      "type": "resources",
      "description": "Frost Pack resources",
      "uuid": "1c8ad0e0-6d33-4b4e-9d1a-6a1f4e30b202",
      "version": [1, 0, 0]
    }
  ],
  "metadata": {
    "authors": ["Your Name"],
    "product_type": "addon"
  }
}
```

Note that a resource pack does **not** need `min_engine_version` in older documentation, but including it is harmless and clearer.

12.4 Linking the two packs

By default, activating a behaviour pack does not activate its resource pack — the player must turn on both. That is a poor experience and a frequent source of "the mob is invisible" bug reports.

Fix it with a **dependency** in the *behaviour* pack's manifest pointing at the *resource* pack's **header UUID**:

JSON

```
"dependencies": [  
  {  
    "uuid": "1c8ad0e0-6d33-4b4e-9d1a-6a1f4e30b201",  
    "version": [1, 0, 0]  
  }  
]
```

Now activating the BP activates the RP automatically.

⚠ Dependency versions must match exactly

The `version` in the dependency must match the target pack's `header.version` exactly. If you bump the RP to `[1, 0, 1]` and forget to update the dependency in the BP, the BP will fail to load with a "missing dependency" error. Bump both together, every time.

You can also make the dependency point the other way (RP depends on BP), but $BP \rightarrow RP$ is the convention because behaviour is what players think they are installing.

12.5 Manifests for scripts

When your behaviour pack uses the Script API, add a second module and a script dependency:

```

"modules": [
  {
    "type": "data",
    "uuid": "1c8ad0e0-6d33-4b4e-9d1a-6a1f4e30b102",
    "version": [1, 0, 0]
  },
  {
    "type": "script",
    "language": "javascript",
    "entry": "scripts/main.js",
    "uuid": "1c8ad0e0-6d33-4b4e-9d1a-6a1f4e30b103",
    "version": [1, 0, 0]
  }
],
"dependencies": [
  {
    "uuid": "1c8ad0e0-6d33-4b4e-9d1a-6a1f4e30b201",
    "version": [1, 0, 0]
  },
  {
    "module_name": "@minecraft/server",
    "version": "2.0.0"
  },
  {
    "module_name": "@minecraft/server-ui",
    "version": "2.0.0"
  }
]

```

- `language` is always `"javascript"` (TypeScript must be compiled to JS first).
- `entry` is the path from the pack root to your main script file.
- Script module versions are strings and must be a version the game ships. A wrong version string prevents the entire pack from loading.

12.6 Manifests for world templates

A world template packages a whole world. Its manifest sits in the world folder:

JSON

```
{
  "format_version": 2,
  "header": {
    "name": "Frost Realm",
    "description": "A frozen survival world.",
    "uuid": "...",
    "version": [1, 0, 0],
    "base_game_version": [1, 21, 0],
    "lock_template_options": true
  },
  "modules": [
    {
      "type": "world_template",
      "uuid": "...",
      "version": [1, 0, 0]
    }
  ]
}
```

Note `base_game_version` instead of `min_engine_version`, and `lock_template_options` which stops players changing world settings.

12.7 Manifests for skin packs

JSON

```
{
  "format_version": 2,
  "header": {
    "name": "Frost Skins",
    "uuid": "...",
    "version": [1, 0, 0]
  },
  "modules": [
    {
      "type": "skin_pack",
      "uuid": "...",
      "version": [1, 0, 0]
    }
  ]
}
```

Skin packs also need a `skins.json` listing each skin and its geometry.

12.8 Getting and managing UUIDs

Every UUID in every manifest across every pack you make must be unique. For the two-pack add-on above, that is four UUIDs (plus one more if you add scripts).

Generate them at: `uuidgenerator.net`, PowerShell's `[guid]::NewGuid()`, VS Code's UUID Generator extension, or automatically via bridge.

Keep a record. Put a `uuids.txt` in your project noting which UUID is which. When you later need to write a dependency, or work out why two packs conflict, you will be glad you did.

TEXT

```
BP header : 1c8ad0e0-6d33-4b4e-9d1a-6a1f4e30b101
BP data   : 1c8ad0e0-6d33-4b4e-9d1a-6a1f4e30b102
BP script : 1c8ad0e0-6d33-4b4e-9d1a-6a1f4e30b103
RP header : 1c8ad0e0-6d33-4b4e-9d1a-6a1f4e30b201
RP res    : 1c8ad0e0-6d33-4b4e-9d1a-6a1f4e30b202
```

⚠ Never reuse UUIDs across projects

Copying an old pack folder as a starting point for a new project and forgetting to regenerate the UUIDs is extremely common. The result is that the two packs are the same pack as far as Minecraft is concerned, so only one loads, and which one is unpredictable. Regenerate all of them, first thing.

12.9 Manifest checklist

Before you package, confirm every one of these:

- ☐ `format_version` is the number `2`, unquoted.
- ☐ All UUIDs are valid 36-character UUIDs, all different from each other.
- ☐ No UUID is shared with any other pack you have made.
- ☐ `version` is an array of three integers, not a string.
- ☐ `min_engine_version` is an array of three integers and is $\leq$ your test version.
- ☐ The BP module type is `data`; the RP module type is `resources`.
- ☐ If using scripts: the script module has `language` and `entry`, and `@minecraft/server` is in `dependencies` with a **string** version.
- ☐ The BP depends on the RP, with matching version arrays.
- ☐ `pack.name` and `pack.description` exist in `texts/en_US.lang` if you used translation keys.
- ☐ The file is called `manifest.json`, in the pack root, valid JSON, with no trailing commas.

13 Finalising the addon

Your add-on works in the development folders. Now you turn it into a single file other people can install. This section covers final testing, cleanup, zipping and the `.mcaddon` format.

13.1 Final testing pass

Before packaging, run through this deliberately. Bugs found now are cheap; bugs found by other people are not.

Fresh-world test. Create a brand-new world with only your packs active. Development folders can mask problems because files you deleted are still cached, or because another pack of yours is quietly providing something.

Test every feature:

- Summon each mob with `/summon`. Kill it. Check the loot.
- Let each mob spawn naturally (or at least confirm the spawn rule file loads without errors).
- Give yourself each item. Check the name, the icon, the stack size, the tooltip, the use behaviour.
- Place each block. Break it. Check the drop, the sound, the light, the collision, and each permutation.
- Craft everything.
- Watch every animation, including the transitions between them.
- Check the content log is completely clean. Any warning is a real problem, even if the thing appears to work.

Test in survival mode. Creative hides a lot: durability, mining speed, damage, hunger, mob aggression.

Test on a low-end device if you can. Frame rate problems from tick functions or huge textures show up on phones and older consoles long before they show on a PC.

Test the multiplayer case if relevant. Some things behave differently for a second player, particularly scripts and client-side Molang.

13.2 Cleaning up

Remove development leftovers:

- Delete `.bbmodel` files, `.pdn` / `.xcf` source images, and any `notes.txt` from inside the packs. Keep them — just keep them outside.
- Delete unused textures, models and JSON files. Every unused file is download size.
- Remove any debugging `say` commands, test recipes, or `/give`-everything functions.
- Delete `.DS_Store`, `Thumbs.db`, and any editor cruft.

Check names and text:

- Every custom mob, item and block has a proper name in `en_US.lang` (no raw identifiers showing).
- The pack name and description in the manifest (or lang file) are what you want players to see.
- `pack_icon.png` exists in both packs and is 128×128.

Bump the version. Update `header.version` and both module versions in both manifests, and update the dependency version in the BP to match the RP's new version.

Validate the JSON. In VS Code, check the Problems panel is empty across the whole workspace.

13.3 Folder structure for packaging

Get this exactly right — it is the most common packaging failure.

Your two pack folders each contain their `manifest.json` at the **top level**:

```
TEXT

frostdpack_BP/
├─ manifest.json      ← directly inside, not in a sub-folder
├─ pack_icon.png
├─ entities/
└─ ...

frostdpack_RP/
├─ manifest.json
├─ pack_icon.png
├─ entity/
└─ ...
```

13.4 The three file formats

Extension	Contains	Use for
<code>.mcpack</code>	One pack (BP or RP)	Distributing a single pack
<code>.mcaddon</code>	Multiple packs, or <code>.mcpack</code> files	Distributing a BP+RP add-on — this is what you want
<code>.mctemplat e</code>	A world template	Distributing a whole world

All three are ordinary ZIP files with a renamed extension. Minecraft associates them so a double-click imports them.

13.5 Making a `.mcpack` (single pack)

1. Open the pack folder (e.g. `frostpack_BP`).
2. Select **all the contents** — `manifest.json`, `pack_icon.png`, and the folders. **Select the contents, not the folder itself.**
3. Right-click → **NanaZip > Add to archive** (or Windows' **Send to > Compressed (zipped) folder**).
4. Set the archive format to **zip** (not 7z, not tar — Minecraft only reads zip).
5. Name it `frostpack_BP.zip`.
6. Rename the file to `frostpack_BP.mcpack`.

⚠ The doubled-folder mistake

If you right-click the *folder* and zip it, the archive contains `frostpack_BP/manifest.json` instead of `manifest.json`. Minecraft looks for the manifest at the root of the archive, does not find it, and rejects the pack. Always zip the **contents**.

Verify by opening the finished archive: you should see `manifest.json` immediately, with no folder to click into first.

13.6 Making a `.mcaddon` (the full add-on)

The `.mcaddon` is a zip containing your two pack **folders** — this one *does* use folders, unlike `.mcpack`.

1. Make a new empty folder called `FrostPack`.
2. Copy `frostpack_BP` and `frostpack_RP` into it, as folders.

TEXT

```
FrostPack/
├── frostpack_BP/
│   ├── manifest.json
│   └── ...
└── frostpack_RP/
    ├── manifest.json
    └── ...
```

1. Go inside `FrostPack`, select both pack folders, right-click → zip.
2. Rename the resulting `.zip` to `FrostPack.mcaddon`.

Alternatively, a `.mcaddon` can contain two `.mcpack` files instead of two folders. Both work; folders are simpler.

Verify before sharing. Open the `.mcaddon` in NanaZip. You should see exactly two folders, and inside each, a `manifest.json` at that folder's top level. If you see a third wrapping folder, or a manifest one level too deep, redo it.

13.7 Exporting from bridge. v2

If you used bridge., all of the above is one menu item: **File > Export > .mcaddon**. It builds the correct structure automatically and puts the file in your Downloads folder. This is the single strongest argument for using bridge. — packaging mistakes simply stop happening.

13.8 Automating packaging

For repeated releases, a small script saves time and prevents mistakes. PowerShell, run from the folder containing both packs:

```
POWERSHELL

$out = "FrostPack.mcaddon"
if (Test-Path $out) { Remove-Item $out }
Compress-Archive -Path "frostpack_BP", "frostpack_RP" -DestinationPath
"FrostPack.zip"
Rename-Item "FrostPack.zip" $out
```

Note that `Compress-Archive` on the folder names produces the correct structure (folders inside the zip), which is what `.mcaddon` wants.

13.9 Documentation to ship alongside

Not required, but it makes the difference between an add-on people use and one they abandon:

- **A README** — what it adds, how to install, which Minecraft version it needs, whether any experimental toggles are required.
- **Screenshots** — one good image of each mob and block.
- **A version/changelog** so returning players know what is new.
- **Known issues** — being upfront about a bug is better than a confused bug report.
- **A licence** if you are sharing publicly. State clearly whether people may modify or redistribute it.

13.10 Sharing

- **MCPEDL / mcpedl.com** — the largest community site for Bedrock add-ons.
- **CurseForge** — hosts Bedrock add-ons.
- **Your own site, Discord, or Google Drive** — perfectly fine for sharing with friends.
- **The Minecraft Marketplace** — the official paid store. It requires being an approved partner, following the Marketplace content guidelines, and a review process. Not a first-add-on route.

⚠ Do not redistribute vanilla content

The vanilla behaviour and resource packs are provided for reference. Do not ship copies of vanilla files, textures or sounds in your pack. Reference them by identifier instead — your pack can use `minecraft:stone` without containing it.

13.11 Version control and archiving

Once you release version 1.0.0:

- **Tag it.** In Git, `git tag v1.0.0`. Without Git, keep a copy of the exact folder you released as `frostpack_1.0.0/`.
- **Keep the released `.mcaddon`.** If a player reports a bug, you need the exact build they have.
- **Never edit a released version in place.** Bump to `1.0.1`, fix, re-release.

13.12 Finalising checklist

- ☐ Tested in a fresh world with only these packs.
- ☐ Tested in survival, not just creative.
- ☐ Content log completely clean.
- ☐ All names appear correctly (no raw identifiers).
- ☐ `pack_icon.png` in both packs.
- ☐ Source files and debugging leftovers removed from inside the packs.
- ☐ Versions bumped in both manifests, and the BP's dependency version updated to match.
- ☐ Zipped as **zip**, with `manifest.json` at the correct depth.
- ☐ Extension renamed to `.mcaddon`.
- ☐ Archive opened and structure verified by eye.
- ☐ Installed the `.mcaddon` on a machine that has never had the dev folders — the real test.

14 Importing addon to Minecraft

You have a `.mcaddon` . Here is how it gets into the game, on every platform.

14.1 Windows

1. **Double-click the `.mcaddon` file.** Minecraft launches (or comes to the front) and shows an "Importing..." message, then "Successfully imported".
2. That is genuinely all. The packs are unpacked into `behavior_packs` and `resource_packs` and appear in the game's pack lists.
3. If double-clicking opens the wrong program, right-click → **Open with** → **Minecraft for Windows**, and tick *Always use this app*.

14.2 Android

1. Download the `.mcaddon` to the device, or transfer it by USB.
2. Open your **Files** app, find the download, and tap it.
3. Choose **Minecraft** if asked which app should open it.
4. The game launches and imports.

If tapping does nothing, the browser may have saved it as `.zip` . Rename the file to end in `.mcaddon` using a file manager, then tap again.

14.3 iOS and iPadOS

1. Download the file in Safari. It goes to the **Files** app (usually **Downloads** in *On My iPhone*).
2. Open **Files**, tap the `.mcaddon` .
3. If it does not open Minecraft directly, tap the **Share** button and choose **Copy to Minecraft**.

Safari sometimes renames files or unzips them automatically. If the file arrives as a folder or a `.zip` , use the Files app to rename it back to `.mcaddon` .

14.4 Xbox, PlayStation and Nintendo Switch

Consoles cannot import files directly. Two routes:

- **Via a world.** Apply the add-on to a world on a PC or phone, then upload that world to **Realms** and join it from the console. Everyone in the Realm gets the content.

- **Via cloud sync.** If the console and another device share a Microsoft account with cloud saves, a world saved with the packs applied will carry them.

Only the Marketplace can distribute add-ons to consoles as installable content.

14.5 Activating the add-on in a world

Importing installs the packs. It does not turn them on.

For a new world:

1. **Play > Create New World.**
2. In the left-hand menu, click **Behaviour Packs**.
3. Under **My Packs**, find your pack and click it, then click **Activate**.
4. Click **Resource Packs** and do the same. *(If you set up the dependency correctly, activating the BP does this for you automatically.)*
5. Set the rest of your world options and click **Create**.

For an existing world:

1. From the world list, click the **pencil icon** next to the world.
2. Scroll to **Behaviour Packs** and **Resource Packs** in the left menu.
3. Activate both.
4. Click **Play**.

⚠ Adding a behaviour pack to an existing world

Behaviour packs can change how a world works. Back up the world first (**Settings > Storage**), or copy the folder from `minecraftWorlds`). Removing a behaviour pack from a world that contains your custom blocks and mobs will make those blocks and mobs disappear, since the game no longer has definitions for them.

14.6 Confirming it worked

Load the world and run:

TEXT

```
/give @s frostpack:frost_shard  
/summon frostpack:frost_wraith  
/setblock ~ ~ ~ frostpack:frost_lantern
```

If the identifiers autocomplete as you type, the behaviour pack is active. If the mob appears but is invisible or an untextured shape, the behaviour pack is on and the resource pack is not.

14.7 If the import fails

- **"Import failed" / nothing happens.** The zip structure is wrong. Open the `.mcaddon` and check the folder depth — see Finalising the addon.
- **The pack imports but does not appear in the list.** Manifest problem: bad UUID, invalid JSON, or `min_engine_version` higher than the game version.
- **It says the pack is already installed.** A previous version with the same UUID is present. Delete it in `Settings > Storage > Resource Packs / Behaviour Packs`, or bump your pack's version and re-import.
- **The behaviour pack is greyed out.** It has a dependency the game cannot find — usually the resource pack, with a version that does not match.
- **It imports but content is missing in game.** Turn on the content log (`Settings > Creator`) and read what it says. It will name the file.

14.8 Updating an installed add-on

1. Increase `header.version` and the module versions in the manifests (and the BP's dependency version to match the RP).
2. Rebuild the `.mcaddon`.
3. Import it. Because the UUIDs match and the version is higher, Minecraft replaces the old copy.

If the UUIDs are the same **and** the version is the same or lower, the game may keep the old copy silently. This is the most common "my update did nothing" cause.

14.9 Uninstalling

`Settings > Storage > Resource Packs` (or **Behaviour Packs**), select the pack, and delete it. Deleting a behaviour pack that a world is using will remove that content from the world, so back up first.

That is the whole journey — from an empty folder, through models, textures, folders, JSON, manifests and packaging, to a file that anyone can double-click and play.

Good luck testing those wonderful add-ons! Go and break something, read the content log, fix it, and break something better.

15 Common mistakes byte three

This byte covers the last stretch: **manifests, finalising, packaging and importing**. These are the mistakes that turn a working add-on into a file nobody can install.

15.1 Manifest mistakes

- **format_version** written as `"2"`. It must be the number `2`, unquoted. This alone prevents the pack from loading.
- **version** written as `"1.0.0"`. It must be `[1, 0, 0]` — an array of three integers.
- **min_engine_version** written as a string or set higher than the player's game version, so the pack silently never appears.
- **Duplicate UUIDs**. Between header and module in the same manifest, or between two of your packs, or copied from someone else's pack. Every UUID everywhere must be unique.
- **A UUID with the wrong number of characters** after a bad copy-paste. It must be 36 characters including hyphens, in 8-4-4-4-12 groups.
- **Changing the header UUID after release**. Every existing world loses the pack, and players' worlds break.
- **Wrong module type**. `data` for behaviour, `resources` for resource. Swapping them means the game looks for the wrong kind of content and finds nothing.
- **Behaviour pack dependency version does not match the resource pack's actual version**. You bumped one and not the other, and now the BP refuses to load with a missing-dependency error. Bump both, always.
- **A dependency pointing at a module UUID instead of the header UUID**. Dependencies always reference the target pack's **header** UUID.
- **Script dependency version as an array**. `@minecraft/server` versions are **strings**: `"version": "2.0.0"`, not `[2, 0, 0]`.
- **A script module version that the game does not ship**. The whole pack fails with a script error. Check the docs for the exact string.
- **Script module missing `entry` or `language`**, or `entry` pointing at a file that does not exist.
- **Trailing comma** after the last entry in `modules` or `dependencies`.
- **pack.name** used as the manifest name with no matching entry in `en_US.lang`, so the pack shows literally as "pack.name" in the list.
- **manifest.json** saved with a BOM or as UTF-16. Save as plain UTF-8.
- **manifest.json** in a sub-folder rather than the pack root.

15.2 Packaging mistakes

- **Zipping the folder instead of its contents for a `.mcpack`**. The manifest ends up one level too deep and Minecraft rejects the pack. For a `.mcpack`, `manifest.json` must be at the very top of the archive.
- **Zipping the contents instead of the folders for a `.mcaddon`**. The opposite mistake — a `.mcaddon` *does* contain the two pack folders. Getting these two rules the wrong way round is the most common packaging error there is.
- **Using 7z or RAR format**. Minecraft only reads **zip**. The extension being `.mcaddon` does not change what is inside.
- **Not renaming the extension**, leaving a `.zip` that Windows will not hand to Minecraft.
- **The extension is really `.mcaddon.zip`** because Windows hides extensions. Turn on `View > Show > File name extensions`.
- **Leaving development junk inside**. `.bbmodel` files, `.pdn` sources, `Thumbs.db`, `.DS_Store`, `notes.txt`, unused textures. Harmless but bloats the download.
- **Shipping vanilla files**. Copies of vanilla textures, models or sounds should not be in your pack. Reference vanilla content by identifier instead.
- **Forgetting `pack_icon.png`**, so the pack shows as a blank pane and looks broken.
- **Packaging the wrong copy**. You have a source folder and a development folder, and you zipped the stale one. Verify by opening a file inside the finished archive.
- **Password-protecting or encrypting the archive** by accident in NanaZip's dialog.

15.3 Import mistakes

- **Double-clicking opens the wrong app**. Right-click → Open with → Minecraft for Windows.
- **The browser saved the file as `.zip`**. Rename it back before opening.
- **iOS unzipped the file into a folder**. Use the Files app to re-download or rename; Safari sometimes decompresses automatically.
- **Importing over an existing install with the same version number**. The game keeps the old copy. Bump the version.
- **Expecting an import to activate the packs**. Importing installs; you still have to activate them on a world.
- **Only activating the behaviour pack**. Without the resource pack, custom mobs are invisible and items show the missing-texture checker. Set up the BP→RP dependency so this cannot happen.
- **Expecting add-ons to work on a console via file import**. They cannot; use a Realm or cloud-synced world.
- **Trying to import into Minecraft: Java Edition**. Add-ons are Bedrock only. Java uses a completely different mod and datapack system.

15.4 Post-release mistakes

- **Editing a released version in place.** Bump the version and re-release instead, so players can tell which build they have.
- **No changelog,** so nobody knows whether to update.
- **Not stating the required Minecraft version,** leading to bug reports from people on old builds.
- **Not stating that an experimental toggle is required.** If your add-on needs Beta APIs or another experiment, say so in the description, in the README, and on the download page.
- **Losing the released build.** Keep the exact `.mcaddon` you published — you will need it to reproduce a bug report.
- **Never testing an actual import.** Everything working in `development_behavior_packs` proves nothing about the packaged file. Test the real `.mcaddon` on a machine (or account) that has never had the dev folders — ideally a friend's device.

15.5 Compatibility mistakes

- **Assuming your identifiers are unique.** If another add-on uses the same namespace, the two will collide. A distinctive namespace is your only protection.
- **Overwriting vanilla files.** A file in `entities/zombie.json` in your BP replaces the vanilla zombie for every player using your pack, which will break other add-ons. Only do this deliberately, and say so in your description.
- **Assuming pack order does not matter.** When two resource packs provide the same file, the one higher in the list wins. Your pack may be under someone else's.
- **Using experimental features without warning,** then being surprised when a game update removes or changes them.
- **Not re-testing after a Minecraft update.** Formats and APIs do change. Load your add-on after each major game version and check the content log.

You now have the complete path from nothing to a finished, distributable Bedrock add-on. It began with choosing tools that show you the files rather than hiding them — Blockbench for models, textures and animation, Visual Studio Code with Blockception's schemas for the JSON, bridge. v2 for scaffolding and one-click packaging, Paint.NET for pixel work, and Minecraft itself with the content log turned on as the only debugger that matters. From there you set up two pack skeletons with unique UUIDs, proved the empty shell loads, and only then started making content. You built mob geometry as a tree of bones with pivots at the joints, painted textures at the right sizes with the right transparency and material settings, drew 16×16 item icons that read at real scale, and made block textures that tile without a visible seam. You learnt that the folder a file sits in is part of the code, that `entity/` and `entities/` are different places, and that an identifier which does not match in both packs is the cause of most invisible mobs.

The Coding section is where the add-on became real. Bedrock's data-driven model is four ideas repeated everywhere: a **description** that identifies a thing, **components** that are always true, **component groups** that can be swapped in and out, and **events** that do the swapping. Filters decide when things happen, Molang computes values at runtime, and animation controllers act as state machines on both the visual and behaviour sides. Around that core sit the specialised files — spawn rules that decide where a mob appears, loot tables that decide what it drops, recipes, trade tables, particles, and functions full of commands. When JSON runs out of expressive power, the Script API takes over with real event handlers and per-tick logic. Finally you wrote proper manifests, linked the behaviour pack to the resource pack with a dependency so players cannot activate half your add-on, zipped the packs with the manifest at exactly the right depth, and turned the result into a `.mcaddon` that installs with a double-click.

If you take one habit away, make it this: **test after every small change, with the content log on**. Bedrock rarely tells you that something is wrong unless you ask it to, and the difference between an evening of progress and an evening of frustration is almost always how many changes you made between tests. The second habit is to read the vanilla packs. Every technique in this guide is used somewhere in `bedrock-samples`, written by the people who built the engine, and when the documentation is thin, the vanilla wolf, villager or lantern will show you exactly how it is meant to be done. The third is to keep your scope small enough to finish — one mob, one item, one block, working end to end, teaches you more than a planned ten-mob pack that never loads. Everything after that first complete add-on is repetition of patterns you already understand, and the ambitious version you actually want to build stops being an aspiration and becomes a matter of time.

17 Extra section: ideas

A pile of add-on ideas, sorted by what they would teach you. Each one is a real project you could finish, with a note on which systems it exercises.

17.1 Beginner ideas — one system each

- **A single new ore.** A block that generates underground, drops a gem, and crafts into a tool set. *Teaches:* blocks, loot tables, recipes, feature rules, item components.
- **A passive farm animal.** A new breedable, tempt-able, milkable-equivalent creature. *Teaches:* entity components, breeding, ageable, loot tables, spawn rules.
- **A food item with an effect.** Eat it, gain night vision for thirty seconds. *Teaches:* item components, `minecraft:food`, effects.
- **A decorative block set.** Five variants of a stone type with matching slabs and stairs. *Teaches:* blocks, permutations, geometry, recipes — repetitive on purpose.
- **A recoloured vanilla mob.** Point a new entity at `geometry.creeper` with your own texture and different stats. *Teaches:* the BP/RP link with almost no art work.
- **A lantern that turns on and off.** A block with a `lit` state, an interact event, and light emission on one permutation. *Teaches:* block states, permutations, events.

17.2 Intermediate ideas — several systems together

- **A hostile mob with a ranged attack.** A creature that shoots a custom projectile entity with its own model and impact effect. *Teaches:* `minecraft:shooter`, projectile entities, particles, animation controllers.
- **A tameable pet.** Feed it your custom item, it becomes yours, follows you, sits when you interact, and fights for you. *Teaches:* `minecraft:tameable`, component groups, owner filters, sitting animations.
- **A custom boss.** High health, multiple phases driven by entity properties, a boss bar, summoned minions, and a unique drop. *Teaches:* damage sensors, entity properties, behaviour animation controllers, `summon_entity`, `minecraft:boss`.
- **A mount.** A rideable creature with its own saddle item, controlled movement, and a jump. *Teaches:* `minecraft:rideable`, `minecraft:input_ground_controlled`, attachables.
- **A biome-specific villager variant** with a custom trade table selling your items. *Teaches:* trade tables, `minecraft:trade_table`, dialogue.
- **A magic wand.** A tool with a cooldown that fires a spell — particles, sound, and an area effect. *Teaches:* `minecraft:cooldown`, the Script API or a behaviour animation controller.
- **A custom armour set** with a full attachable model and a set bonus. *Teaches:* attachables, armour geometry, wearable components, scripts for the set bonus.

- **A growable crop.** A block with growth stages, random ticking, a seed item, and a harvest drop. *Teaches:* `minecraft:random_ticking`, *block states, permutations with geometry swaps, loot conditions.*
- **A furnace-like machine block.** A custom crafting station with its own recipe tag. *Teaches:* `minecraft:crafting_table`, *recipe tags, block interaction.*

17.3 Advanced ideas — full projects

- **A new dimension-flavoured biome set.** Custom biomes with your own trees, ores, mobs and fog. *Teaches:* *biomes, features, feature rules, fog definitions — the deepest end of Bedrock data.*
- **A dungeon add-on.** Custom structures placed by feature rules, filled with custom chests via loot tables, guarded by custom mobs, with a unique boss and reward. *Teaches:* *structures, feature rules, loot tables, spawning, boss design.*
- **A tech/automation pack.** Machines that move items, using scripts, dynamic properties and block entities. *Teaches:* *the Script API in depth.*
- **An RPG class system.** Custom UI forms, persistent player data, abilities on cooldowns, levelling. *Teaches:* `@minecraft/server-ui`, *dynamic properties, scripting architecture.*
- **A survival overhaul.** Rebalanced vanilla mobs, new tool tiers, harder nights, temperature mechanics. *Teaches:* *overriding vanilla entities safely, tick functions, scripts.*
- **A minigame world.** A world template with scripted game logic, scoreboards, a lobby, and custom UI. *Teaches:* *world templates, functions, scripting, `.mctemplate` packaging.*
- **A creature pack.** Twenty mobs sharing one skeleton and one animation set, differentiated by texture, scale and behaviour. *Teaches:* *efficient asset reuse, render controller arrays, entity properties.*

17.4 Themed ideas — for when you want a subject, not a system

- **Deep sea:** anglerfish that light up in the dark, pressure-crushing depths, submarine blocks, bioluminescent coral.
- **Sky islands:** floating structures, gliding creatures, wind currents that push the player, cloud blocks.
- **Clockwork:** brass machinery blocks, wind-up automatons that follow a path, gears with rotation states.
- **Fungal:** a spreading mushroom block that random-ticks outward, spore particles, mycelium creatures.
- **Weather:** a storm elemental that only spawns during thunderstorms, lightning-charged items, a barometer block.
- **Frost:** the running example in this guide — wraiths, frost shards, lanterns, ice that spreads.
- **Ruins:** crumbling structures with rare loot, guardian statues that animate when approached, ancient tools with lore text.
- **Beekeeping / husbandry:** more animals, more products, more crafting chains — depth rather than novelty.
- **Miniatures:** a shrink potion, tiny variants of vanilla mobs via `minecraft:scale`, rideable insects.
- **Haunted:** ghosts with `entity_alphablend`, phase-through walls behaviour, a lantern that reveals them.

17.5 Ideas for practising specific skills

To practise	Build
Geometry and bones	A mob with a long segmented tail animated with a phase-offset <code>math.sin</code> per segment
Molang	An animation with no keyframes at all, driven entirely by expressions
Render controllers	One mob with six texture variants chosen randomly at spawn
Component groups	A mob with four distinct behaviour states it moves between
Block permutations	A pipe block that connects visually to its neighbours
Loot tables	A chest loot table with a rare item and a proper weight distribution
Particles	A weather-like ambient particle that only appears in one biome
Scripting	A block that stores and displays a count using dynamic properties
Packaging	A pack with subpacks offering two texture resolutions

17.6 Choosing what to make

Three questions that pick a good project:

1. **Can you describe it in one sentence?** If not, it is two projects.
2. **What is the one new thing you will learn?** Pick a project with exactly one unfamiliar system in it. Two unfamiliar systems at once means you cannot tell which one is broken.
3. **Would you play it?** Add-ons take longer than you expect. Enthusiasm is the resource that runs out first.

Start with the smallest version that is still recognisably the idea. Ship it. Then make version two.